

**Diseño e Implementación de un Agente en Espacio de  
Usuario**  
**para la Gestión Dinámica de Frecuencia (DVFS) en  
Sistemas**  
**Heterogéneos mediante Modelos Ligeros de Machine  
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

**Director**

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

# Agradecimientos

# Resumen

**Título:** Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

**Autores:** Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

**Palabras Clave:** DVFS; eficiencia energética; computación de alto rendimiento; sistemas heterogéneos CPU–GPU; producto energía–retardo; telemetría microarquitectónica; aprendizaje automático supervisado.

## Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que, antes de despachar cada operación de una aplicación científica, decide conjuntamente el dispositivo de ejecución y su estado de frecuencia, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimien-

to ni en una sobrecarga de inferencia que anule el beneficio energético. Que la decisión sea conjunta y no únicamente de frecuencia responde a una restricción física medida sobre la plataforma: en la CPU del nodo experimental, reducir el reloj en un factor de cuatro disminuye la potencia solo un 28 %, de modo que alargar la ejecución rara vez compensa; en el acelerador, en cambio, el escalado de frecuencia sí abre un margen energético apreciable. En un sistema heterogéneo la pregunta por la frecuencia carece de sentido sin resolver antes en qué dispositivo se ejecuta, y el DVFS es el mecanismo de actuación que hace que esa elección tenga consecuencia energética.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; construcción del conjunto de decisión y selección de un modelo supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental de la primera fase, y de la segunda, el catálogo dual — en el que cada configuración de cómputo existe en una variante de CPU y una de GPU funcionalmente equivalentes, lo que permite comparar ambos dispositivos sobre la misma unidad de decisión —, la campaña de adquisición completa de su eje de CPU, y el procedimiento de construcción del conjunto de decisión, entrenamiento y validación anidada, implementado y auditado de extremo a extremo.

Su aporte metodológico central es un procedimiento reproducible para atribuir tiempo y energía al costo real de despacho de una operación — separando el costo que se paga una sola vez por proceso del que se paga en cada llamada —, con trazabilidad explícita de la calidad de cada observación y verificación directa de cada mecanismo del instrumento contra el estado real del hardware. A ese procedimiento se añaden dos salvaguardas metodológicas que condicionan cómo deben leerse sus resultados: una regla simétrica de frontera energética, que impide atribuir a una carga de CPU el consumo que el propio instrumento provoca en el acelerador ocioso, y una compuerta explícita sobre la distribución de la variable objetivo, que declara cuándo una comparación entre modelos es interpretable y cuándo constituye únicamente una validación del procedimiento. La comparación de modelos que responde a la pregunta de investigación requiere el eje del acelerador, cuya adquisición se encuentra en curso al cierre de este documento.

# Abstract

**Title:** Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

**Authors:** Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

**Keywords:** DVFS; energy efficiency; high performance computing; heterogeneous CPU–GPU systems; energy–delay product; microarchitectural telemetry; supervised machine learning.

## **Description:**

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that, before dispatching each operation of a scientific application, jointly decides the execution device and its frequency state, aiming to optimise the Energy–Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit. Making the decision joint rather than frequency-only follows from a physical constraint measured on the platform: on the CPU of the experimental node, reducing the

clock by a factor of four lowers power draw by only 28 %, so stretching execution seldom pays off; on the accelerator, by contrast, frequency scaling does open an appreciable energy margin. In a heterogeneous system the question of which frequency to use is meaningless without first resolving which device executes the work, and DVFS is the actuation mechanism that gives that choice its energy consequence.

The work is organised in four phases: workload characterisation and dataset construction; construction of the decision dataset and selection of a lightweight supervised model; implementation of the control service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol of the first phase, and from the second, the dual catalogue — in which every compute configuration exists as a functionally equivalent CPU variant and GPU variant, allowing both devices to be compared over the same decision unit —, the complete acquisition campaign of its CPU axis, and the procedure for building the decision dataset, training and nested validation, implemented and audited end to end.

Its central methodological contribution is a reproducible procedure for attributing time and energy to the true dispatch cost of an operation — separating the cost paid once per process from the cost paid on every call — with explicit traceability of the quality of each observation and direct verification of every instrument mechanism against the real state of the hardware. Two further safeguards condition how its results must be read: a symmetric energy-frontier rule, which prevents attributing to a CPU workload the consumption that the instrument itself induces on the idle accelerator, and an explicit gate on the distribution of the target variable, which declares when a comparison between models is interpretable and when it constitutes only a validation of the procedure. The model comparison that answers the research question requires the accelerator axis, whose acquisition is in progress at the time of writing.

# Índice general

|                                                                                                              |           |
|--------------------------------------------------------------------------------------------------------------|-----------|
| <b>Agradecimientos</b>                                                                                       | <b>1</b>  |
| <b>Resumen</b>                                                                                               | <b>2</b>  |
| <b>Abstract</b>                                                                                              | <b>4</b>  |
| <b>Introducción</b>                                                                                          | <b>11</b> |
| Planteamiento y justificación del problema . . . . .                                                         | 13        |
| Objetivos . . . . .                                                                                          | 15        |
| <b>1. Marco de referencia</b>                                                                                | <b>16</b> |
| 1.1. Marco Conceptual . . . . .                                                                              | 16        |
| 1.1.1. Termodinámica del silicio y potencia CMOS . . . . .                                                   | 16        |
| 1.1.2. Modelo Roofline y regímenes de limitación del rendimiento . . . . .                                   | 17        |
| 1.1.3. Telemetría microarquitectónica . . . . .                                                              | 19        |
| 1.1.4. Espacios de ejecución: <i>user-space</i> y <i>kernel-space</i> . . . . .                              | 22        |
| 1.1.5. Interfaces estándar de observabilidad e instrumentación . . . . .                                     | 23        |
| 1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states<br>y <i>governors</i> . . . . . | 25        |
| 1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks . . . . .                                    | 27        |
| 1.1.8. Aprendizaje automático supervisado ligero . . . . .                                                   | 27        |
| 1.1.9. Hiperparámetros y su optimización . . . . .                                                           | 29        |
| 1.1.10. Validación anidada y fuga de información . . . . .                                                   | 29        |
| 1.1.11. Producto Energía–Retardo (EDP) . . . . .                                                             | 30        |
| 1.1.12. Selección de dispositivo en sistemas heterogéneos . . . . .                                          | 31        |
| 1.1.13. Costo de despacho y su amortización . . . . .                                                        | 31        |
| 1.2. Marco Legal . . . . .                                                                                   | 32        |
| 1.3. Estado del Arte . . . . .                                                                               | 32        |

|                                                                                                                  |           |
|------------------------------------------------------------------------------------------------------------------|-----------|
| <b>2. Metodología</b>                                                                                            | <b>36</b> |
| 2.1. Enfoque metodológico . . . . .                                                                              | 36        |
| 2.2. Diseño metodológico . . . . .                                                                               | 37        |
| 2.2.1. Población y muestra . . . . .                                                                             | 37        |
| 2.2.2. Plataforma experimental . . . . .                                                                         | 38        |
| 2.2.3. Instrumentos de recolección . . . . .                                                                     | 39        |
| 2.2.4. Overhead del propio instrumento de medición . . . . .                                                     | 40        |
| 2.2.5. Contrato temporal de despacho: regiones fría y caliente . . . . .                                         | 41        |
| 2.2.6. Variables observadas . . . . .                                                                            | 42        |
| 2.3. Fase 1: Recolección de información y caracterización de cargas . . . . .                                    | 44        |
| 2.3.1. Verificación previa de la plataforma . . . . .                                                            | 44        |
| 2.3.2. Catálogo de cargas de trabajo . . . . .                                                                   | 44        |
| 2.3.3. Calibración y criterio de etiquetado . . . . .                                                            | 46        |
| 2.3.4. Caracterización de intensidad operacional y calibración Roofline en GPU                                   | 47        |
| 2.3.5. Dimensionamiento de las cargas GPU y verificación de invarianza de<br>la intensidad operacional . . . . . | 50        |
| 2.3.6. Matriz experimental y control de sesgos . . . . .                                                         | 51        |
| 2.3.7. Post-procesamiento y control de calidad de los datos . . . . .                                            | 55        |
| 2.3.8. Medición directa de FLOPs por ventana y su validación empírica . .                                        | 58        |
| 2.3.9. Reproducibilidad . . . . .                                                                                | 61        |
| 2.4. Fase 2: Construcción del conjunto de datos y del modelo de selección . . . .                                | 62        |
| 2.4.1. Unidad de decisión y su correspondencia con los objetivos . . . . .                                       | 62        |
| 2.4.2. Variable objetivo: la configuración de mínimo EDP . . . . .                                               | 63        |
| 2.4.3. Campañas de adquisición del catálogo dual . . . . .                                                       | 64        |
| 2.4.4. Frontera de medición de la energía . . . . .                                                              | 68        |
| 2.4.5. Estructura del conjunto de datos en dos niveles . . . . .                                                 | 69        |
| 2.4.6. Las dos estrategias de decisión . . . . .                                                                 | 70        |
| 2.4.7. Características de entrada y prevención de fuga . . . . .                                                 | 72        |
| 2.4.8. Protocolo de validación . . . . .                                                                         | 73        |
| 2.4.9. Compuerta de validez sobre la distribución de la etiqueta . . . . .                                       | 75        |
| 2.4.10. Alcance del eje de CPU en aislamiento . . . . .                                                          | 76        |
| 2.4.11. Reproducibilidad del procedimiento de modelado . . . . .                                                 | 77        |
| 2.5. Fase 3: Implementación del agente de selección . . . . .                                                    | 77        |
| 2.6. Fase 4: Validación experimental y análisis de resultados . . . . .                                          | 79        |
| 2.7. Aspectos éticos . . . . .                                                                                   | 80        |



|                                                                                       |            |
|---------------------------------------------------------------------------------------|------------|
| <b>3. Resultados</b>                                                                  | <b>81</b>  |
| 3.1. Campaña del catálogo dual sobre el eje de CPU . . . . .                          | 81         |
| 3.2. Efecto del observador en la medición energética del acelerador ocioso . . . .    | 82         |
| 3.3. Distribución de la configuración óptima en el eje de CPU . . . . .               | 83         |
| 3.4. El catálogo dual completo y el costo de arranque de CUDA . . . . .               | 83         |
| 3.4.1. La región caliente: dónde vive la ventaja real de la GPU . . . . .             | 85         |
| 3.5. Sensibilidad de la etiqueta a la resolución temporal de la región fría . . . . . | 86         |
| 3.6. Verificación de extremo a extremo del procedimiento de modelado . . . . .        | 87         |
| 3.7. Reanálisis por tamaño y amortización (Fase R1) . . . . .                         | 88         |
| 3.7.1. El mapa de amortización $K_{\text{break\_even}}$ . . . . .                     | 88         |
| 3.7.2. La decisión de dispositivo y su separación de la frecuencia . . . . .          | 90         |
| 3.7.3. El <i>headroom</i> de DVFS por estado . . . . .                                | 90         |
| 3.7.4. El piso de ruido y su descomposición regional . . . . .                        | 91         |
| 3.7.5. El resultado de baselines y su alcance . . . . .                               | 92         |
| 3.7.6. El <i>gap</i> estado $\times K$ . . . . .                                      | 93         |
| 3.7.7. El comportamiento de movimiento de datos . . . . .                             | 94         |
| <b>4. Discusión</b>                                                                   | <b>95</b>  |
| 4.1. Sobre la unidad de decisión y la variable objetivo . . . . .                     | 95         |
| <b>5. Conclusiones</b>                                                                | <b>97</b>  |
| 5.1. Limitaciones . . . . .                                                           | 98         |
| 5.2. Trabajo Futuro . . . . .                                                         | 99         |
| <b>Repositorio y recursos complementarios</b>                                         | <b>102</b> |

# Índice de tablas

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 2.1. Características de la plataforma experimental . . . . .               | 39 |
| 2.2. Señales de telemetría de CPU . . . . .                                | 43 |
| 2.3. Composición del catálogo experimental . . . . .                       | 46 |
| 2.4. Estados de frecuencia (Fase 1) . . . . .                              | 52 |
| 2.5. Rejilla fina de frecuencia . . . . .                                  | 52 |
| 2.6. Taxonomía de clasificación de frecuencia . . . . .                    | 54 |
| 2.7. Taxonomía de anotaciones de calidad . . . . .                         | 58 |
| 2.8. FLOPs por instrucción retirada . . . . .                              | 59 |
| 2.9. Matriz experimental del catálogo dual . . . . .                       | 66 |
| 2.10. Fórmulas analíticas de descriptores estáticos . . . . .              | 71 |
| 2.11. Espacios de hiperparámetros por familia . . . . .                    | 74 |
| 3.1. EDP en región caliente, CPU vs. GPU . . . . .                         | 85 |
| 3.2. Mapa de amortización $K_{\text{break\_even}}$ por operación . . . . . | 89 |
| 3.3. Coincidencia dispositivo REF vs. oráculo, por estado . . . . .        | 90 |
| 3.4. Headroom de frecuencia sobre REF, por estado . . . . .                | 91 |
| 3.5. Piso de ruido de medición, global y por región . . . . .              | 91 |
| 3.6. Mejor baseline en <code>gpu_ready</code> , por régimen . . . . .      | 92 |

# Índice de figuras

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1.1. Modelo Roofline conceptual . . . . .                         | 17 |
| 1.2. Contadores de núcleo y contadores de <i>uncore</i> . . . . . | 21 |
| 1.3. Espacios de usuario y de kernel . . . . .                    | 23 |

# Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como `ondemand` o `schedutil`, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente

en espacio de usuario basado en modelos ligeros de aprendizaje automático que, antes de despachar cada fase de una aplicación científica, decide el punto operativo bajo el cual debe ejecutarse. Esa decisión es conjunta sobre dos variables: el dispositivo que ejecuta la fase — CPU o acelerador — y el estado de frecuencia aplicado a ese dispositivo. Que ambas variables se resuelvan juntas y no por separado no es una ampliación de conveniencia sino una consecuencia de la física del nodo: como se documenta en el capítulo de resultados, en la CPU de la plataforma experimental una reducción del reloj en un factor de cuatro rebaja la potencia únicamente en un 28 %, mientras el tiempo de ejecución se estira entre 2.21 y 4.05 veces, de modo que el escalado de frecuencia por sí solo casi nunca mejora el Producto Energía–Retardo (EDP); en el acelerador, en cambio, ese mismo escalado sí produce ahorros apreciables. En un sistema heterogéneo, por tanto, preguntarse a qué frecuencia ejecutar sin haber resuelto antes dónde ejecutar deja fuera el grado de libertad que concentra el margen energético disponible. El DVFS sigue siendo el mecanismo de actuación del agente; la elección de dispositivo es lo que le da consecuencia energética en una arquitectura heterogénea.

La unidad sobre la que se toma esa decisión es una llamada a una operación — una fase de una aplicación HPC mayor — y no un intervalo temporal arbitrario dentro de ella. Esta precisión importa porque delimita todo el diseño experimental que sigue: el conjunto de datos no se construye sobre ventanas de muestreo tratadas como observaciones independientes, sino sobre configuraciones de cómputo completas, cada una medida en ambos dispositivos y en toda la rejilla de frecuencias disponible.

Ahora bien, antes de que un modelo de aprendizaje automático pueda decidir sobre el punto operativo, es necesario disponer de un conjunto de datos que asocie cada configuración de cómputo con el costo energético y temporal que efectivamente produce en cada dispositivo y a cada frecuencia. Construir ese conjunto no es un paso preparatorio trivial: exige resolver de forma explícita cómo se atribuye la telemetría al proceso medido, con qué resolución temporal se observa, qué parte del costo de una ejecución corresponde realmente al despacho de la operación y cuál a la inicialización que una aplicación real pagaría una sola vez, y qué observaciones deben descartarse por no ser representativas. Estas decisiones condicionan la validez de todo lo que se construya después, razón por la cual el presente documento dedica una parte sustancial de su desarrollo metodológico a la construcción y verificación del instrumento de medición.

## Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [34] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [6].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3, 5].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [1], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo

capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [34, 6, 3, 5, 1]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A esta dificultad se añade una restricción que el presente trabajo midió directamente sobre su plataforma experimental y que condiciona el alcance de cualquier política basada únicamente en frecuencia. La viabilidad del DVFS no depende solo de que el mecanismo exista y funcione, sino del rango dinámico de potencia del dispositivo: si al reducir el reloj la potencia disipada baja poco mientras el tiempo de ejecución se alarga mucho, el producto energía-retardo empeora aunque la decisión de frecuencia sea la correcta según el régimen de la carga. Ese es el caso observado en la CPU del nodo experimental, donde una reducción del reloj en un factor de cuatro rebaja la potencia apenas un 28 %; y no lo es en el acelerador, donde el mismo mecanismo sí produce ahorros medibles. La consecuencia es que, en un nodo heterogéneo, el margen energético no está distribuido de forma simétrica entre los dispositivos, y una política que solo module la frecuencia renuncia al grado de libertad que concentra ese margen: la elección de dónde ejecutar cada fase de la aplicación.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

**¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?**

# Objetivos

## Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS) en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

## Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.



# Capítulo 1

## Marco de referencia

### 1.1. Marco Conceptual

El ajuste dinámico de frecuencia y voltaje para la optimización energética se fundamenta en la relación no lineal entre los parámetros de operación del hardware y la disipación de calor. A continuación, se definen los principios termodinámicos, arquitectónicos y de aprendizaje computacional que rigen el diseño de este agente de control en espacio de usuario, así como los conceptos de instrumentación y medición que sustentan la construcción del conjunto de datos experimental.

#### 1.1.1. Termodinámica del silicio y potencia CMOS

La disipación de potencia en un circuito CMOS moderno se descompone en potencia estática (corrientes de fuga) y potencia dinámica, originada por la conmutación de cargas capacitivas. Como DVFS actúa sobre frecuencia y voltaje, su efecto principal recae sobre la potencia dinámica, modelable para una compuerta como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde  $\alpha$  es el factor de actividad,  $C$  la capacitancia efectiva conmutada y  $V$ ,  $f$  el voltaje y la frecuencia de operación [9]. Dado que operar a mayor frecuencia suele exigir mayor voltaje por integridad temporal, reducir frecuencia habitualmente permite también reducir voltaje, con una caída de potencia más que proporcional por la dependencia cuadrática en  $V$ . Este acoplamiento es la base física del compromiso energía–rendimiento en DVFS: cuánto se gana en potencia al bajar el punto operativo depende del comportamiento computacional

predominante de la aplicación, no solo de la reducción de frecuencia en sí.

### 1.1.2. Modelo Roofline y regímenes de limitación del rendimiento

El modelo Roofline analiza el rendimiento de una aplicación en función de su intensidad operacional y de los límites físicos del hardware:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde  $P_{\text{pico}}$  es el rendimiento aritmético pico,  $I$  la intensidad operacional (operaciones por byte transferido desde memoria) y  $BW_{\text{pico}}$  el ancho de banda pico [41]. Si el término activo es  $I \cdot BW_{\text{pico}}$  el kernel es *memory-bound*; si es  $P_{\text{pico}}$ , es *compute-bound*.

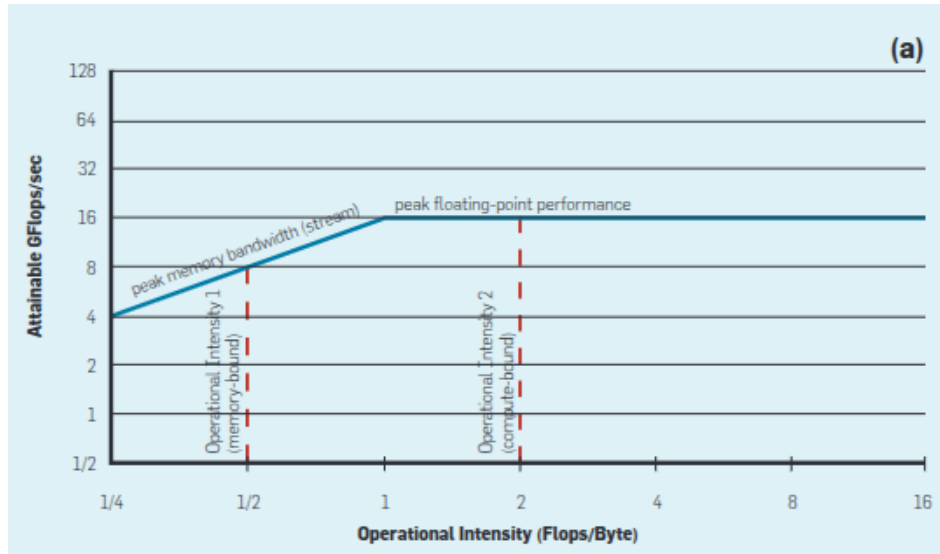


Figura 1.1: Modelo Roofline conceptual: relación entre el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Fuente: tomada de Williams et al. [41, Fig. 1(a)].

La intensidad operacional  $I$  relaciona el trabajo aritmético con los bytes transferidos desde el nivel de memoria modelado, no con el tamaño nominal del conjunto de trabajo [41, p. 66]. Por ello, que los datos excedan la caché de último nivel solo indica que parte de ellos deberá atravesar niveles inferiores de la jerarquía, pero no determina cuánto tráfico se genera ni cuánta reutilización ocurre. La multiplicación de matrices densas ilustra esta diferencia: mediante bloqueo o mosaico (*tiling*), los bloques pueden reutilizarse y su movimiento entre niveles de memoria se amortiza, sin requerir que las matrices completas residan simultáneamente en caché [10, Secs. 3–4]. Para una cantidad fija de operaciones, esa reducción del

tráfico aumenta la intensidad operacional. El tamaño del conjunto de trabajo frente a la caché es, por tanto, un indicio útil, pero no un criterio suficiente para clasificar una carga como *memory-bound* o *compute-bound*.

### Punto de inflexión y criterio de clasificación

La frontera entre regímenes es el punto de inflexión o *ridge point*, donde ambas ramas de (1.2) se igualan:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

En este trabajo,  $I_{\text{ridge}}$  se adopta como umbral operativo de etiquetado: una intensidad observada inferior se clasifica como *memory-bound* y una superior como *compute-bound*, de acuerdo con la separación de regímenes que plantea el modelo Roofline [41]. La caracterización Roofline puede determinar techos de cómputo por separado para distintos tipos de dato y precisiones aritméticas [2]. En consecuencia, cuando esos techos difieren, el valor de  $P_{\text{pico}}$  empleado en (1.3) debe corresponder a la precisión de las operaciones medidas; cada precisión requiere entonces su propio  $I_{\text{ridge}}$ . Una carga de precisión mixta no puede compararse directamente con un único techo sin definir previamente cómo se tratará esa mezcla.

Los valores de fábrica sirven como referencia inicial, pero no representan necesariamente los techos alcanzables bajo la configuración experimental. La caracterización Roofline recomienda estimarlos mediante microkernels y condiciones de ejecución representativas [28]. Por ello, este trabajo determina empíricamente los techos de cómputo y ancho de banda con cargas diseñadas para saturarlos por separado, ejecutadas bajo la misma afinidad de núcleos que la campaña experimental. Así,  $I_{\text{ridge}}$  corresponde a la plataforma y a la configuración efectivamente evaluadas, no solo a una cifra de la ficha técnica.

### Cálculo de la intensidad operacional observada

Aplicar el criterio anterior exige calcular la intensidad sobre un mismo intervalo de observación:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde  $\text{FLOPs}_i$  representa el trabajo aritmético realizado y  $B_i^{\text{DRAM}}$  los bytes transferidos hacia o desde memoria principal durante el intervalo  $i$ . Ambos términos deben correspon-

der al mismo intervalo temporal; dividir magnitudes obtenidas con granularidades distintas produciría una intensidad sin consistencia física.

Los contadores de rendimiento no miden FLOPs como una magnitud abstracta, sino eventos definidos por una microarquitectura concreta. Incluso eventos aparentemente directos, como las instrucciones retiradas, pueden presentar diferencias de implementación y sensibilidad a las condiciones de medición [40]. Por ello, cualquier conversión desde eventos PMU hacia FLOPs requiere verificar la disponibilidad, la codificación y la semántica del evento utilizado.

El tráfico hacia memoria principal se observa desde un dominio distinto: los contadores *uncore* del controlador de memoria integrado (IMC), que permiten contabilizar comandos de lectura y escritura enviados a la DRAM [17]. Estos contadores tienen alcance de sistema y no pueden atribuirse directamente a un proceso individual. Cuando su granularidad temporal es más gruesa que la de los contadores de núcleo, los FLOPs deben agregarse sobre el mismo intervalo antes de calcular la intensidad operacional.

### 1.1.3. Telemetría microarquitónica

Los procesadores modernos incorporan Unidades de Monitorización de Rendimiento (PMU), compuestas por contadores fijos y programables que permiten observar eventos asociados al flujo de instrucciones y a la jerarquía de memoria [39, 19]. Su cantidad y sus capacidades dependen de la microarquitectura, pero el número de eventos que pueden observarse simultáneamente es limitado, lo que motiva el uso de mecanismos de multiplexación cuando la demanda supera los contadores disponibles [26].

#### Multiplexación de contadores y escalado de medidas

Cuando los eventos solicitados exceden los contadores físicos disponibles, el kernel puede alternarlos periódicamente sobre ellos. En ese caso, cada lectura se escala mediante la relación entre el tiempo durante el cual el evento estuvo habilitado y el tiempo durante el cual contó efectivamente. El resultado es una extrapolación que supone una tasa de eventos aproximadamente estable durante los intervalos no observados, supuesto que pierde validez cuando el comportamiento cambia dentro de la ventana [26].

Para evitar esta fuente de incertidumbre, el conjunto de eventos empleado en este trabajo se dimensionó dentro del presupuesto de contadores físicos simultáneos de la plataforma y se comprobó durante la calificación del instrumento. Los tiempos de habilitación y conteo conservados por el instrumento proporcionan, además, una señal de control sobre el funcio-

namiento de la medición de referencia.

### Eventos genéricos y eventos crudos

Los eventos genéricos son una abstracción portable que el kernel traduce a la codificación concreta de la microarquitectura; los eventos crudos se especifican mediante la codificación nativa del procesador (selector de evento, máscara de subevento y calificadores adicionales) [39, 19]. Cuando el kernel carece de una traducción para un evento disponible en el procesador, puede ser necesario usar su codificación cruda, lo que traslada al instrumento la responsabilidad de verificar que esa codificación corresponde a la microarquitectura evaluada. Una codificación incorrecta puede producir valores sin significado físico aun cuando la apertura del contador no genere un error explícito.

### Contadores de núcleo y contadores de *uncore*

Los contadores de núcleo observan eventos atribuibles a un núcleo concreto y pueden asociarse a un proceso. Los contadores de *uncore* observan recursos compartidos del zócalo, como el controlador de memoria, la caché de último nivel distribuida y las interconexiones, y por tanto son de alcance de sistema, no atribuibles a un proceso específico. Los contadores del controlador de memoria contabilizan el tráfico que alcanza la memoria principal, tanto por accesos de demanda como por precarga [17]. Por ello, constituyen la fuente empleada en este trabajo para el denominador de la ecuación (1.4) (sección 1.1.2), aunque su acceso está sujeto a una categoría de privilegio distinta (sección 1.1.5).

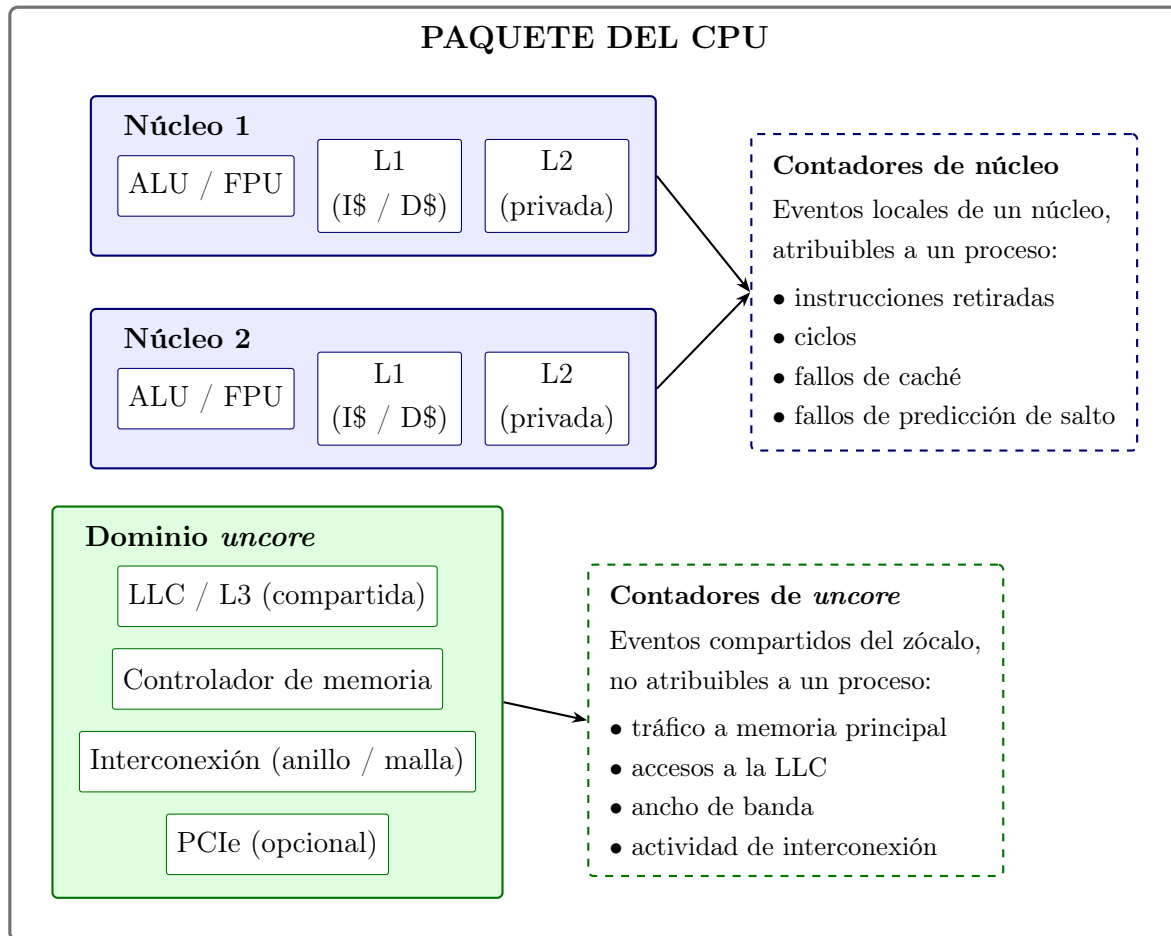


Figura 1.2: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Los contadores de núcleo observan eventos locales de un núcleo concreto y pueden asociarse a un proceso; los de *uncore* observan recursos compartidos del zócalo y son de alcance de sistema. Elaboración propia.

### Métricas microarquitectónicas de eficiencia

Las instrucciones por ciclo (IPC), la tasa de fallos de caché y la fracción de ciclos de estancamiento describen aspectos complementarios del comportamiento de la ejecución. El IPC relaciona las instrucciones retiradas con los ciclos transcurridos [11, Ch. 6], mientras que los eventos de caché permiten observar la frecuencia relativa de accesos que no son atendidos en el nivel representado por el evento configurado [11, Ch. 2].

Los ciclos de estancamiento contabilizan intervalos en los que la ejecución no progresa. Esta señal puede aumentar por dependencias de datos, latencias de memoria u otras restricciones del procesador, por lo que no identifica por sí sola la causa del estancamiento. En este trabajo se utiliza como una característica complementaria del clasificador, no como sustituto

de la intensidad operacional ni como criterio directo para asignar la etiqueta *memory-bound*. La clasificación se determina mediante la intensidad operacional observada y el punto de inflexión correspondiente.

## Ventanas de muestreo y naturaleza temporal de la observación

En modo de conteo, cada lectura representa el valor acumulado desde la habilitación o el último reinicio del contador [23]. Por ello, la unidad de análisis no es una lectura individual, sino la diferencia entre dos lecturas consecutivas, teniendo en cuenta posibles desbordamientos [19]. En contraste, el modo de reporte periódico de `perf stat` entrega deltas por intervalo, que no deben volver a diferenciarse [24].

La elección del período de muestreo establece un compromiso. Un período corto ofrece mayor resolución temporal, pero exige observaciones más frecuentes; en herramientas de reporte intervalado como `perf stat`, esta frecuencia puede elevar la sobrecarga de medición [24]. Un período largo acumula más eventos, pero puede integrar en una sola observación cambios ocurridos dentro del intervalo. Por ello, las tasas derivadas deben calcularse sobre el tiempo efectivamente transcurrido entre lecturas, empleando una base temporal que no experimente saltos discontinuos, como `CLOCK_MONOTONIC` [22].

### 1.1.4. Espacios de ejecución: *user-space* y *kernel-space*

Los sistemas operativos modernos separan la ejecución en dominios de privilegio diferenciados, apoyados en modos de operación del procesador. En modo usuario, el software solo accede a memoria de espacio de usuario; en modo kernel, puede acceder también al espacio del kernel [20, ch. 2]. Operaciones sensibles, como la gestión de memoria, el acceso a hardware y la entrada/salida, requieren privilegios de kernel, lo que impide que procesos ordinarios interfieran con estructuras críticas [20, ch. 2]. Por eso el software de aplicación no accede a recursos privilegiados de forma directa, sino mediante mecanismos de mediación: en sistemas Unix y Linux, principalmente llamadas al sistema [27]. Cualquier mecanismo de monitoreo o control implementado fuera del kernel debe apoyarse, en consecuencia, en interfaces explícitamente expuestas por la plataforma.

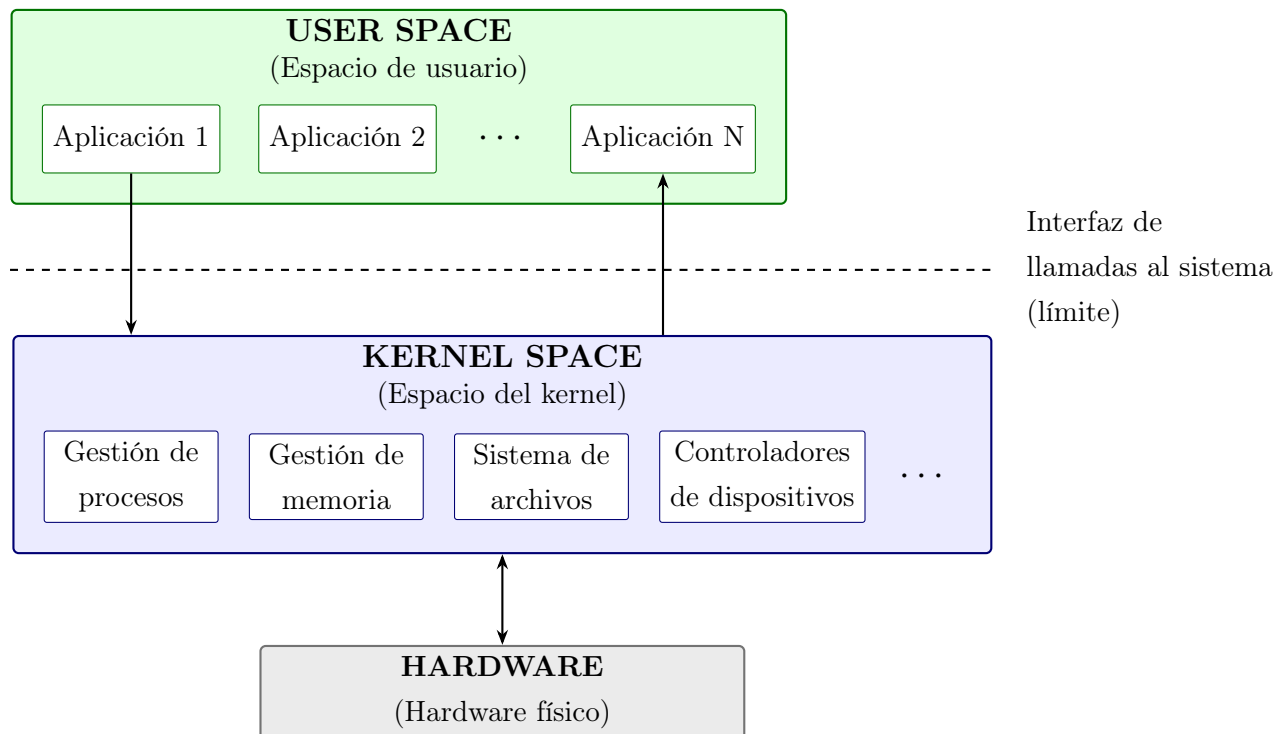


Figura 1.3: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico, mediada por la interfaz de llamadas al sistema. Elaboración propia.

### 1.1.5. Interfaces estándar de observabilidad e instrumentación

La observabilidad de sistemas heterogéneos CPU–GPU depende de interfaces que expongan, desde *user-space*, métricas de rendimiento, utilización y consumo sin modificar el kernel ni los controladores. Las plataformas modernas exponen `perf_event` para contadores de CPU, Intel RAPL para energía de CPU, y NVIDIA Management Library (NVML) para telemetría de GPU [5, 39, 7, 29].

#### La interfaz `perf_event` y los modos de atribución

`perf_event` es la infraestructura estándar en Linux para acceder a la PMU desde *user-space* [39]. El alcance de atribución de la medida es determinante para la validez experimental: la medición de alcance de CPU contabiliza todo lo que ocurre en un procesador dado, incluida actividad ajena al experimento; la medición por proceso contabiliza únicamente ese proceso, con independencia de sobre qué CPU se planifique. Cuando la carga es paralela, la interfaz permite heredar el contador a los descendientes creados tras habilitarlo, de modo que la medida abarque el árbol completo de ejecución y solo ese árbol: la combinación de atribución



por proceso con herencia adoptada en este trabajo. Con herencia habilitada, sin embargo, la interfaz no admite lectura agrupada de varios contadores en una operación; cada evento se lee de forma independiente, lo que introduce un desfase de orden microsegundo entre lecturas de una misma ventana, despreciable frente a períodos de muestreo del orden del milisegundo pero declarado aquí como característica conocida del instrumento.

## Niveles de privilegio en el acceso a contadores

El acceso a la infraestructura de monitorización desde el espacio de usuario está regulado en Linux por el parámetro `perf_event_paranoid`, que limita de forma progresiva las capacidades de los usuarios sin privilegios [23]. En sus niveles más restrictivos, este parámetro impide recolectar eventos de ámbito de sistema, categoría a la que pertenecen los contadores de *uncore* (sección 1.1.3), y solo permite la medición del propio proceso. Esto tiene una implicación metodológica directa: la disponibilidad de las métricas no depende únicamente del hardware, sino de la política de administración del sistema, lo que convierte a la caracterización previa de la plataforma en un paso obligatorio para el diseño experimental.

## Intel RAPL

RAPL (*Running Average Power Limit*) expone energía acumulada para dominios como paquete del procesador, núcleos y DRAM [7, 35]. Reporta mediante registros específicos del modelo, en unidades dependientes de la plataforma, sin cobertura exhaustiva del consumo total del nodo ni disponibilidad uniforme entre microarquitecturas [35]. Al ser acumulativos, los registros pueden desbordarse, lo que exige que el instrumento registre el valor máximo del rango antes de capturar, para distinguir un desbordamiento real de un consumo negativo espurio. RAPL es una capacidad del procesador, no una interfaz universal: microarquitecturas anteriores a su introducción carecen de ella sin alternativa por software, por lo que su disponibilidad es criterio de admisibilidad para cualquier plataforma de este tipo de estudio.

## NVIDIA Management Library

NVML permite consultar el estado operativo de una GPU NVIDIA (utilización, potencia, memoria y otros parámetros del dispositivo) [29], orientada a la telemetría del acelerador como dispositivo completo y no a eventos microarquitectónicos de un kernel concreto [5, 29]. Esta diferencia de granularidad es relevante para el problema abordado: la utilización que NVML reporta indica qué fracción del tiempo hubo trabajo activo, no cuántas operaciones por byte realizó ese trabajo, así que caracterizar el régimen de ejecución en GPU exige una estrategia distinta a la de CPU (sección 2.3.4).

### 1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y *governors*

DVFS ajusta el punto operativo del hardware mediante cambios coordinados de frecuencia y voltaje, apoyado en reguladores de voltaje y unidades de control de potencia dedicadas que ejecutan las transiciones [13]. El estándar ACPI formaliza esto mediante *performance states* (P-states, niveles operativos dentro del estado activo, con P0 como el de mayor rendimiento) y *C-states* (niveles de inactividad) [13, 11].

Sobre esta base operan los *governors*: políticas que deciden cuándo y cómo transicionar entre P-states, sin implementar el mecanismo físico. Las políticas de frecuencia fija (*performance*, *powersave*) sitúan al procesador en un extremo operativo; los gobernadores dinámicos (*ondemand*, *conservative*, *schedutil*) ajustan la frecuencia según carga observada o, en el caso de *schedutil*, integrada con el planificador [13]. En Linux, dos controladores dominan: *acpi-cpufreq*, genérico, que expone un gobernador *userspace* capaz de fijar una frecuencia objetivo explícita, e *intel\_pstate*, por defecto en muchas generaciones Intel, que solo soporta *performance/powersave* y no incluye gobernador *userspace* [13]. Bajo este segundo controlador, fijar un punto operativo concreto se logra restringiendo el rango permitido mediante límites inferior y superior, no escribiendo una frecuencia objetivo directamente. Esta diferencia tiene consecuencias operativas directas: un agente de control portable debe detectar el controlador activo y adaptar su estrategia de actuación en consecuencia. Adicionalmente, las transiciones entre P-states no son instantáneas: del orden de 10 ms, o 1 ms con *hardware P-states* [13], por lo que una política de control debe amortizar ese costo y no decidir a un ritmo superior al que el sistema puede materializar.

### Dominios de frecuencia y aislamiento experimental

El dominio de frecuencia (el conjunto de procesadores lógicos afectados simultáneamente por una misma solicitud de cambio) puede abarcar un zócalo completo o reducirse a cada núcleo individual, según la generación de hardware. Si el dominio excede los núcleos asignados al experimento, la actuación puede alterar procesos ajenos y viceversa, comprometiendo tanto la validez interna del experimento como la convivencia en infraestructura compartida. Por ello, el dominio real de frecuencia, consultado en la plataforma y no supuesto a partir del modelo de procesador, es una verificación previa obligatoria antes de habilitar cualquier actuación. De forma relacionada, cuando *simultaneous multithreading* comparte los recursos de un núcleo físico entre dos procesadores lógicos, la política de asignación debe declararse explícitamente y verificarse contra la topología real, para evitar que hilos de medición y de carga compitan por los mismos recursos.

Conviene distinguir aquí dos decisiones que, por involucrar ambas a *SMT*, pueden confundirse entre sí pese a operar en capas distintas del sistema. La primera es una política de *colocación* de la carga: cuando dos procesadores lógicos comparten las unidades de ejecución y la jerarquía de caché de un mismo núcleo físico, ejecutar la medición en ambos simultáneamente introduce contención entre los propios hilos medidos, distorsionando el resultado por una razón ajena a la frecuencia. La mitigación, restringir la carga a un único hilo lógico por núcleo físico y dejar el resto sin uso, resuelve exactamente ese problema, y solo ese: decide dónde se *ejecuta* la carga, no qué frecuencia puede *solicitar* cada procesador lógico del sistema.

Esa distinción importa porque dejar sin uso al segundo procesador lógico de cada núcleo no equivale a deshabilitar *SMT*, una operación de firmware fuera del alcance de un experimento sin privilegios administrativos sobre una plataforma compartida. El procesador lógico no utilizado sigue activo ante el sistema operativo, con su propia política de *cpufreq* independiente y escribible, y su gobernador queda en el extremo de mayor rendimiento por defecto salvo que se le restrinja explícitamente, exactamente como cualquier otro procesador lógico del sistema. Que nunca ejecute la carga medida no le impide seguir solicitando el punto operativo más alto que el firmware permita, y como ambos hermanos comparten el mismo reloj físico subyacente pese a exponer políticas de control nominalmente independientes, el mecanismo de coordinación de límites del hardware puede conceder ese punto sobre el núcleo físico compartido con independencia del límite, más bajo, declarado en la política del hermano restringido y efectivamente en uso. Una política de colocación correcta y ya verificada no protege, por tanto, contra esta segunda vía de interferencia: la disciplina de aislamiento experimental, declarar y verificar el dominio real de frecuencia, debe extenderse también a los hermanos de *SMT* que la política de colocación decidió dejar sin uso, no solo a los procesadores lógicos que efectivamente ejecutan la carga medida.

## Particularidades de DVFS en GPU

En GPU, DVFS no se reduce al esquema de CPU: el comportamiento energético y temporal depende de la interacción entre al menos dos dominios funcionales, procesamiento (multiprocesadores de *streaming*) y memoria del dispositivo, que no responden igual a un cambio de frecuencia [12]. Cuando el rendimiento está limitado por cómputo, la frecuencia del dominio de procesamiento impacta más directamente el tiempo de ejecución; cuando predomina la limitación por memoria, esa sensibilidad disminuye y el dominio de memoria pasa a ser más determinante [12, 1]. Además, potencia, rendimiento y energía no varían de forma independiente con la frecuencia: bajar la frecuencia puede reducir la potencia instantánea sin

reducir la energía total si el tiempo de ejecución aumenta demasiado, y viceversa [12, 1].

### 1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks

En HPC, un kernel es una rutina computacional bien delimitada que concentra una fracción relevante del trabajo de una aplicación — una multiplicación de matrices, una transformada de Fourier, una actualización de *stencil* [32]. Al encapsular un patrón dominante de ejecución más estable que la aplicación completa, el kernel es una unidad de observación especialmente adecuada para analizar regímenes *compute-bound* y *memory-bound*.

Un benchmark es una carga diseñada o seleccionada para medir y comparar el comportamiento de un sistema bajo condiciones reproducibles, facilitando comparación entre arquitecturas o políticas de control [11, Ch. 12]. Un microbenchmark es una forma más reducida, orientada a aislar el comportamiento de un componente o mecanismo específico — por ejemplo, ancho de banda de memoria o densidad aritmética [11, Ch. 12] —, y es el instrumento adecuado para la calibración Roofline de la sección 1.1.2, mientras que los benchmarks representativos constituyen el objeto de caracterización.

Las suites de benchmarks científicos suelen reportar, al finalizar, una medida agregada de trabajo realizado — pero esa medida no siempre corresponde a operaciones de punto flotante: algunos programas cuantifican su trabajo en unidades distintas (números aleatorios, claves ordenadas) según la naturaleza del problema. Como el numerador de la intensidad operacional (ecuación (1.4)) debe expresarse en FLOPs para ser comparable con un techo calibrado en esas unidades, verificar explícitamente la unidad reportada por cada carga es un criterio de admisión al catálogo experimental, no un detalle de implementación (sección 2.2.1).

### 1.1.8. Aprendizaje automático supervisado ligero

El aprendizaje supervisado infiere una función que mapea variables de entrada hacia una salida a partir de datos etiquetados, dividiendo el conjunto en entrenamiento y prueba para evaluar generalización [31]. En contextos con restricciones de tiempo y recursos, un clasificador ligero es aquel cuya inferencia exige baja complejidad computacional y memoria acotada, sin sacrificar capacidad razonable de generalización. Entre los clasificadores supervisados clásicos más relevantes para este problema se encuentran:

**Bosques Aleatorios (*Random Forest*).** Método de aprendizaje en conjunto que combina múltiples árboles de decisión entrenados de forma independiente, agregando sus predicciones

(votación mayoritaria en clasificación) [15]. Dos mecanismos estocásticos — *bootstrap* sobre las muestras de entrenamiento y selección aleatoria de un subconjunto de variables en cada división — garantizan baja correlación entre árboles, mitigando el sobreajuste característico de árboles individuales profundos [15].

**Regresión Logística.** Estima la probabilidad de pertenencia a una clase mediante un predictor lineal transformado por la función sigmoide:

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n, \quad P(y = 1 \mid x) = \frac{1}{1 + e^{-z}} \quad (1.5)$$

Sus ventajas son simplicidad, bajo costo de inferencia e interpretabilidad; su limitación principal es la frontera de decisión lineal, insuficiente cuando la separación entre clases depende de relaciones fuertemente no lineales [14].

**Árbol de decisión.** Modelo que particiona recursivamente el espacio de características mediante umbrales sobre una variable a la vez, hasta alcanzar hojas suficientemente puras según un criterio de impureza. Su interés en este trabajo es doble: ofrece una frontera de decisión no lineal a un costo de inferencia muy bajo — recorrer una rama es una secuencia de comparaciones escalares —, y es directamente inspeccionable, de modo que la regla aprendida puede leerse y contrastarse contra el comportamiento físico esperado. Su limitación conocida es la varianza: un árbol profundo ajustado sobre pocos ejemplos es sensible a perturbaciones menores del conjunto de entrenamiento, que es justamente lo que los métodos de ensamble mitigan.

**XGBoost.** Ensamble de árboles potenciados de forma secuencial y aditiva, donde cada nuevo árbol corrige los residuos del modelo actual, incorporando regularización directamente en el objetivo de aprendizaje para controlar la complejidad [16]. Frente a la regresión logística es más expresivo; frente a un árbol individual, más robusto — a costa de mayor complejidad estructural.

Estas cuatro familias no se presentan como alternativas equivalentes entre las que elegir por preferencia, sino como una escala ordenada de capacidad expresiva y de costo de inferencia: un piso lineal, una frontera no lineal interpretable, un ensamble por promediado y un ensamble por potenciación regularizada. Compararlas bajo condiciones idénticas permite establecer si la complejidad adicional se traduce en una mejora real de la decisión o si el problema se resuelve igual de bien en el extremo barato de esa escala, lo cual es una respuesta legítima y no un resultado fallido.

### 1.1.9. Hiperparámetros y su optimización

Los modelos anteriores no quedan determinados únicamente por los datos. Cada familia expone un conjunto de **hiperparámetros** — la fuerza de regularización de la regresión logística, la profundidad máxima de un árbol, el número de estimadores de un ensamble, la tasa de aprendizaje de la potenciación — cuyos valores no se estiman durante el ajuste sino que se fijan antes de él y condicionan qué función puede aprenderse. Dejarlos en sus valores por defecto es una decisión, no una omisión neutral: compara implementaciones bajo una configuración arbitraria en lugar de comparar familias bajo su mejor configuración alcanzable, y penaliza sistemáticamente a las familias con más grados de libertad.

La búsqueda de esos valores puede plantearse por rejilla exhaustiva, por muestreo aleatorio o mediante métodos que aprovechen la información de las evaluaciones previas. La **optimización bayesiana** pertenece a esta tercera clase: construye un modelo sustituto de la relación entre configuración y desempeño, y lo emplea para decidir qué configuración probar a continuación, concentrando el presupuesto de evaluaciones en las regiones prometedoras del espacio. Su ventaja sobre la rejilla es relevante cuando cada evaluación es costosa y el espacio tiene dimensiones continuas, ambas condiciones presentes aquí.

Cuando el criterio de calidad no es único, el problema deja de tener un óptimo escalar. En este trabajo concurren dos objetivos legítimos y en tensión: la calidad de la decisión, y el costo de emitirla, dado que la pregunta de investigación exige explícitamente que el sobre costo de la inferencia no degrade el rendimiento. Un modelo que decidiera marginalmente mejor a costa de una latencia sustancialmente mayor no sería preferible. Para este caso se emplea la **optimización multiobjetivo**, cuyo resultado no es una configuración única sino un **frente de Pareto**: el conjunto de configuraciones para las cuales no existe otra que mejore un objetivo sin empeorar el otro. Elegir un punto de ese frente es una decisión de diseño que debe declararse explícitamente, y en este trabajo se declara en la sección 2.4.8.

Los algoritmos evolutivos multiobjetivo son una vía habitual para aproximar ese frente. El empleado aquí ordena la población de configuraciones por dominancia de Pareto y, dentro de cada nivel de dominancia, favorece las soluciones situadas en regiones menos densas del frente, con lo que la aproximación resultante no se concentra en un solo extremo del compromiso entre objetivos.

### 1.1.10. Validación anidada y fuga de información

Optimizar hiperparámetros introduce un riesgo metodológico específico. Si la configuración se elige observando el mismo conjunto sobre el que después se reporta el desempeño, la

métrica resultante ya no estima la capacidad de generalización: mide cuán bien se ajustó la búsqueda a ese conjunto particular. El sesgo resultante es optimista y crece con el número de configuraciones evaluadas, de modo que una búsqueda más intensa produce una estimación más engañosa.

La **validación anidada** resuelve esa circularidad separando dos bucles. El bucle externo reserva una partición que no interviene en ninguna decisión de modelado y sobre la cual se reporta el desempeño final. Dentro de cada partición externa, un bucle interno — que solo ve los datos de entrenamiento de esa partición — ejecuta la búsqueda de hiperparámetros y selecciona una configuración. El modelo así configurado se evalúa una única vez sobre la partición externa reservada. Así, ninguna información de la partición de prueba participa en la elección de hiperparámetros.

A esa separación se añade una segunda, independiente de la anterior: la **fuga de la variable objetivo** hacia las características de entrada. Ocurre cuando una característica codifica, directa o indirectamente, información que solo estaría disponible después de conocer el resultado que se pretende predecir. En un problema de selección de configuración, el caso más evidente es incluir el tiempo o la energía medidos de la configuración candidata: el modelo alcanzaría un desempeño casi perfecto sin haber aprendido nada útil, dado que en despliegue esas cantidades son precisamente lo que no se conoce. La prevención de esta fuga no se resuelve por inspección visual del conjunto de datos, sino declarando explícitamente qué columnas están prohibidas y verificándolo de forma automática, como se describe en la sección 2.4.7.

### 1.1.11. Producto Energía–Retardo (EDP)

Evaluar una estrategia de optimización energética exige una métrica que integre energía y rendimiento simultáneamente: minimizar solo una de las dos puede resultar globalmente inconveniente (ahorro energético con degradación severa, o máximo desempeño con costo energético desproporcionado). El Producto Energía–Retardo cumple ese rol [21]:

$$EDP = E \cdot T^w \tag{1.6}$$

donde  $E$  es la energía consumida,  $T$  el tiempo de ejecución y  $w$  un factor de ponderación;  $w = 1$  da el EDP clásico,  $w = 2$  una variante que penaliza más fuertemente la degradación temporal (*Energy–Delay<sup>2</sup> Product*) [1, 21]. Su utilidad radica en evitar decisiones sesgadas por una sola dimensión: reducir únicamente la potencia puede inducir pérdidas de rendimiento

que terminen aumentando la energía total consumida, por lo que una función multiobjetivo basada en EDP resulta más adecuada para seleccionar configuraciones DVFS [1].

En este trabajo el EDP cumple dos papeles distintos que conviene no confundir. Como *métrica de evaluación*, es el criterio con el que se compara el sistema propuesto frente a las alternativas, conforme al cuarto objetivo específico. Como *variable objetivo de aprendizaje*, es además la cantidad que el modelo predice directamente: en lugar de inferir una etiqueta de régimen y derivar de ella una decisión de frecuencia, el modelo se entrena para identificar cuál configuración minimiza el EDP de la operación. La razón de esta elección se desarrolla en la sección 2.4.2 y se apoya en un resultado propio: el umbral que separa los regímenes del modelo Roofline se desplaza con la frecuencia de operación, de modo que una etiqueta de régimen no es una propiedad estable de la carga sino del par carga-frecuencia, y por tanto un objetivo de aprendizaje frágil.

### 1.1.12. Selección de dispositivo en sistemas heterogéneos

En un nodo que integra CPU y acelerador, ejecutar una operación admite al menos dos realizaciones distintas del mismo resultado numérico, con perfiles de tiempo y energía que no guardan una relación fija entre sí: dependen de la operación, de su tamaño y del punto operativo de cada dispositivo. La decisión de dónde ejecutar es, por tanto, un grado de libertad independiente del escalado de frecuencia y anterior a él, dado que el conjunto de estados de frecuencia disponibles es una propiedad del dispositivo elegido.

Dos magnitudes gobiernan esa decisión. La primera es el **rango dinámico de potencia** de cada dispositivo: la fracción en que su potencia disipada disminuye al reducir el reloj. Cuando ese rango es estrecho, la potencia baja poco mientras el tiempo se alarga, y el producto energía-retardo empeora aunque la elección de frecuencia sea correcta según el régimen de la carga; el escalado de frecuencia solo rinde donde el rango dinámico lo permite. Puesto que esa magnitud es una propiedad de la implementación física del dispositivo y no del mecanismo DVFS en abstracto, no es trasladable entre plataformas y debe medirse en cada una.

La segunda es el **costo de despacho** hacia el dispositivo, que se desarrolla a continuación.

### 1.1.13. Costo de despacho y su amortización

Ejecutar una operación en un acelerador conectado por un bus exige transferir los operandos hacia su memoria y recuperar el resultado, además de disponer de un contexto de



ejecución y de los recursos de las bibliotecas de cómputo. Estos costos no son homogéneos entre sí, y la distinción es determinante para decidir correctamente:

- El costo de **inicialización** — creación del contexto de ejecución, carga de los núcleos de las bibliotecas, reserva de memoria del dispositivo — se paga una sola vez por proceso. En una aplicación compuesta por muchas fases, la primera que se envía al acelerador lo asume y las siguientes lo encuentran ya disponible.
- El costo **por llamada** — transferencia de operandos y recuperación de resultados — se paga en cada despacho y no se amortiza entre operaciones.

De esta separación se sigue una consecuencia directa sobre la decisión: una operación cuya duración de cómputo sea pequeña frente a su costo por llamada no conviene en el acelerador con independencia de la frecuencia que se le aplique, mientras que la misma operación con un tamaño de problema mayor puede sí convenir. Existe, por tanto, una frontera de conveniencia dependiente del tamaño, y localizarla es parte de lo que el modelo debe aprender.

El costo de inicialización, por su parte, no debe imputarse a cada operación individual sino repartirse entre todas las que el proceso envía al dispositivo. Atribuirlo íntegro a cada llamada sesgaría la comparación en contra del acelerador y describiría mal la situación de una aplicación real. De ahí que la caracterización de este trabajo separe explícitamente ambos costos mediante el contrato temporal descrito en la sección 2.2.5, y que el umbral a partir del cual la inicialización queda amortizada se trate como una magnitud a medir y no como un supuesto.

## 1.2. Marco Legal

## 1.3. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. En este contexto, el DVFS y las técnicas afines de *frequency capping* y *power capping* se han consolidado como mecanismos de control relevantes tanto en procesadores como en aceleradores GPU. Sin embargo, la eficacia de estos mecanismos depende de forma crítica del comportamiento dinámico de la carga de trabajo, lo que ha impulsado el desarrollo de modelos de caracterización, predicción y control cada vez más sofisticados [34, 6, 5, 1, 12].

Una primera línea de trabajo corresponde a la evaluación empírica del impacto de DVFS

sobre aplicaciones y kernels HPC. Calore et al. analizaron técnicas DVFS sobre procesadores y aceleradores modernos desde el punto de vista del usuario, mostrando que el beneficio energético no es uniforme y que depende del carácter *compute-bound* o *memory-bound* de las partes críticas de la aplicación. Este resultado es relevante porque confirma que la optimización energética no puede formularse como una política global única, sino como una decisión sensible al perfil de ejecución de cada kernel o fase [5]. En una línea similar, Simsek et al. estudiaron simulaciones astrofísicas sobre GPU y mostraron que la instrumentación del código y el ajuste estático y dinámico de frecuencia permiten reducir el consumo energético con pérdidas moderadas de rendimiento, reportando reducciones de energía por GPU de hasta 7.82 % con una pérdida de rendimiento de 2.95 % [34]. Más recientemente, Costa et al. evaluaron *frequency capping* y *power capping* en un sistema exascale, observando que la efectividad relativa de cada mecanismo depende tanto de la naturaleza de la aplicación como de la escala de ejecución; en particular, *frequency capping* tendió a ofrecer mejores resultados energéticos a escala, mientras *power capping* resultó más adecuado en ciertos escenarios de nodo único o utilización irregular [6].

Una segunda línea de investigación se centra en los métodos *offline* de selección de frecuencia óptima. En este grupo destaca el trabajo de Guerreiro et al., quienes propusieron un esquema de clasificación de aplicaciones GPGPU consciente de DVFS, capaz de inferir, a partir de eventos de hardware recolectados a una sola frecuencia, cómo cambiarán tiempo de ejecución, potencia y energía al recorrer el resto del espacio de frecuencias. Su propuesta permitió encontrar pares de frecuencias casi óptimos, con ahorros medios de energía de 16 % a 20 % y picos de hasta 36 %, según la arquitectura evaluada [12]. De forma complementaria, Ali et al. desarrollaron un método automatizado y portable para seleccionar la frecuencia óptima de GPU a partir de tres etapas: caracterización de características, modelado analítico de potencia y tiempo de ejecución, y selección multiobjetivo mediante EDP y ED<sup>2</sup>P. Su metodología requiere recolectar métricas en una configuración base y luego estimar el comportamiento en el resto del espacio DVFS, alcanzando ahorros promedio de 29.6 % de energía con 5.2 % de pérdida de rendimiento en una arquitectura y 22.6 % con 4.7 % en otra [1]. Estos trabajos constituyen antecedentes sólidos porque muestran que el espacio DVFS puede explorarse sin fuerza bruta y con buena portabilidad. No obstante, comparten una limitación importante: su lógica de decisión es esencialmente *offline* o precomputada, por lo que no aborda de manera explícita la adaptación fina a la multifasicidad intra-ejecución de aplicaciones HPC generales.

Una tercera línea aborda la caracterización y clasificación de cargas o kernels a partir de telemetría de bajo nivel. Shekofteh et al. demostraron que la clasificación de kernels GPU

puede realizarse con un conjunto reducido de métricas, seleccionadas para minimizar *overhead* sin sacrificar precisión, lo que confirma que la identificación *online* de comportamientos como *compute-bound* y *memory-bound* es técnicamente viable. Sin embargo, su objetivo principal fue mejorar la planificación y co-ejecución de kernels, no gobernar DVFS [33]. De manera análoga, Littman y Deakin exploraron el uso de aprendizaje automático para clasificar límites de rendimiento a partir de contadores de rendimiento, apoyando la idea de que la frontera *compute-bound/bandwidth-bound* puede inferirse desde datos y no solo mediante inspección manual del modelo Roofline [25]. En conjunto, estos trabajos respaldan la hipótesis de que una fase de ejecución puede representarse mediante una firma microarquitectónica observable, aunque no proponen por sí mismos un lazo de control energético que traduzca dicha clasificación en decisiones DVFS.

La cuarta línea corresponde a los mecanismos *online* de control dinámico durante ejecución. El antecedente más fuerte en esta categoría es DRLCAP, que propone un marco general de *frequency capping* de GPU en tiempo de ejecución basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema para detectar cambios de fase, aprende una política adaptativa y reduce en promedio 22 % de la energía de GPU con menos de 3 % de degradación en arquitecturas NVIDIA, además de reportar resultados sobre AMD [38]. Este trabajo demuestra que el control *online* de frecuencia de GPU es factible y efectivo, pero también deja clara una diferencia conceptual frente a enfoques más ligeros: se trata de una política basada en aprendizaje por refuerzo profundo, orientada a GPU, sin formular explícitamente el problema como clasificación supervisada ligera de fases seguida de una política discreta interpretable. Por tanto, aunque constituye un antecedente directo, no agota el espacio de soluciones posibles para agentes ligeros en espacio de usuario.

Finalmente, una restricción frecuentemente subestimada en la literatura es el *overhead* del propio mecanismo de control. Velicka et al. mostraron que la latencia de conmutación de frecuencia en GPU no es despreciable, varía entre arquitecturas y entre pares de frecuencias, y puede llegar a ser suficientemente alta como para comprometer el beneficio de políticas excesivamente reactivas [37]. Esta observación es central, porque implica que un esquema *online* no debe limitarse a detectar fases correctamente: también debe amortizar el costo del cambio de frecuencia y evitar decisiones demasiado finas o frecuentes.

Aun cuando la literatura reciente ha avanzado en control dinámico de frecuencia durante ejecución, la mayoría de los trabajos revisados se concentra en el dominio GPU de forma aislada. DRLCAP, por ejemplo, propone un marco *online* guiado por aprendizaje por refuerzo profundo, pero su problema de control está centrado en la GPU y no en una política coordinada sobre CPU y GPU dentro de un nodo heterogéneo [38]. Del mismo modo,

los métodos de selección óptima de frecuencia de Ali et al. y los modelos de clasificación conscientes de DVFS de Guerreiro et al. se enfocan en la selección de configuraciones para GPU [1, 12]. Aunque existen propuestas de coordinación entre CPU, GPU y memoria, como SparseDVFS, estas se desarrollan en el contexto de inferencia de redes neuronales en dispositivos *edge*, con una lógica de partición por bloques y una señal de control centrada en la esparsidad de operadores, por lo que su alcance, supuestos de carga y entorno de despliegue difieren sustancialmente del problema de HPC científico general abordado aquí [42].

En conjunto, esta revisión muestra que la literatura ya dispone de componentes importantes del problema, aunque todavía de forma fragmentada: existen métodos *offline* robustos para seleccionar frecuencias óptimas, mecanismos de caracterización y clasificación de cargas, y controladores *online* avanzados para GPU. Sin embargo, sigue abierta una brecha en torno a soluciones ligeras, implementables en espacio de usuario, que integren caracterización *online* de fases de ejecución, telemetría estándar de baja intrusión y decisiones DVFS coordinadas sobre CPU y GPU en nodos heterogéneos de alto rendimiento. En ese sentido, el aporte de este trabajo no radica en afirmar que la clasificación de fases o el control *online* sean completamente inéditos, sino en proponer una integración distinta: un agente ligero, implementable sin modificaciones al kernel, que utilice modelos supervisados clásicos para inferir el régimen de ejecución y traduzca esa inferencia en decisiones DVFS de bajo *overhead*, con foco explícito en la optimización del Producto Energía–Retardo.

# Capítulo 2

## Metodología

### 2.1. Enfoque metodológico

La presente investigación se desarrolla bajo un enfoque cuantitativo de tipo experimental aplicado, orientado al cumplimiento del objetivo general del proyecto. El propósito metodológico consiste en determinar en qué medida un agente de gestión DVFS permite maximizar la eficiencia energética sin degradar significativamente el rendimiento de la aplicación en ejecución. Dicho impacto se evalúa a través de métricas cuantificables y reproducibles, principalmente el tiempo de ejecución, la energía consumida y el Producto Energía–Retardo (EDP).

El carácter experimental se sustenta en la manipulación deliberada de dos variables independientes — el dispositivo que ejecuta la operación y el estado de frecuencia impuesto a ese dispositivo — sobre un conjunto controlado de cargas de trabajo, observando su efecto sobre variables dependientes de rendimiento y consumo. El carácter aplicado responde a que el producto de la investigación no es únicamente conocimiento descriptivo sobre el comportamiento energético de las cargas, sino un artefacto software funcional cuya utilidad se evalúa empíricamente frente a las alternativas ya disponibles en el sistema operativo.

El desarrollo se estructura en cuatro fases secuenciales e interdependientes, que abarcan desde la caracterización de cargas y extracción de telemetría hasta la validación empírica del sistema. Este capítulo describe en detalle el diseño de la primera fase, por ser la que se encuentra ejecutada al momento de redacción del presente documento, y presenta el diseño previsto para las fases restantes.

## 2.2. Diseño metodológico

### 2.2.1. Población y muestra

La población de estudio está constituida por el conjunto de operaciones de cómputo que componen las aplicaciones científicas ejecutables en sistemas heterogéneos CPU–GPU. Debido a la imposibilidad de abarcar la totalidad de estas, se define una muestra no probabilística de tipo intencional, compuesta por operaciones de álgebra lineal, transformadas y esquemas de acceso a memoria de amplia presencia en aplicaciones HPC reales. El uso de operaciones representativas y de una muestra intencional es coherente con la necesidad de trabajar con cargas controladas y reproducibles dentro del alcance de un proyecto de pregrado.

La estructura de la muestra difiere de la de un catálogo de benchmarks convencional, y esa diferencia responde directamente a la unidad de decisión adoptada en la sección 2.4.1. Puesto que el sistema debe decidir entre ejecutar una operación en la CPU o en el acelerador, la muestra se organiza como un **catálogo dual**: cada configuración de cómputo — una operación con un tamaño de problema concreto — existe en dos variantes funcionalmente equivalentes, una implementada sobre bibliotecas de CPU y otra sobre bibliotecas del acelerador, verificadas ambas contra el mismo criterio de corrección y emitiendo ambas el mismo contrato temporal de la sección 2.2.5. Un identificador de configuración compartido enlaza las dos variantes, y es la llave que permite construir la etiqueta de la ecuación (2.2).

Un catálogo compuesto por benchmarks heterogéneos entre sí — cada uno resolviendo un problema distinto — no admitiría esa comparación: no existiría el par sobre el cual decidir. La contrapartida de esta elección es que las cargas del catálogo son operaciones y no aplicaciones completas, y por tanto la evaluación del sistema sobre una aplicación multifásica real corresponde a la Fase 4 (sección 2.6) y no a la caracterización.

Cada operación se instancia sobre una rejilla de tamaños de problema, y no en un único tamaño, porque el tamaño es precisamente el parámetro que desplaza la frontera de conveniencia entre ambos dispositivos: una operación pequeña puede no amortizar el costo de transferencia hacia el acelerador que la misma operación, con un tamaño mayor, sí amortiza. La rejilla se densifica en la banda donde el tamizaje preliminar situó esa frontera.

El criterio de inclusión en el catálogo experimental combina tres condiciones. En primer lugar, la carga debe disponer de una verificación interna de corrección que permita descartar ejecuciones inválidas, de modo que una corrida cuyo resultado numérico no sea correcto no contamine el conjunto de datos. En segundo lugar, la carga debe reportar de forma explícita el volumen de trabajo realizado, requisito necesario para estimar el numerador de la intensidad

operacional según lo descrito en la sección 1.1.2. En tercer lugar, ese volumen de trabajo debe estar expresado en operaciones de punto flotante y no en otra unidad (números aleatorios generados, claves ordenadas), pues de lo contrario no sería comparable con un techo Roofline calibrado en FLOPs; las cargas que cuantifican su trabajo en otras unidades quedan excluidas del catálogo de caracterización, con independencia de su relevancia como benchmarks en otros contextos.

### 2.2.2. Plataforma experimental

Los experimentos se ejecutan sobre un nodo de cómputo heterogéneo cuyas características se resumen en la Tabla 2.1. La selección de la plataforma no fue arbitraria: además de la disponibilidad de un acelerador gráfico instrumentable mediante NVML, se estableció como condición de admisibilidad la presencia de una interfaz de medición energética interna accesible sin privilegios administrativos, dado que, como se argumentó en el marco conceptual, dicha capacidad es una propiedad del procesador que no admite sustitución por software.

La identidad precisa de la microarquitectura no es un dato meramente descriptivo en este trabajo: varios de los eventos crudos de PMU que el instrumento utiliza (sección 1.1.5) carecen de mapeo genérico en el kernel de este nodo y se abren mediante una codificación específica del procesador, verificada empíricamente y restringida en el código a coincidir exactamente con el par `cpu family/model` de la Tabla 2.1; ese mismo gateo es lo que impide, por diseño, que el instrumento aplique sin verificación una codificación cruda a un nodo distinto de este.

Tabla 2.1: Características de la plataforma experimental.

| Componente                                            | Descripción                                                                                                   |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Procesador                                            | 2 × Intel Xeon Gold 5315Y                                                                                     |
| Microarquitectura                                     | Ice Lake-SP (10 nm, generación de servidor lanzada en 2021)                                                   |
| Identificador CPUID ( <code>cpu family/model</code> ) | 6 / 106 — el mismo par leído en tiempo de ejecución desde /                                                   |
| Extensiones SIMD relevantes                           | AVX2, FMA3, AVX-512 ( <code>avx512f</code> , <code>avx512bw</code> , <code>avx512cd</code> , <code>avx</code> |
| Núcleos                                               | 8 núcleos físicos por zócalo, 2 hilos por núcleo (32 lógicos)                                                 |
| Nodos NUMA                                            | 2 (uno por zócalo)                                                                                            |
| Jerarquía de caché                                    | L1d 48 KiB, L1i 32 KiB, L2 1.25 MiB por núcleo; L3 12 MiB                                                     |
| Tamaño de línea de caché                              | 64 bytes                                                                                                      |
| Controlador de frecuencia                             | <code>intel_pstate</code>                                                                                     |
| Rango de frecuencia                                   | 0.8–3.6 GHz                                                                                                   |
| Dominio de frecuencia                                 | Por núcleo lógico                                                                                             |
| Medición energética                                   | Intel RAPL (dominios de paquete y DRAM por zócalo)                                                            |
| Acelerador                                            | NVIDIA A100 PCIe 40 GB                                                                                        |
| Gestor de recursos                                    | Slurm                                                                                                         |

Las condiciones de ejecución se controlan mediante asignación exclusiva del nodo a través del gestor de recursos, de modo que ninguna carga ajena al experimento comparta el procesador durante las mediciones. Los procesadores lógicos empleados se restringen a hilos primarios de un mismo zócalo, evitando la asignación simultánea de hilos hermanos del mismo núcleo físico, de acuerdo con lo expuesto en la sección 1.1.6.

### 2.2.3. Instrumentos de recolección

La recolección de telemetría se implementa mediante un instrumento propio, desarrollado específicamente para este trabajo, compuesto por dos capas con responsabilidades separadas.

La capa inferior es un ejecutor escrito en C++ que lanza la carga de trabajo como proceso hijo y adjunta los contadores de hardware sobre ella. El diseño de esta capa responde directamente a los requisitos de atribución expuestos en la sección 1.1.5: el proceso hijo se detiene inmediatamente después de su creación y antes de ejecutar la primera instrucción de la carga, momento en el cual el proceso padre abre los contadores asociados a ese identificador de proceso con herencia habilitada; solo entonces se reanuda la ejecución. Este protocolo garantiza que ninguna instrucción de la carga medida se ejecute fuera del intervalo de observación y que la medida abarque el árbol completo de procesos e hilos que la carga genere, sin



incluir actividad ajena.

El muestreo se realiza mediante un hilo productor dedicado, fijado a un procesador lógico distinto de los asignados a la carga, que a intervalos regulares lee los contadores y deposita las muestras en una estructura circular de productor y consumidor únicos. Un hilo consumidor, igualmente fijado a un procesador lógico separado, extrae las muestras y las persiste. Esta separación evita que la actividad de escritura a disco interfiera con la periodicidad del muestreo, y el aislamiento de ambos hilos respecto de los núcleos medidos reduce su interferencia sobre la carga observada. La ausencia de pérdidas en la estructura circular se registra como indicador de integridad de cada corrida.

La capa superior es un orquestador escrito en Python, responsable de la lógica experimental: interpreta el manifiesto que define la campaña, detecta las capacidades de la plataforma, ejecuta las verificaciones previas, realiza la calibración, planifica y lanza cada combinación experimental, aplica el estado de frecuencia correspondiente, valida el resultado de cada corrida y transforma las muestras crudas en el conjunto de datos final. La separación entre ambas capas responde a un criterio de responsabilidad: la capa C++ resuelve la medición de baja latencia, donde el costo de la instrumentación es crítico, mientras que la capa Python resuelve la lógica experimental, donde prima la claridad y la trazabilidad sobre el rendimiento.

#### 2.2.4. Overhead del propio instrumento de medición

Todo instrumento de medición perturba, en alguna medida, aquello que mide. La capa C++ descrita en la sección anterior fue diseñada para minimizar esa perturbación — hilos productor y consumidor aislados en núcleos separados de los medidos, estructura de datos sin bloqueos —, pero el diseño por sí solo no cuantifica cuánto tiempo añade la instrumentación activa frente a la misma carga sin instrumentar. Esa magnitud se midió directamente, no se asumió despreciable: cada combinación experimental del eje de CPU se ejecuta por partida doble, una vez con la telemetría completa activa y una vez como línea base sin ella, ambas bajo el mismo estado de frecuencia y la misma repetición, y se compara su tiempo de pared.

Sobre 540 pares medidos en la campaña final de CPU, el sobre costo de instrumentación tiene una media del 1.95 %, estable entre las nueve cargas del catálogo — no se concentra en una carga particular ni crece con la duración de la corrida. Esta caracterización se llevó a cabo sobre el eje de CPU; su extensión sistemática al eje de GPU, donde el muestreo lo realiza en su mayoría la biblioteca de gestión del dispositivo y no el hilo productor propio, queda como verificación pendiente y no se reporta aquí como magnitud medida.

Una vez caracterizado el sobre costo y confirmada su estabilidad, remedirlo en cada repe-

tición de cada campaña deja de aportar información nueva. Las campañas posteriores a esta caracterización declaran el par línea base–telemetría únicamente sobre la primera repetición de cada combinación, como vigilancia contra una desviación futura del instrumento, y no como remediación completa — una decisión que reduce en una fracción sustancial el número de corridas necesarias sin renunciar a la capacidad de detectar si el propio instrumento cambia de comportamiento.

### 2.2.5. Contrato temporal de despacho: regiones fría y caliente

Medir el costo de ejecutar una operación en un dispositivo exige decidir antes qué parte del tiempo de vida del proceso pertenece a ese costo. La decisión no es cosmética: en el acelerador, la creación del contexto de ejecución y la carga de los núcleos de las bibliotecas de cómputo constituyen un costo sustancial que una aplicación real paga **una sola vez**, mientras que la transferencia de operandos y resultados se paga en **cada** llamada. Atribuir ambos costos indistintamente a cada operación describiría mal la situación que el agente debe decidir, y lo haría de forma sesgada en contra del acelerador, que es precisamente uno de los dos términos de la comparación.

Por esa razón, cada carga del catálogo dual emite un contrato temporal explícito, compuesto por cinco marcas de tiempo absolutas tomadas con un reloj monótono — el mismo que sella las ventanas de muestreo, lo que las hace directamente comparables. Ese contrato delimita dos regiones:

- La región **fría** comienza con los datos de entrada ya residentes en memoria del anfitrión y antes de cualquier llamada al dispositivo, e incluye la creación del contexto, los descriptores y planes de las bibliotecas, las asignaciones de memoria, la primera transferencia de operandos, la primera ejecución de la operación y el regreso del resultado al anfitrión. Una marca intermedia separa, dentro de ella, la preparación de recursos del primer despacho propiamente dicho. En el análogo de CPU esta región comprende la inicialización perezosa real de la biblioteca y el primer cómputo.
- La región **caliente** comprende las llamadas posteriores, que reutilizan contexto, descriptores y asignaciones pero vuelven a transferir todos los operandos y todos los resultados de cada operación.

La generación de los datos de entrada y la verificación de corrección del resultado quedan deliberadamente fuera de ambas regiones: son costos del banco de pruebas y no del despacho que se pretende caracterizar.

El instrumento valida este contrato al cerrar cada corrida y falla de forma cerrada: una

marca ausente, duplicada o no monótona invalida la corrida completa en lugar de producir un dato de procedencia ambigua. Disponer de las fronteras en tiempo absoluto es lo que permite integrar tiempo y energía sobre exactamente el mismo intervalo, y es también lo que hace medible por separado el costo único de inicialización, insumo del umbral de amortización que se evalúa en la Fase 4.

### 2.2.6. Variables observadas

Una ventana de muestreo se define como el intervalo entre dos lecturas consecutivas de los contadores de núcleo. El trabajo realizado en ella se obtiene mediante la diferencia entre ambas lecturas; por esta razón, la primera lectura de una corrida no produce una ventana. Su duración corresponde al tiempo realmente transcurrido, medido con un reloj monótono, y no al período nominal solicitado.

En el instrumento implementado, cada lectura de los contadores de núcleo se marca con `CLOCK_MONOTONIC`. La duración de una ventana se obtiene de la diferencia entre marcas consecutivas y no del período nominal configurado. Las salidas de *uncore* producidas por `perf stat -I` ya representan deltas por intervalo, por lo que se procesan sin volver a diferenciarlas.

La Tabla 2.2 resume las señales de CPU capturadas por el instrumento. El conjunto de eventos de núcleo se dimensionó para permanecer dentro del presupuesto de contadores físicos simultáneos de la plataforma, comprobado durante la calificación del instrumento. Las mediciones de RAPL y del controlador de memoria tienen sus propios intervalos de muestreo y se asocian temporalmente con las ventanas de núcleo durante el procesamiento.

Tabla 2.2: Señales de telemetría de CPU capturadas por el instrumento.

| Señal                                                                  | Propósito                                                                                                                                                                                 |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Instrucciones retiradas                                                | Cálculo del IPC y de la tasa de instrucciones.                                                                                                                                            |
| Ciclos de reloj                                                        | Base temporal para el IPC y para la fracción de ciclos de estancamiento.                                                                                                                  |
| Referencias y fallos de caché                                          | Cálculo de la tasa de fallos y de los fallos por cada mil instrucciones. También proporcionan un indicador auxiliar de presión sobre la jerarquía de memoria.                             |
| Ciclos totales de estancamiento de la ejecución                        | Indicador complementario de pérdida de progreso. No identifica por sí solo si el estancamiento fue causado por memoria.                                                                   |
| Operaciones de punto flotante retiradas (escalar, 128, 256 y 512 bits) | Cálculo de los FLOPs por ventana mediante la ponderación correspondiente al ancho vectorial (ecuación (2.1)). Constituyen el numerador de la intensidad operacional.                      |
| Tiempos de habilitación y conteo del evento de referencia              | Control de la medición y soporte para el escalado aplicado por la interfaz de contadores.                                                                                                 |
| Comandos CAS de lectura y escritura del IMC                            | Estimación del tráfico que alcanza la DRAM. La suma de ambos contadores, multiplicada por 64 bytes por comando, constituye el denominador de la intensidad operacional.                   |
| Energía acumulada por dominio RAPL                                     | Cálculo del incremento de energía y de la potencia media sobre el intervalo de muestreo de RAPL asociado temporalmente con la ventana.                                                    |
| Frecuencia observada de cada núcleo delegado                           | Lectura de <code>scaling_cur_freq</code> para cada núcleo lógico asignado a la carga, utilizada para contrastar el nivel solicitado con el comportamiento observado durante la ejecución. |

A partir de las lecturas de núcleo se calculan el IPC, los fallos por cada mil instrucciones, la tasa de fallos de caché y la fracción de ciclos de estancamiento. Los FLOPs se agregan sobre cada intervalo de los contadores del IMC antes de dividirlos por los bytes transferidos hacia o desde la DRAM. De esta manera, la intensidad operacional y la etiqueta de entrenamiento

dependen del tráfico observado por el controlador de memoria, no de una aproximación basada en los fallos de caché.

## **2.3. Fase 1: Recolección de información y caracterización de cargas**

La primera fase tiene por objetivo caracterizar el comportamiento de las cargas bajo distintos estados de frecuencia, junto con el instrumento y el protocolo que garantizan la validez de esa caracterización. Se describe a continuación el procedimiento completo.

### **2.3.1. Verificación previa de la plataforma**

Antes de ejecutar cualquier medición se aplica una batería de verificaciones automáticas sobre el nodo, orientada a confirmar que las condiciones supuestas por el diseño experimental se cumplen efectivamente. Estas verificaciones se agrupan en cinco familias: capacidades del entorno (estado de las tecnologías de aceleración de frecuencia, afinidad NUMA de los núcleos asignados, política de multihilo declarada, dominio real de frecuencia, número de contadores simultáneos disponibles); integridad del catálogo (existencia y permisos de ejecución de cada binario, correspondencia de su suma de verificación, validez del criterio de éxito declarado, suficiencia de memoria); disponibilidad de instrumentación (dominios energéticos solicitados frente a los presentes, corrección del manejo de desbordamiento); condiciones de almacenamiento y presupuesto; y, cuando la campaña contempla control de frecuencia, permisos efectivos de escritura y cobertura del dominio.

Las verificaciones se clasifican en bloqueantes y no bloqueantes. Una verificación bloqueante que no se satisface detiene la campaña antes de tocar el sistema, bajo el principio de que es preferible no producir datos a producir datos cuya validez no puede sostenerse. Este mecanismo se ejecuta de forma automática como parte del comando que lanza la campaña, y no como un paso manual previo, de modo que no exista la posibilidad de iniciar una medición sin haberlo aplicado.

### **2.3.2. Catálogo de cargas de trabajo**

El catálogo experimental se compone de cargas de dos roles distintos. Las cargas de calibración sirven para determinar empíricamente los techos del modelo Roofline sobre la plataforma: un microbenchmark de ancho de banda de memoria, cuyo conjunto de trabajo excede ampliamente la capacidad de la caché de último nivel para garantizar que el tráfico

alcance la memoria principal, proporciona la estimación de  $BW_{\text{pico}}$ ; y un microbenchmark de densidad aritmética sobre datos residentes en caché proporciona la estimación de  $P_{\text{pico}}$ . Las cargas de conjunto de datos son los benchmarks cuyo comportamiento se pretende caracterizar.

La Tabla 2.3 resume la composición final del catálogo, ya estabilizada tras dos extensiones sucesivas: el conjunto inicial de siete cargas (seis kernels de una suite de benchmarks paralelos más la referencia de álgebra lineal densa) se amplió con dos cargas adicionales, caracterizadas primero en el punto de operación nativo, al confirmarse que la distribución de etiquetas resultante del conjunto inicial por sí solo no cubría de forma suficiente el régimen intermedio entre los dos extremos del modelo Roofline. Todas las cargas de conjunto de datos provienen de suites de benchmarks científicos de amplia adopción, se emplean sin modificar su código fuente y se compilan sobre la plataforma experimental con las opciones por defecto de cada suite, registrando la suma de verificación del binario resultante. Este registro cumple una función metodológica precisa: permite confirmar, antes de cada corrida individual, que el binario ejecutado es exactamente el mismo que se caracterizó, descartando la posibilidad de que una recompilación silenciosa altere las condiciones a mitad de campaña. Dado que un mismo código fuente compilado con cadenas de herramientas distintas produce binarios funcionalmente equivalentes pero con sumas de verificación diferentes, este registro se mantiene asociado a la plataforma en la que se realizó la compilación.

Tabla 2.3: Composición del catálogo experimental de cargas de trabajo.

| Rol               | Cantidad | Propósito                                                                                                                                                                                                                                                                                         |
|-------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Calibración       | 2        | Estimación empírica de $BW_{\text{pico}}$ (ancho de banda sostenido de memoria) y de $P_{\text{pico}}$ (densidad aritmética sobre datos en caché).                                                                                                                                                |
| Conjunto de datos | 6        | Kernels de una suite de benchmarks científicos paralelos, seleccionados por reportar su trabajo en operaciones de punto flotante y cubrir distintos patrones de acceso a memoria (resolución de sistemas por métodos implícitos, multigrid, gradiente conjugado, transformada rápida de Fourier). |
| Conjunto de datos | 1        | Multiplicación densa de matrices sobre una biblioteca de álgebra lineal optimizada, como referencia de régimen intensivo en cómputo.                                                                                                                                                              |
| Conjunto de datos | 1        | Dinámica molecular de N-cuerpos sobre una malla regular, de una suite de benchmarks heterogéneos originalmente concebida para acelerador gráfico y compilada aquí en su variante paralela de CPU.                                                                                                 |
| Conjunto de datos | 1        | Triple multiplicación de matrices densas, de una suite de <i>kernels</i> de rendimiento en memoria compartida, con verificación numérica propia además de la suma de verificación del binario.                                                                                                    |

Las dos cargas incorporadas en la segunda ronda comparten precisión doble y expectativa cualitativa de régimen intermedio; ambas se admitieron al catálogo bajo el mismo criterio de inclusión de la sección 2.2.1, sin trato diferenciado por haberse incorporado en un momento posterior del proyecto.

### 2.3.3. Calibración y criterio de etiquetado

La calibración se ejecuta al inicio de cada campaña, sobre la misma asignación de núcleos y con la misma política de afinidad que se emplea después en las corridas de caracterización, de

modo que los techos obtenidos correspondan a la configuración realmente utilizada. A partir de los valores medidos se calcula el punto de inflexión según la ecuación (1.3). Como control de validez, los valores obtenidos se contrastan contra una referencia empírica declarada en el manifiesto de la campaña, y una discrepancia superior a una tolerancia establecida detiene la ejecución: esta verificación protege frente a calibraciones realizadas bajo condiciones anómalas que pasarían inadvertidas si el valor resultante se aceptara sin contraste.

Sobre esta base se define el etiquetado. Cada ventana de muestreo recibe una etiqueta derivada exclusivamente de la comparación entre su intensidad operacional observada y el punto de inflexión calibrado, sin intervención de juicio manual. Adicionalmente, cada carga del catálogo declara una expectativa cualitativa de régimen, proveniente de la literatura sobre la suite correspondiente. Es importante subrayar que ambas anotaciones cumplen funciones distintas y no deben confundirse: la etiqueta derivada del modelo constituye la variable objetivo del problema de clasificación, mientras que la expectativa declarada actúa únicamente como control de coherencia. La correspondencia entre ambas, evaluada de forma agregada por carga, permite detectar errores sistemáticos del instrumento, pero la expectativa nunca sustituye ni corrige la etiqueta medida.

Como control adicional de estabilidad, cada campaña incluye repeticiones de una carga de referencia, sobre las cuales se calcula la dispersión relativa de las métricas de eficiencia. Una dispersión elevada indica que las condiciones de medición no son suficientemente estables como para sostener comparaciones finas entre corridas, y se registra como advertencia asociada a la campaña.

### 2.3.4. Caracterización de intensidad operacional y calibración Roofline en GPU

A diferencia de CPU, en GPU **no existe una medición de FLOPs por ventana**. La telemetría muestreada durante la corrida en el dominio GPU proviene exclusivamente de NVIDIA Management Library (NVML), y por cada muestra produce únicamente potencia, utilización (de cómputo y de memoria), reloj del multiprocesador de streaming, temperatura y energía acumulada del dispositivo — ninguna de estas señales cuantifica cuántas operaciones de punto flotante realizó el kernel ni cuántos bytes movió. La intensidad operacional de cada carga GPU se obtiene, en cambio, mediante un procedimiento distinto y separado, más cercano en espíritu a la calibración de los techos del modelo Roofline que a un muestreo continuo: una caracterización dinámica *fuera de línea* del kernel, ejecutada con un depurador de instrucciones (NVIDIA Nsight Compute) [30] antes — y de forma independiente — de la campaña de adquisición de telemetría NVML en tiempo real. Nsight Compute expone con-



tadores de hardware capaces de contabilizar, por cada lanzamiento de kernel, tanto el tráfico total hacia memoria del dispositivo como el número de instrucciones aritméticas de punto flotante efectivamente ejecutadas, discriminadas por tipo de operación (suma, multiplicación y multiplicación–suma fusionada) y por precisión (simple o doble); a partir de estos conteos, la intensidad operacional de la carga se obtiene como el cociente entre el total de operaciones de punto flotante — contando cada multiplicación–suma fusionada como dos operaciones — y el total de bytes transferidos, agregados sobre un número fijo de lanzamientos consecutivos del kernel.

El valor así obtenido es un único número por carga, no una serie por ventana: se declara como una propiedad fija del catálogo, análoga a la suma de verificación del binario, y el post-procesamiento lo copia sin variación a cada una de las muestras NVML de esa corrida. Esto es metodológicamente válido únicamente porque, igual que en CPU, la intensidad operacional de un kernel es una propiedad de su algoritmo y del tamaño del problema que resuelve, no del estado de frecuencia bajo el cual se ejecuta ni del instante de la corrida en que se observa — por lo que no es necesario, ni válido dado el costo de intrusión del perfilado detallado sobre el tiempo de ejecución, repetir esta medición dentro de cada corrida de la campaña principal. Esta vía constituye, igual que en CPU, una medición directa de ambos términos de la ecuación (1.4), aunque agregada por lanzamiento de kernel en vez de por ventana temporal, y calculada una única vez fuera de línea en vez de continuamente durante la ejecución.

Esta discriminación por precisión, sin embargo, presupone que Nsight Compute se invoque solicitando los contadores de **ambas** precisiones simultáneamente para cada carga, y no únicamente los de la precisión que el catálogo declara para ella por diseño de kernel: una auditoría posterior (ARC-110) encontró que las herramientas de caracterización usadas en este trabajo inicialmente seleccionaban un único juego de contadores según esa declaración, de modo que una carga con operaciones reales en la precisión no solicitada las habría subestimado sin ninguna señal de que la mezcla existía. Corregido para solicitar siempre ambas precisiones y contrastar la declaración del catálogo contra lo observado, el método de un único techo Roofline por precisión — el empleado en este trabajo — solo es válido para cargas empíricamente homogéneas en una precisión; una carga con proporciones no triviales de ambas precisiones no admite compararse contra un único ridge sin antes decidir explícitamente cómo tratar esa mezcla (excluirla, tratarla como una categoría aparte, o extender el modelo a varios techos), decisión que este trabajo no adopta unilateralmente para ninguna carga de su catálogo.

Conviene señalar explícitamente por qué el mecanismo de perfilado en GPU no hereda la restricción de presupuesto de contadores físicos simultáneos discutida para CPU en la sección

1.1.3, ni el riesgo de multiplexación asociado a ella. Un contador de PMU de CPU, abierto mediante `perf_event_open`, se muestrea en vivo mientras la carga corre una sola vez; cuando el número de eventos solicitados excede el número de contadores físicos disponibles, el kernel reparte el tiempo de esa única ejecución entre ellos y extrapola cada lectura mediante el factor de escalado tiempo-habilitado/tiempo-contando (sección 1.1.3), un procedimiento que asume una tasa de evento uniforme durante todo el intervalo y que resulta especialmente riesgoso cuando la ventana de observación es corta frente al período de rotación del kernel. Nsight Compute, en cambio, no muestrea en vivo una única ejecución: cuando las métricas solicitadas exceden lo que el hardware de contadores de la GPU puede capturar en una sola pasada, la herramienta *reproduce* el lanzamiento del kernel completo tantas veces como sea necesario, una pasada por cada subconjunto de métricas, bajo la condición de que el kernel sea determinista frente a sus mismas entradas. Cada métrica se obtiene así de una ejecución íntegra del kernel, no de una fracción de tiempo compartida con otras métricas, por lo que no existe una extrapolación lineal equivalente a la de CPU ni una ventana de observación corta que pueda quedar a mitad de una rotación. La precisión aritmética (simple o doble) se discrimina, en consecuencia, sin ningún compromiso de presupuesto: da igual cuántas métricas o precisiones se soliciten, el costo se traduce en más pasadas de perfilado, no en una restricción de cuántas pueden coexistir — un costo asumible precisamente porque, como se explicó arriba, esta medición ocurre una única vez por carga, fuera de la campaña de adquisición de telemetría en tiempo real.

La calibración de los techos del modelo Roofline en GPU sigue el mismo principio general descrito para CPU — microbenchmarks dedicados a saturar por separado cada límite, bajo las condiciones reales de la plataforma —, con dos particularidades propias del dispositivo. Primera, dado que las GPU consideradas ejecutan operaciones de simple y doble precisión a tasas pico sustancialmente distintas,  $P_{\text{pico}}$  se calibra por separado para cada precisión, y el punto de inflexión de la ecuación (1.3) se calcula igualmente por separado, de modo que una carga expresada en una precisión se compara exclusivamente contra el techo correspondiente a esa misma precisión. Segunda, la carga empleada para calibrar  $P_{\text{pico}}$  se implementó como un microbenchmark propio de multiplicación–suma fusionada sobre datos residentes en registros, en lugar de recurrir a una biblioteca de álgebra lineal optimizada de terceros: una biblioteca de este tipo puede seleccionar internamente, sin que el usuario lo solicite explícitamente, una ruta de cómputo acelerada por hardware especializado no representativo de la aritmética general de punto flotante del dispositivo, lo que produciría un techo de cómputo inflado frente al alcanzable por las cargas del catálogo que no emplean dicha ruta, y en consecuencia clasificaría erróneamente como *memory-bound* cargas que, medidas contra el techo de aritmética vainilla correspondiente a su precisión, resultan *compute-bound*. Este riesgo es formalmente

equivalente al ya discutido en la sección 2.2.1 para el caso de una carga que cuantifica su trabajo en una unidad distinta de FLOPs: en ambos casos, comparar magnitudes que no están expresadas en el mismo referente invalida la clasificación resultante, aunque el error no se manifieste como una falla explícita del instrumento.

### 2.3.5. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional

Puesto que la intensidad operacional de cada carga GPU se fija mediante la medición directa descrita en la sección 2.3.4, cualquier ajuste posterior de los parámetros de ejecución de una carga — por ejemplo, para alcanzar una duración de corrida suficiente para los fines descritos en la sección 2.3.7 — exige distinguir explícitamente entre dos categorías de parámetros según su efecto sobre dicha medición.

La primera categoría comprende los parámetros que repiten el mismo patrón de cómputo un mayor número de veces sin alterar el tamaño del problema resuelto en cada repetición — un número de iteraciones de un mismo esquema numérico, o la duración simulada de un proceso cuyo paso de integración permanece fijo —. Un ajuste de esta naturaleza no puede alterar la intensidad operacional del kernel, dado que cada repetición ejecuta exactamente el mismo balance entre trabajo aritmético y tráfico de memoria que la anterior; en consecuencia, este tipo de ajuste puede aplicarse sin necesidad de repetir la medición de la sección 2.3.4. La segunda categoría comprende los parámetros que alteran el tamaño real del problema — la geometría de una malla, el número de elementos de una simulación de N-cuerpos, o la resolución de una imagen —, para los cuales dicha invarianza no puede asumirse: un problema de mayor tamaño puede modificar la proporción entre trabajo aritmético y tráfico de memoria, entre otras razones porque el conjunto de datos activo deja de residir en un nivel de la jerarquía de caché que sí alcanzaba a contener al problema original, incrementando el tráfico efectivo hacia memoria principal de forma no proporcional al aumento del trabajo aritmético. Por ello, todo ajuste de la segunda categoría se trata como una modificación que invalida la caracterización previa de la carga, y se somete obligatoriamente a una nueva medición mediante el procedimiento de la sección 2.3.4 antes de aceptar cualquier dato adquirido bajo la nueva configuración; cuando dicha remediación confirma un cambio material en la intensidad operacional, se verifica adicionalmente que la carga conserve su clasificación cualitativa frente al punto de inflexión vigente, dado que un cambio de magnitud no implica necesariamente un cambio de régimen.

Cuando una carga no dispone de un parámetro de la primera categoría — por ejemplo, porque su implementación no contempla una noción de repetición interna del mismo cómputo

—, la única vía disponible para extender su duración es un parámetro de la segunda categoría, con el costo de verificación que ello implica; y cuando, adicionalmente, el modelo de ejecución del dispositivo impone un límite estructural sobre dicho parámetro — por ejemplo, un límite documentado sobre el número de unidades de ejecución concurrentes direccionables por un único lanzamiento de kernel —, la duración máxima alcanzable para esa carga queda acotada por dicho límite de hardware, con independencia de cualquier criterio estadístico de suficiencia de muestreo. En tales casos, la imposibilidad de extender la corrida más allá del límite estructural del dispositivo se declara como una limitación del instrumento específica de esa carga, y no se fuerza una extensión que comprometería la validez de la caracterización ya obtenida.

### 2.3.6. Matriz experimental y control de sesgos

La matriz experimental se define como el producto cartesiano entre las cargas del catálogo, los estados de frecuencia y las repeticiones. Los niveles fijos se expresan como fracciones del rango real detectado en tiempo de ejecución, y no como valores absolutos de frecuencia, con el fin de que la misma definición de campaña sea aplicable a plataformas con rangos distintos.

Este trabajo emplea dos rejillas de frecuencia con propósitos distintos, y conviene distinguirlas desde el principio para no leer una como versión incompleta de la otra. La rejilla de la Tabla 2.4 — seis niveles — gobierna la campaña de caracterización descrita en esta fase, cuyo objetivo es establecer la forma de la respuesta de cada carga al escalado de frecuencia; para determinar si esa respuesta es monótona, dónde se sitúa su óptimo y cuál es el rango dinámico de potencia del dispositivo, seis puntos bastan y añadir más solo multiplica el costo de la campaña sin cambiar la conclusión. La rejilla de la Tabla 2.5 — ocho niveles, de paso más fino — gobierna la campaña del catálogo dual descrita en la sección 2.4.3, cuyo objetivo es distinto: allí cada nivel es una *acción* entre las que el modelo debe elegir, y el espacio de acciones determina cuán fina puede ser la decisión que el sistema es capaz de emitir. Una rejilla gruesa limitaría al selector por construcción, con independencia de su calidad.

Tabla 2.4: Estados de frecuencia de la campaña de caracterización (Fase 1).

| Nivel | Definición                            | Propósito                       |
|-------|---------------------------------------|---------------------------------|
| REF   | Gobernador nativo, sin control manual | Línea base de comparación       |
| F0    | Extremo superior del rango            | Límite de rendimiento y consumo |
| F1    | 75 % del rango                        | Punto intermedio alto           |
| F2    | 50 % del rango                        | Punto intermedio central        |
| F3    | 25 % del rango                        | Punto intermedio bajo           |
| F4    | Extremo inferior del rango            | Límite inferior                 |

Tabla 2.5: Rejilla fina de frecuencia empleada como espacio de acciones del selector. Las fracciones se aplican sobre el rango real de cada dispositivo, por lo que los mismos identificadores designan frecuencias absolutas distintas en la CPU y en el acelerador.

| Nivel | Fracción del rango | CPU (MHz) | GPU (fracción) |
|-------|--------------------|-----------|----------------|
| REF   | Gobernador nativo  | —         | Nativo         |
| F0    | 1.000              | 3200      | 1.000          |
| F1    | 0.833              | 2800      | 0.833          |
| F2    | 0.667              | 2400      | 0.667          |
| F3    | 0.500              | 2000      | 0.500          |
| F4    | 0.333              | 1600      | 0.333          |
| F5    | 0.167              | 1200      | 0.167          |
| F6    | 0.000              | 800       | 0.000          |

Las frecuencias de CPU de la Tabla 2.5 no son las que resultarían de aplicar directamente las fracciones al rango nominal de la plataforma, y la diferencia no es cosmética. El controlador de frecuencia de este procesador solo admite objetivos alineados a un escalón de 100 MHz; una fracción intermedia produce en general un valor no alineado, que el controlador rechaza o redondea silenciosamente, en cuyo caso el nivel solicitado y el aplicado dejan de coincidir sin que ninguna verificación agregada lo advierta. Por esa razón los niveles intermedios se redondean explícitamente al escalón de 100 MHz más próximo antes de solicitarse, mientras que los dos extremos conservan sin redondear los valores mínimo y máximo que el propio nodo reporta, dado que esos extremos no son necesariamente múltiplos del escalón en toda plataforma. Esta corrección se identificó al fallar tres campañas consecutivas sobre el mismo nivel intermedio, y su ausencia no producía datos erróneos sino la detención de la campaña, que es el modo de falla preferible entre los dos.

Para reducir el sesgo experimental se adoptan cinco medidas. Primera, cada combinación se ejecuta en múltiples repeticiones independientes, entendidas como relanzamientos completos del binario y no como iteraciones internas, de modo que cada repetición parta de un estado de caché y de predictores equivalente. Segunda, el orden de ejecución de las combinaciones se aleatoriza a partir de una semilla registrada en el manifiesto, con el fin de evitar que una deriva temporal — por ejemplo, térmica — se confunda con el efecto de la carga o de la frecuencia; el registro de la semilla preserva la reproducibilidad del orden. Tercera, las condiciones de afinidad, número de hilos y asignación de núcleos se fijan explícitamente y de forma idéntica para todas las corridas, en lugar de heredarse del entorno. Cuarta, al concluir la campaña se verifica que el estado de frecuencia del sistema haya sido restaurado a su configuración original, y este mismo mecanismo de restauración se registra para ejecutarse también ante una terminación anómala.

Quinta, la matriz de frecuencia fija exige deshabilitar, además de acotar el rango por núcleo, el mecanismo de aceleración dinámica del procesador (*Turbo Boost*). Escribir un rango estrecho de frecuencia por núcleo no basta por sí solo para fijar el reloj físico bajo carga: mientras la aceleración dinámica permanece activa a nivel de plataforma, el gestor de energía del procesador puede seleccionar autónomamente un reloj superior al rango declarado, de modo que un nivel nominalmente fijo — por ejemplo, F2 en la Tabla 2.4 — no garantiza por sí solo que el núcleo haya operado efectivamente a ese reloj durante la carga. Por esta razón, la aceleración dinámica se deshabilita a nivel de plataforma antes de aplicar cualquier nivel fijo de la matriz, su desactivación efectiva se verifica por relectura antes de iniciar la medición, se comprueba que no haya cambiado de estado durante toda la ejecución de la carga, y se restaura a su configuración original al finalizar — bajo el mismo principio de verificación por relectura y restauración garantizada, incluso ante terminación anómala, que ya rige la cuarta medida. La verificación de que el reloj efectivamente observado coincide con el nivel solicitado, dentro de una tolerancia declarada, se aplica sobre la lectura de cada núcleo delegado por separado (Tabla 2.2) y no únicamente sobre un núcleo representativo. Esta verificación distingue dos niveles de exigencia, de acuerdo con lo que cada tipo de anomalía realmente indica. La integridad estructural de la traza — ausencia de una lectura, un número de núcleos leídos distinto del delegado, o un núcleo representativo inconsistente entre lecturas consecutivas — sigue rechazando la corrida completa: esa condición evidencia que el propio mecanismo de lectura falló, no que el reloj se haya desviado, y ninguna ventana individual puede compensar esa pérdida de trazabilidad. La desviación respecto a la tolerancia declarada, en cambio, se evalúa y anota por ventana individual, según la taxonomía de la Tabla 2.6, en lugar de rechazar la corrida completa ante la primera lectura fuera de tolerancia. Esta distinción responde a un defecto identificado empíricamente en el criterio anterior: cuando

los núcleos delegados se desvían por igual entre sí — por ejemplo, ante una dilución del reloj estimado durante un tramo de inactividad sincronizada de todos los hilos, y no ante una falla de actuación genuina —, ningún criterio basado en la dispersión entre núcleos detecta la anomalía, mientras que un criterio agregado que rechaza la corrida entera ante una sola lectura fuera de rango sí la penaliza, aun cuando el resto de sus ventanas cumplió el objetivo sin incidente. La anotación por ventana evita ambos extremos: ninguna desviación queda oculta, y ninguna corrida se pierde por completo a causa de una fracción ínfima de sus ventanas.

Tabla 2.6: Taxonomía de clasificación de frecuencia por ventana de muestreo.

| Anotación                         | Condición                                                                                                                                                                                                                              |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Válida                            | Todos los núcleos delegados leídos en la ventana caen dentro de la tolerancia declarada respecto al reloj efectivamente aplicado.                                                                                                      |
| No confiable                      | Al menos un núcleo delegado se aparta de la tolerancia, fuera del margen de gracia posterior a una transición de nivel.                                                                                                                |
| No verificable (margen de gracia) | La ventana cae dentro del margen de exclusión inmediatamente posterior a una transición de nivel, donde el transitorio físico de conmutación aún no se ha resuelto; la desviación, si la hay, no se atribuye a una falla de actuación. |
| No aplicable (gobernador nativo)  | La corrida se ejecuta bajo el gobernador nativo de la plataforma (nivel de referencia), sin un objetivo de frecuencia fija contra el cual evaluar la tolerancia.                                                                       |

Esta anotación se conserva como columna independiente de la calidad general de la ventana descrita en la Tabla 2.7: una ventana puede ser válida en calidad general y, simultáneamente, no confiable en frecuencia, o viceversa. Solo las ventanas anotadas como válidas o no aplicables cuentan hacia el mínimo de ventanas exigido para aceptar la corrida; las anotadas como no confiables o no verificables se conservan íntegras en el conjunto de datos, con su anotación, pero no contribuyen a ese mínimo. Adicionalmente, cada corrida produce un resumen de cobertura de frecuencia — fracción de ventanas válidas y longitud de la racha más larga de ventanas no confiables consecutivas — que no condiciona por sí solo la aceptación, pero permite auditar después, corrida por corrida, cuánta de su duración total quedó efectivamente verificada en frecuencia.

### 2.3.7. Post-procesamiento y control de calidad de los datos

Las muestras crudas se transforman en el conjunto de datos final mediante un procedimiento de diferenciación entre lecturas consecutivas, conforme a lo descrito en la sección 1.1.3. Cada fila resultante representa una ventana de ejecución e incorpora, además de las métricas derivadas, un conjunto de campos de trazabilidad que permiten reconstruir su procedencia: identificador de corrida, carga, nodo, estado de frecuencia solicitado y observado, referencias a los archivos de calibración empleados y suma de verificación del binario ejecutado.

Un elemento central de este procedimiento es que ninguna observación se descarta silenciosamente. Cada ventana recibe una anotación de calidad que describe su condición, según la taxonomía de la Tabla 2.7, y se conserva en el conjunto de datos con dicha anotación. Esta decisión responde a un criterio de transparencia: el filtrado posterior es una decisión del análisis, no del instrumento, y la proporción de ventanas en cada categoría constituye en sí misma un indicador de la calidad de la campaña.

Entre estas categorías, la exclusión por calentamiento delimita explícitamente, para cada carga del catálogo, un intervalo inicial de la corrida declarado como transitorio de arranque, correspondiente al período en que la ejecución aún no alcanza un comportamiento estacionario — por ejemplo, mientras se completan la asignación de memoria, la creación de hilos y la sincronización inicial de la carga. Las ventanas cuya marca temporal cae dentro de ese intervalo se anotan como excluidas por calentamiento y no se emplean en el análisis, de modo que la región efectivamente caracterizada corresponda a la fase repetitiva del kernel y no a su arranque. A diferencia de otras anotaciones de la Tabla 2.7, que se derivan de una condición verificable en la propia ventana, la duración de este intervalo no se fija arbitrariamente por carga ni se estima a partir de una expectativa general de arranque: se determina mediante un procedimiento estadístico aplicado sobre la serie de telemetría de cada corrida, descrito a continuación.

Un aspecto favorable para la aplicación de este procedimiento es que la duración del calentamiento nunca condiciona la adquisición: el intervalo declarado se emplea exclusivamente como criterio de filtrado durante el post-procesamiento, y no se comunica en ningún momento al ejecutor de la capa C++, de modo que toda corrida ya adquirida contiene, desde el primer instante, la totalidad del transitorio de arranque real, con independencia del valor de calentamiento vigente al momento de adquirirla. Esta propiedad permite recalcular y ajustar el criterio de calentamiento de forma retroactiva sobre corridas ya realizadas, sin necesidad de una nueva adquisición.

El procedimiento se desarrolla en dos etapas, aplicadas sobre una señal representativa



del estado de carga del dispositivo — instrucciones por ciclo en el caso de CPU, utilización del dispositivo reportada por la interfaz de gestión de GPU en el caso de GPU —, calculada sobre ventanas móviles consecutivas de tamaño fijo.

En la primera etapa se aplica un criterio de estabilidad basado en el coeficiente de variación (CV %) de dicha señal, siguiendo el principio de evaluación estadísticamente rigurosa de desempeño propuesto por Georges et al. [8]: se declara alcanzado el fin del transitorio de arranque en el primer instante a partir del cual el CV % se mantiene por debajo de un umbral en dos ventanas consecutivas — exigencia de dos ventanas, y no de una sola, para no confundir el fin del arranque con una fluctuación transitoria dentro de una fase todavía inestable, ni con un cambio de fase legítimo del propio kernel que ocurra más adelante en la corrida —, y al punto detectado se le aplica un margen de seguridad multiplicativo antes de declararlo como intervalo de calentamiento definitivo. Una limitación de este criterio, identificada durante su verificación empírica sobre la señal de utilización de GPU, merece señalarse explícitamente: dado que el coeficiente de variación se define como el cociente entre la desviación estándar y la media, una ventana cuya señal permanece en cero satisface trivialmente el umbral, de modo que el criterio, aplicado sin más, puede declarar estable el reposo inicial del dispositivo — previo a que exista actividad real — en lugar de la fase de carga sostenida que se pretende delimitar, precisamente el resultado opuesto al buscado. Este modo de falla se corrige exigiendo, adicionalmente al umbral de CV %, que la media de la ventana supere un piso mínimo de actividad antes de aceptar la ventana como estable.

En la segunda etapa, reservada para las cargas donde el criterio de CV % no logra identificar ningún punto que satisfaga la condición anterior — un modo de falla conocido de este tipo de heurística de umbral fijo —, se aplica como respaldo un método de detección de puntos de cambio (*change point detection*) sobre la misma señal: en lugar de asumir que el estado estable es plano dentro de un umbral, el método busca recursivamente la partición de la serie que más reduce la suma de errores cuadráticos dentro de cada segmento resultante, en la línea de los métodos empleados para caracterizar estadísticamente el comportamiento de arranque de máquinas virtuales [4]. Cuando la serie presenta más de un punto de cambio — por ejemplo, una fluctuación breve y espuria previa al inicio real de la carga sostenida —, se descarta el criterio de aceptar sin más el primer punto detectado, y se selecciona en su lugar el primer segmento cuya media alcanza una fracción declarada de la meseta de mayor carga observada en toda la corrida, de modo que una fluctuación que no llega a sostenerse no se confunda con el fin genuino del arranque.

Las dos etapas anteriores operan sobre una única señal por dispositivo y, por construcción, no pueden distinguir por sí solas una transición genuina de una fluctuación que

coincide por casualidad con el criterio de aceptación. Por esta razón, el valor de calentamiento adoptado en el catálogo para cada carga de GPU se sometió, durante su calibración, a una verificación adicional aplicada por inspección directa de la traza cruda de potencia reportada por la interfaz de gestión del dispositivo: se confirmó que el intervalo propuesto por alguna de las dos etapas coincidiera con un salto identificable de potencia, del régimen de reposo al régimen de carga sostenida, y no únicamente con la señal primaria empleada para la detección. Esta verificación se aplicó, sin excepción, a las cargas de GPU recalibradas conforme a este criterio; no constituye una etapa adicional codificada dentro del procedimiento estadístico descrito arriba — que sí se ejecuta de forma sistemática, sin intervención, sobre toda carga del catálogo — sino un contraste puntual realizado durante la calibración, cuyo resultado se fijó como valor final una sola vez. Para tres cargas de GPU con unidades individuales de trabajo demasiado breves para que la cadencia de muestreo del dispositivo resolviera su actividad, ninguna transición de potencia resultó identificable a lo largo de la corrida, con independencia de cuánto se alargó su duración total conforme al criterio de la sección 2.3.5; en esos casos se concluyó que la señal disponible no sostenía una medición confiable del intervalo de calentamiento, el valor se conservó en su configuración por defecto, y la imposibilidad de medirlo se documentó explícitamente como evidencia negativa en el catálogo, en lugar de forzar un valor no respaldado por la medición. Ninguna carga de CPU fue sometida a este contraste: la señal de potencia de paquete del procesador no se empleó con este propósito en el desarrollo de este trabajo, de modo que el intervalo de calentamiento de las cargas de CPU descansa únicamente en las dos etapas estadísticas descritas arriba.

Tabla 2.7: Taxonomía de anotaciones de calidad por ventana de muestreo.

| Anotación                  | Condición                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------|
| Válida                     | La ventana satisface todos los criterios de calidad.                                                          |
| Sin diferencia previa      | Primera muestra de la corrida; carece de lectura anterior contra la cual diferenciar.                         |
| Excluida por calentamiento | La ventana pertenece al intervalo inicial declarado como transitorio de arranque.                             |
| Contadores degradados      | Diferencia negativa, valor ausente, o fracción de tiempo contando inferior al umbral admitido.                |
| Intensidad indefinida      | No fue posible calcular la intensidad operacional, por ausencia de trabajo reportado o de tráfico observable. |
| Energía inválida           | La lectura energética correspondiente no es válida en una campaña que la requiere.                            |
| Sin lectura de frecuencia  | No se dispone de la frecuencia efectivamente observada durante la ventana.                                    |

De forma complementaria, cada corrida completa se somete a una validación posterior que examina su integridad global: ausencia de pérdidas en la estructura de muestreo, correspondencia de la suma de verificación del binario, cumplimiento del criterio de éxito propio de la carga y unicidad del identificador de corrida. Una corrida que no supere esta validación se registra como rechazada, conservando sus artefactos para inspección, en lugar de eliminarse.

### 2.3.8. Medición directa de FLOPs por ventana y su validación empírica

El diseño original de este instrumento estimaba los FLOPs de cada ventana mediante un prorrateo del total reportado por el propio programa, en proporción a las instrucciones retiradas de cada ventana, bajo el supuesto de que la densidad de operaciones de punto flotante por instrucción retirada se mantiene aproximadamente constante dentro de una misma corrida. Dicho supuesto no se dio por válido sin verificación. Antes de adoptarlo como definitivo, se evaluó si la plataforma experimental permitía sustituirlo por una medición directa, siguiendo el mismo criterio de no asumir capacidades del hardware sin confirmarlas empíricamente que rige el resto de este instrumento (secciones 1.1.5 y 1.1.3). Esta sección documenta ese procedimiento de verificación y su resultado; el prorrateo original, descartado tras esta validación, queda documentado como antecedente metodológico en el registro de cambios del

proyecto (ARC-27.1), no en el instrumento.

**Encoding y ponderación.** Para la plataforma experimental (Intel Xeon Gold 5315Y, Ice Lake-SP), la familia de eventos `FP_ARITH_INST_RETIRED` utiliza el *EventSelect* `0xC7`; cada modalidad aritmética se distingue mediante su *Unit Mask* (`UMask`), de modo que el encoding completo de cada sub-evento no se reduce a `0xC7` por sí solo. Los cuatro sub-eventos de doble precisión empleados se distinguen por el ancho del registro vectorial — escalar, y empaquetado de 128, 256 y 512 bits — mediante los `UMask` `0x01`, `0x04`, `0x10` y `0x40` respectivamente, sin alias simbólico expuesto por el kernel de Linux en este nodo, por lo que se programan como evento crudo mediante `perf_event_open`. Estos eventos contabilizan instrucciones retiradas, no FLOPs de forma abstracta: convertir su conteo requiere el número de elementos de doble precisión por instrucción (1, 2, 4 y 8 para escalar/128/256/512 bits) y el tipo de operación. Las instrucciones *fused multiply-add* (FMA) no exigen una ponderación adicional: la propia definición del evento hace que una FMA incremente el contador en 2 por elemento en vez de 1, precisamente para que la cuenta ya represente FLOPs y no instrucciones en bruto [18] (Tabla 2.8). El total de FLOPs medidos en una ventana se obtiene entonces ponderando cada sub-evento únicamente por su número de elementos:

$$\text{FLOPs}_i^{\text{HW}} = 1 \cdot \Delta N_{\text{escalar},i} + 2 \cdot \Delta N_{128,i} + 4 \cdot \Delta N_{256,i} + 8 \cdot \Delta N_{512,i} \quad (2.1)$$

Tabla 2.8: FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento `FP_ARITH_INST_RETIRED` de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.

| Ancho vectorial | Elementos FP64 | FLOPs (ADD/MUL) | FLOPs (FMA) |
|-----------------|----------------|-----------------|-------------|
| Escalar         | 1              | 1               | 2           |
| 128 bits        | 2              | 2               | 4           |
| 256 bits        | 4              | 4               | 8           |
| 512 bits        | 8              | 8               | 16          |

El encoding se determinó a partir de la definición oficial de Intel para esta microarquitectura [18], corroborada contra la tabla de eventos de LIKWID, que documenta el grupo `FLOPS_DP` con esta misma ponderación para Ice Lake-SP [36], en lugar de inferirse a partir de documentación genérica de otras microarquitecturas.

**Diseño de la verificación.** Se construyó un instrumento de prueba independiente del harness de producción, con el mismo protocolo de atribución por PID e `inherit=1` descrito en la sección 1.1.5, capaz de abrir simultáneamente los diez contadores que componían el

presupuesto inicial: los seis entonces establecidos (sección 1.1.3) más los cuatro sub-eventos de `FP_ARITH_INST_RETIRED` de la ecuación (2.1). Este instrumento se ejecutó contra dos kernels de régimen opuesto del catálogo experimental: uno intensivo en cómputo (multiplicación de matrices densas, con un volumen de trabajo conocido analíticamente) y uno intensivo en memoria (un kernel multi-hilo de la suite NAS Parallel Benchmarks), en ambos casos bajo la afinidad de núcleos real de la campaña.

**Resultados.** Para el kernel de cómputo denso, cuyo total de FLOPs se conoce exactamente por construcción ( $2 \cdot \text{iteraciones} \cdot n^3$ ), la ecuación (2.1) reprodujo ese total con un error relativo de 0.29 % sin afinidad fijada y de 0.30 % bajo la afinidad real de campaña. Para el kernel intensivo en memoria, el contraste se hizo contra el total de FLOPs que el propio kernel reporta al finalizar, arrojando un error relativo de 7.48 %; este valor es mayor que el del primer kernel, pero es consistente con una causa identificada y ajena al contador: el tiempo que dicho kernel cronometra internamente cubre únicamente su ciclo de cómputo principal y excluye la fase final de verificación del resultado, la cual también ejecuta operaciones de punto flotante que el contador de hardware sí registra por abarcar el proceso completo. En ambos casos, los diez contadores abrieron simultáneamente sin evidencia de multiplexación (razón tiempo-contando sobre tiempo-habilitado igual a 1 en los diez), confirmando que esa combinación era sostenible bajo el estado del nodo observado en aquella verificación. Una prueba de humo posterior encontró que la activación del vigilante NMI del sistema operativo había reducido el presupuesto efectivo; la configuración de producción se ajustó entonces a nueve eventos retirando `L2_LINES_IN_ALL`, sin eliminar ninguno de los cuatro sub-eventos que sostienen la medición de FLOPs.

**Confirmación a escala de campaña.** Con la verificación anterior favorable, la medición directa se integró al harness de producción como única fuente de FLOPs por ventana — sin conservar el prorrato original como respaldo, una vez confirmado que no aportaba nada en la plataforma y el alcance de este trabajo (sección 1.1.2) —, y se ejecutó una campaña real completa: siete kernels del catálogo, matriz declarada a seis estados de frecuencia (REF y F0-F4), tres repeticiones por combinación, 66 corridas aceptadas o rechazadas por los criterios de la sección 2.3.7 y siguientes. Al momento de esta campaña, el nodo aún no tenía concedido el permiso administrativo de escritura de frecuencia, de modo que los seis niveles se ejecutaron en la práctica sobre la frecuencia nativa del procesador; esto no compromete la validación de la medición de FLOPs, que ocurre por ventana con independencia del estado de frecuencia, pero significa que esta campaña específica no debe citarse como evidencia de que el control de frecuencia (DVFS) en sí mismo fue ejercitado. Sobre aproximadamente 1.29 millones de

ventanas con delta válido en esa campaña, el 100 % obtuvo su valor de FLOPs mediante la ecuación (2.1), y ninguna ventana resultó con los contadores de punto flotante degradados. Este resultado, a una escala varios órdenes de magnitud mayor que la verificación inicial de dos kernels, respalda que la disponibilidad y estabilidad del encoding no es un caso aislado favorable, sino una propiedad sostenida de la combinación plataforma-instrumento descrita en este capítulo.

**Alcance de esta validación.** Los resultados anteriores se verificaron específicamente para la microarquitectura Ice Lake-SP de la plataforma experimental (Tabla 2.1), gateados por familia y modelo de procesador en el mismo mecanismo que protege los demás eventos crudos de este instrumento (sección 1.1.3); no se extrapolan a otro hardware sin repetir este procedimiento. Tampoco se reclama que el error relativo observado (0.29–7.48 %) sea una cota universal: ambos valores dependen del kernel evaluado y, en el segundo caso, de una particularidad de cómo ese kernel específico delimita su propio tiempo de ejecución. Dado que este trabajo no reclama portabilidad más allá de la plataforma descrita, y que la medición directa estuvo disponible sin excepción en las 66 corridas de la campaña real, el instrumento no conserva ningún mecanismo de respaldo para un nodo hipotético donde este encoding no se haya validado: una ventana en esa situación queda con intensidad operacional indefinida, no con un valor estimado.

### 2.3.9. Reproducibilidad

Toda campaña queda definida por un manifiesto declarativo bajo control de versiones, que especifica las cargas, los estados de frecuencia, el número de repeticiones, el período de muestreo, la asignación de núcleos, la semilla de aleatorización y las referencias de calibración. Junto con los datos, cada campaña persiste el perfil de capacidades detectado en el nodo, los resultados de calibración y los metadatos de cada corrida. El instrumento de medición, el orquestador y las definiciones de campaña se mantienen en un repositorio público, de modo que la campaña sea reproducible a partir de los artefactos publicados.

Dado que la duración total de la matriz completa excede lo que una única asignación del gestor de recursos puede sostener de forma continua, el protocolo contempla explícitamente la posibilidad de reanudar una campaña interrumpida sin comprometer su validez. Dos garantías sostienen esta reanudación. Primera, una huella del protocolo experimental — que cubre, entre otros campos, las cargas declaradas, los estados de frecuencia, la semilla, la política de muestreo y la suma de verificación del propio instrumento — se persiste junto con los primeros resultados y se contrasta contra la del manifiesto vigente en cada reanudación;

una discrepancia detiene la campaña en lugar de mezclar en el mismo conjunto de datos corridas producidas bajo protocolos distintos. Segunda, ante una reanudación genuina — reconocida por la presencia de corridas o de artefactos de calibración previos, incluso si la interrupción ocurrió antes de completar la primera combinación — la calibración no se repite: se recupera del disco la ya obtenida en la sesión anterior, verificando su presencia y validez para cada estado de frecuencia declarado antes de continuar, en lugar de remedirla en silencio y desplazar con ello el punto de inflexión que clasifica las corridas ya aceptadas.

Como verificación adicional de consistencia, toda vez que se modifica un parámetro de ejecución de una carga del catálogo — por ejemplo, a raíz de un ajuste realizado conforme a lo descrito en la sección 2.3.5 — se ejecuta una corrida completa de la campaña que incluya dicha carga, contrastando que la etiqueta derivada, la intensidad operacional y el punto de inflexión empleado resulten idénticos entre las distintas repeticiones independientes de esa misma carga. Esta comparación no sustituye la verificación de calidad por ventana descrita en la Tabla 2.7, sino que actúa como una verificación de nivel superior: una discrepancia entre repeticiones nominalmente idénticas indicaría una fuente de no determinismo en el instrumento o en la propia carga, y no únicamente un artefacto puntual de una corrida concreta.

## 2.4. Fase 2: Construcción del conjunto de datos y del modelo de selección

Esta fase construye el modelo que decide, para cada operación que la aplicación va a despachar, bajo qué configuración conviene ejecutarla. Se describe primero qué se decide y sobre qué unidad, después cómo se adquieren y se transforman los datos que sostienen esa decisión, y finalmente cómo se entrena y se valida el modelo. El orden no es arbitrario: cada una de las decisiones de diseño de las secciones posteriores queda determinada por las dos definiciones que se establecen a continuación.

### 2.4.1. Unidad de decisión y su correspondencia con los objetivos

La unidad sobre la que el modelo emite una predicción es **una llamada a una operación**: una fase de una aplicación HPC compuesta por varias de ellas. No es una ventana de muestreo, ni el programa completo tomado como un solo bloque.

Esta elección es la lectura más literal del segundo objetivo específico, que exige clasificar “las fases de ejecución de las aplicaciones”. El objetivo no prescribe una unidad temporal —

no habla de ventanas ni de intervalos —, y sí exige dos propiedades que esta formulación conserva íntegras: que la identificación ocurra en tiempo de ejecución y que la latencia de inferencia sea baja. El tercer objetivo específico refuerza la misma lectura al pedir políticas *proactivas* en función de la fase inferida: consultar al modelo antes de despachar la operación es proactivo en sentido estricto, a diferencia de una política que observe la ejecución ya en curso y reaccione a ella.

Dos líneas de evidencia sostienen esta unidad. La primera es propia y física: el punto de inflexión del modelo Roofline es el cociente entre el techo de cómputo y el de ancho de banda, y el techo de cómputo depende del reloj de núcleo, de modo que ese punto de inflexión no es una constante de la plataforma sino una función del punto de operación; la pertenencia de una carga a un régimen deja entonces de ser una propiedad intrínseca de la carga para pasar a serlo del par carga–frecuencia, y una etiqueta de régimen, por tanto, no puede ser un objetivo de aprendizaje estable. La segunda es ajena y anterior: los tres sistemas publicados que este trabajo toma como referencia deciden, respectivamente, sobre la aplicación [12], sobre el programa completo tras haber probado y descartado explícitamente el ajuste función por función [5], y sobre el trabajo completo en producción [3].

Conviene precisar por qué la unidad no es un intervalo temporal más fino. Una decisión de frecuencia o de dispositivo emitida a escala de milisegundos exigiría que el comportamiento variara apreciablemente a esa escala dentro de una misma operación; a eso se añade que el costo de conmutar el punto operativo no es despreciable frente a un intervalo de esa duración, de modo que una política a esa escala pagaría las transiciones sin amortizarlas [13, 37].

### 2.4.2. Variable objetivo: la configuración de mínimo EDP

La variable que el modelo predice no es una etiqueta de régimen sino, directamente, la configuración que minimiza el Producto Energía–Retardo para la operación en cuestión:

$$c^*(o) = \arg \min_{(d, f) \in \mathcal{D} \times \mathcal{F}_d} \text{EDP}(o, d, f), \quad (2.2)$$

donde  $o$  es la operación con su tamaño de problema,  $\mathcal{D} = \{\text{CPU}, \text{GPU}\}$  es el conjunto de dispositivos y  $\mathcal{F}_d$  el conjunto de estados de frecuencia disponibles en el dispositivo  $d$ .

Predecir directamente  $c^*$  en lugar de una etiqueta intermedia responde a una razón concreta y no a una simplificación. La distinción *compute-bound/memory-bound* nunca fue el fin del sistema: era un *proxy* de la pregunta “qué punto operativo conviene”. Un proxy es defendible mientras su relación con la variable de interés sea estable, y esa relación no lo es



en este caso, porque el techo de cómputo que fija el punto de inflexión Roofline depende de la frecuencia de operación, de modo que el umbral que define la etiqueta se mueve con la propia variable que se pretende decidir. Eliminar el intermediario suprime esa fuente de inestabilidad y alinea el objetivo de entrenamiento con el cuarto objetivo específico, que evalúa el sistema precisamente en términos de EDP.

La formulación se plantea como clasificación sobre un conjunto discreto de configuraciones, coherente con el hecho de que el hardware solo admite un conjunto finito de estados y con el diseño de política discreta descrito en la Fase 3. Como variante analizable sobre el mismo conjunto de datos, y sin reentrenamiento, se contempla la minimización de energía sujeta a un presupuesto explícito de degradación temporal; formular ese presupuesto como parámetro de la política y no como constante del entrenamiento hace explícito un supuesto que de otro modo quedaría implícito en los datos.

## Resolución de empates y estabilidad del óptimo

La ecuación (2.2) presupone que el mínimo es único, lo cual no está garantizado sobre cantidades medidas. El desempate se resuelve mediante un orden total declarado — menor EDP, después menor energía, después menor tiempo, y finalmente el identificador de la acción — de modo que la etiqueta sea determinista y reproducible, y no dependa del orden en que las filas lleguen al constructor.

Un empate exacto, sin embargo, es menos frecuente que un margen tan estrecho que no se distingue del ruido de medición. Por esa razón cada configuración registra, además de su etiqueta, el margen relativo de EDP entre la mejor y la segunda mejor acción, y una anotación de estabilidad que compara ese margen contra el error estándar combinado de ambas, estimado a partir de la dispersión entre repeticiones. Cuando el margen no supera esa incertidumbre, el óptimo se anota como **incierto**. Esas configuraciones no se excluyen ni se corrigen aumentando repeticiones de forma silenciosa: se conservan con su anotación, porque el hecho de que dos configuraciones sean indistinguibles es en sí mismo un resultado que el análisis debe poder leer.

### 2.4.3. Campañas de adquisición del catálogo dual

El conjunto de datos que sostiene la ecuación (2.2) exige medir cada configuración bajo *todas* las acciones candidatas, y no solo bajo aquella que se sospecha óptima: un argmín sobre un subconjunto de alternativas no es el mínimo del problema. Esa exigencia determina el tamaño de las campañas.

## Rejilla de tamaños

Cada una de las seis operaciones del catálogo dual se instancia sobre una rejilla de tamaños de problema. Las operaciones de estructura matricial o de malla — multiplicación densa, transformada rápida de Fourier, esquema de vecindad sobre malla regular y factorización de Cholesky — emplean trece tamaños entre 64 y 4096, con razón aproximada de 1.5 entre tamaños consecutivos y mayor densidad en la banda de 256 a 512, donde el tamizaje preliminar situó la frontera de conveniencia entre dispositivos. Las operaciones de estructura vectorial — combinación lineal de vectores y producto matriz dispersa–vector — emplean ocho tamaños entre  $10^4$  y  $3,16 \times 10^7$ , con razón aproximada de  $\sqrt{10}$ . El techo de la rejilla vectorial se recortó desde  $10^8$  al comprobarse que la representación dispersa a ese tamaño exigía del orden de 10 GB solo en memoria del anfitrión, por encima del presupuesto asignado a las campañas. El total es de 68 configuraciones distintas.

## Calibración del número de iteraciones por dispositivo

Cada corrida ejecuta la operación un número de veces determinado por el catálogo, de modo que su duración total sea suficiente para producir varias ventanas de muestreo. Ese número no puede ser común a ambos dispositivos: la misma operación al mismo tamaño tarda órdenes de magnitud distintos en la CPU y en el acelerador, y un número único produciría o bien corridas de GPU demasiado breves para muestrear, o bien corridas de CPU de duración inaceptable.

El número de iteraciones se calibra por tanto de forma independiente para cada dispositivo, con procedimientos distintos porque la información disponible es distinta en cada caso. Para la CPU se emplea una tabla empírica de la mediana del tiempo por iteración medida en cada uno de los 68 pares (operación, tamaño), obtenida de la campaña de CPU ya ejecutada; no hay extrapolación, porque la rejilla es finita y fue medida por completo. Para el acelerador se emplea un modelo de dos términos,

$$t_{\text{iter}}(N) = A + B \cdot f(N), \quad (2.3)$$

ajustado por mínimos cuadrados sobre las corridas reales aceptadas de un pase preliminar, donde  $f(N)$  es la función de escalado asintótico de la operación. El término constante  $A$  representa el costo fijo por despacho — lanzamiento del núcleo de cómputo y sincronización —, y el término  $B \cdot f(N)$  el costo que escala con el tamaño. Un modelo de un solo término, ajustado desde un único tamaño de referencia, sobreestima el número de iteraciones en los tamaños pequeños por varios órdenes de magnitud, precisamente porque atribuye al cómputo

un tiempo que en realidad consume el despacho.

De esta calibración independiente se sigue una consecuencia que condiciona todo el procesamiento posterior: las dos variantes de una misma configuración ejecutan números de iteraciones distintos, con razones que en el catálogo vigente van desde 0.2 hasta 36 veces. Sus totales de tiempo y energía no son, por tanto, directamente comparables, y compararlos sin normalizar invalidaría la etiqueta. La normalización correspondiente se describe en la sección 2.4.5 y es un paso obligatorio del constructor, verificado por prueba automatizada.

### Matriz de las dos campañas

La Tabla 2.9 resume ambas campañas. La asimetría entre ellas es deliberada. En la campaña del eje de CPU el acelerador permanece ocioso, y su reloj no es un factor: basta recorrer la rejilla fina de la CPU. En la campaña del eje del acelerador, en cambio, el reloj de la CPU sí afecta al despacho — el anfitrión prepara las transferencias y sincroniza —, de modo que la acción es un par ordenado de niveles, uno por dispositivo. Recorrer las dos rejillas finas completas daría  $8 \times 8 = 64$  acciones por configuración, un costo combinatorio que la campaña no puede absorber; el eje de la CPU se reduce por ello a cuatro niveles — el nativo, ambos extremos y un punto central —, elegidos porque el tamizaje preliminar mostró que la respuesta del despacho al reloj del anfitrión tiene forma de meseta seguida de penalización creciente, y cuatro puntos bastan para capturar esa forma sin asumir linealidad.

Tabla 2.9: Matriz experimental de las dos campañas del catálogo dual.

| <b>Factor</b>                               | <b>Eje de CPU</b> | <b>Eje del acelerador</b> |
|---------------------------------------------|-------------------|---------------------------|
| Configuraciones (operación $\times$ tamaño) | 68                | 68                        |
| Niveles de frecuencia de CPU                | 8                 | 4                         |
| Niveles de frecuencia de GPU                | —                 | 8                         |
| Acciones por configuración                  | 8                 | 32                        |
| Repeticiones por combinación                | 3                 | 3                         |
| <b>Corridas totales</b>                     | <b>1 632</b>      | <b>6 528</b>              |

El espacio de acciones sobre el que se resuelve la ecuación (2.2) es la unión de ambos: 8 acciones de CPU, denotadas por su nivel de reloj, y 32 acciones de acelerador, denotadas por el par de niveles de anfitrión y dispositivo — 40 acciones en total por configuración.

Tres parámetros de estas campañas requieren justificación explícita, por tratarse de decisiones con alternativas defendibles. El primero es el número de repeticiones. Se fija en

tres, y no en el valor de diez empleado en la campaña de caracterización de la Fase 1, porque el propósito es distinto: allí se estimaba la forma de una curva de respuesta y convenía reducir la incertidumbre de cada punto, mientras que aquí se comparan configuraciones entre sí y lo que importa es distinguir la mejor de la segunda. Tres repeticiones permiten estimar una dispersión — necesaria para la anotación de óptimo incierto de la sección 2.4.2 — y son el mínimo que lo permite; elevar ese número de forma uniforme multiplicaría el costo de una campaña ya extensa para reducir la incertidumbre también donde el margen es holgado y no aporta nada. La política de repeticiones adaptativa que se seguiría de este razonamiento — escalar únicamente donde el margen resulte estrecho — queda formulada como trabajo futuro y no se aplica aquí, de modo que el criterio permanece uniforme y por tanto auditable.

El segundo es la cadencia de muestreo, fijada en 1 ms para los contadores del procesador y 5 ms para la interfaz de gestión del acelerador. La diferencia responde al costo de cada consulta: la lectura de contadores del procesador es barata y local, mientras que la consulta al acelerador atraviesa el bus y su costo es varios órdenes de magnitud mayor, hasta el punto de perturbar el estado que pretende observar — efecto cuya magnitud se cuantifica en la sección 3.2. Una cadencia más fina en el acelerador aumentaría esa perturbación sin mejorar la integración, dado que el objetivo se construye sobre agregados de región y no sobre la forma instantánea de la señal.

El tercero es la medición del sobrecosto del instrumento. La caracterización de la sección 2.2.4 estableció que ese sobrecosto es del 1.95 % de media y estable entre cargas; una vez establecido, remedirlo en cada repetición de cada combinación no aporta información nueva. Estas campañas declaran por tanto el par de línea base únicamente sobre la primera repetición de cada combinación, como vigilancia contra una desviación futura del instrumento y no como remediación completa.

## **Ejecución por sesiones y reanudación**

La campaña del eje del acelerador excede lo que puede ejecutarse en una sola reserva del nodo compartido. Se ejecuta por tanto en sesiones sucesivas sobre un mismo directorio de salida, en el que el orquestador detecta las combinaciones ya aceptadas y las omite. La auditoría se aplica por sesión y no únicamente al final: un rechazo sistemático concentrado en un nivel debe detener la campaña y diagnosticarse, en lugar de relajar retrospectivamente los criterios para que los datos pasen. Los artefactos de calibración se miden en la primera sesión y las reanudaciones los cargan, fallando de forma cerrada si falta alguno, de modo que todas las sesiones compartan los mismos techos de referencia.

#### 2.4.4. Frontera de medición de la energía

Decidir entre dos dispositivos exige que la energía atribuida a cada uno se mida sobre la misma frontera. Si la energía de la CPU se contabilizara únicamente sobre sus propios dominios y la del acelerador incluyera además el consumo del anfitrión, la comparación estaría sesgada por construcción. La regla adoptada es por tanto simétrica: en ambos casos se contabiliza la energía del procesador — dominios de paquete y de memoria — **más** la del acelerador, con independencia de cuál de los dos ejecute la operación.

La aplicación de esa regla al eje de la CPU obligó a una decisión que conviene documentar, porque de otro modo el conjunto de datos contendría un artefacto del instrumento presentado como consumo de la carga. Durante la campaña del eje de CPU, en la que ninguna carga ejecuta código en el acelerador y se verificó la ausencia de procesos ajenos sobre él, la interfaz de gestión del dispositivo reportó una potencia media por corrida de entre 49.9 y 67.6 W (mediana 59.5 W), con el dispositivo declarando cero por ciento de utilización y un reloj de 1410 MHz. Una sonda independiente del mismo dispositivo verdaderamente en reposo, con 300 muestras durante 60 s y consultas espaciadas, midió en cambio 34.84 W de media, con máximo de 35.20 W, y observó el reloj en 210 MHz.

La interpretación que sostiene esa comparación es un **efecto del observador**: consultar la interfaz de gestión cada 5 ms impide que el dominio del acelerador alcance su estado profundo de reposo. La lectura no es falsa — el dispositivo consume realmente esa potencia en el estado que el propio muestreo provoca —, pero atribuir esa energía a la carga de CPU sí lo sería, porque una aplicación real que no instrumentara el acelerador no la pagaría. Por esa razón, para las corridas del eje de CPU la serie muestreada se conserva como evidencia del sobrecosto instrumental pero **no** se integra en el objetivo; en su lugar se imputa la línea base medida del dispositivo en reposo, multiplicada por la duración de la región:

$$E_{\text{CPU}} = E_{\text{RAPL, paquete+DRAM}} + P_{\text{reposo}} \cdot t_{\text{región}}, \quad P_{\text{reposo}} = 34,84 \text{ W}. \quad (2.4)$$

Para las corridas del eje del acelerador, en cambio, la serie sí se integra dentro de cada región, porque allí el dispositivo ejecuta realmente la operación y su potencia dependiente del nivel forma parte del candidato que se evalúa. El acelerador ocioso no se imputa como cero en ningún caso: hacerlo favorecería artificialmente a la CPU en la comparación, ocultando que en este nodo el dispositivo consume del orden de 35 W aun sin trabajo asignado. Cada fila del conjunto de datos registra explícitamente cuál de las dos reglas produjo su término de acelerador, junto con el valor de la línea base y la corrida que lo midió, de modo que ninguna cantidad quede sin procedencia declarada.

### 2.4.5. Estructura del conjunto de datos en dos niveles

El conjunto de datos se organiza en dos niveles, y la distinción entre ambos es central para no confundir la resolución con la que se *mide* con la resolución con la que se *decide*.

El **nivel 1** conserva una fila por ventana de muestreo, tal como la produce el instrumento descrito en la sección 2.2.4. Este nivel no alimenta directamente al modelo, pero su granularidad fina sigue siendo necesaria por dos razones independientes del aprendizaje: los contadores de energía RAPL son registros de 32 bits sujetos a desbordamiento, y solo el muestreo periódico con suma de diferencias permite integrar la energía correctamente a través de esos desbordamientos; y la depuración de calidad — exclusión de arranque, verificación de la frecuencia efectiva por ventana, razón de ocupación de los contadores — solo puede aplicarse a esa resolución.

El **nivel 2** contiene una fila por configuración candidata. Su construcción atraviesa cuatro pasos, cada uno de los cuales resuelve un problema concreto:

1. **Integración por región.** Tiempo y energía se acumulan sobre las ventanas que caen dentro de cada región del contrato temporal (sección 2.2.5). Puesto que las fronteras de la región son marcas absolutas y las ventanas tienen duración propia, una ventana puede solaparse parcialmente con la región; en ese caso contribuye en proporción a la fracción solapada, y no se incluye ni se descarta por completo. Las lecturas del acelerador persisten únicamente su marca final, de modo que el inicio de su intervalo se reconstruye con la marca de la muestra anterior, que es el intervalo al que corresponde el incremento de energía reportado.
2. **Normalización por despacho.** Los totales de la región caliente se dividen por el número de iteraciones efectivamente ejecutadas, obtenido del catálogo y no del registro del lanzador. La región fría contiene exactamente un despacho y no se divide. El objetivo se construye entonces sobre magnitudes por despacho,  $EDP_{\text{despacho}} = (E/n) \cdot (T/n)$ , únicas comparables entre dispositivos que ejecutan números distintos de iteraciones.
3. **Agregación de repeticiones.** Las tres repeticiones de cada combinación se promedian, conservando además desviación estándar, coeficiente de variación y extremos. Una combinación con menos repeticiones válidas de las exigidas se marca como no elegible y no participa en la construcción de la etiqueta.
4. **Unión de las variantes.** Las filas de CPU y de acelerador de una misma configuración se enlazan mediante el identificador compartido, lo que permite resolver la ecuación (2.2) sobre el producto completo de dispositivos y frecuencias.

Una consecuencia de este diseño debe declararse explícitamente, porque determina el tamaño efectivo de la muestra: el número de ejemplos de entrenamiento es el número de configuraciones distintas — 68 —, no el número de corridas ni el de ventanas. Presentar como tamaño muestral las decenas de miles de ventanas del nivel 1 sería incorrecto, dado que las ventanas de una misma corrida no son observaciones independientes entre sí ni constituyen decisiones distintas. Las 40 filas candidatas de una misma configuración tampoco lo son: describen alternativas de una única decisión, y por eso se mantienen siempre juntas en las particiones de validación.

### **Complejidad y falla cerrada**

El constructor verifica que cada configuración disponga de todas sus acciones candidatas con el número exigido de repeticiones aceptadas, y produce un informe explícito de la matriz esperada, la presente y la faltante. Si a una configuración le falta alguna alternativa, no se calcula un arg mín parcial sobre las disponibles: el constructor falla de forma cerrada. Un mínimo tomado sobre un subconjunto incompleto sería indistinguible, en el conjunto de datos resultante, de un mínimo genuino, y esa ambigüedad no se puede corregir después.

### **Resolución temporal de la región fría**

La región fría de una configuración pequeña puede ser más breve que el intervalo de muestreo, en cuyo caso su energía se obtiene por integración proporcional de una única ventana parcial. Esas filas se conservan — excluirlas sesgaría el conjunto justamente hacia las configuraciones grandes —, pero se anotan como estimaciones de baja resolución, y las métricas puntuales de telemetría que dependerían de una ventana completa quedan ausentes con su indicador correspondiente, en lugar de rellenarse con cero o con valores de la región caliente. La sensibilidad de la etiqueta a estas filas se cuantifica explícitamente y se reporta en la sección 3.5.

#### **2.4.6. Las dos estrategias de decisión**

El vector de características que el modelo puede recibir no es el mismo en todas las situaciones de despliegue, porque la información disponible antes de decidir depende de si la aplicación ya ejecutó algo o no. En lugar de adoptar un único vector y suponer una situación, se formulan dos estrategias que corresponden a dos momentos distintos, y se evalúan por separado.

## Estrategia sin ejecución previa

Corresponde a la primera aparición de una operación: no se ha ejecutado nada, ningún dispositivo está inicializado y no existe telemetría alguna sobre esa carga. La decisión debe tomarse exclusivamente a partir de lo que se conoce por inspección: qué operación es, sobre qué tamaño, y qué predice la teoría sobre su costo. Todos los candidatos se evalúan bajo el costo de la región fría, dado que ninguno encuentra su dispositivo preparado.

Su vector de entrada se compone de descriptores analíticos, calculados con las fórmulas cerradas de la Tabla 2.10 sin medir nada, y de descriptores de la propia acción candidata:

Tabla 2.10: Fórmulas analíticas de trabajo y de tráfico lógico por despacho, empleadas como descriptores estáticos.  $N$  es el parámetro de tamaño de la operación.

| Operación                       | FLOPs por despacho | Bytes lógicos por despacho |
|---------------------------------|--------------------|----------------------------|
| Multiplicación densa            | $2N^3$             | $24N^2$                    |
| Transformada de Fourier         | $5N^2 \log_2(N^2)$ | $32N^2$                    |
| Combinación lineal de vectores  | $2N$               | $24N$                      |
| Esquema de vecindad sobre malla | $5(N - 2)^2$       | $16N^2$                    |
| Factorización de Cholesky       | $N^3/3$            | $16N^2$                    |
| Matriz dispersa por vector      | $14N$              | $104N + 4$                 |

A partir de estas magnitudes se derivan el logaritmo del tamaño, los logaritmos del trabajo y del tráfico, y la intensidad aritmética analítica — su cociente —, que es el descriptor que sitúa la operación respecto del punto de inflexión del modelo Roofline sin necesidad de medirla. Se añaden los descriptores de la acción: el dispositivo candidato, la fracción de rango de reloj solicitada en cada dispositivo, indicadores de si el nivel corresponde al gobernador nativo, y un indicador de si la acción exige inicializar el dispositivo.

## Estrategia con sondeo

Corresponde a la situación posterior a una primera ejecución real: la aplicación ya despachó esa operación una vez, en un estado de referencia, y esa ejecución produjo telemetría que puede informar las decisiones siguientes sobre la misma operación. La primera ejecución no se desperdicia — es trabajo útil que la aplicación necesitaba de todos modos —, y su costo es precisamente lo que el umbral de amortización de la Fase 4 debe evaluar.

Al vector anterior se añade la telemetría agregada de esa única ejecución de sondeo, tomada de su región fría: tiempo y energía por despacho y sus componentes, potencia media,



y — según el dispositivo que haya realizado el sondeo — instrucciones por ciclo, fallos por cada mil instrucciones, tasa de fallos del último nivel de caché, fracción de ciclos de estancamiento, tasa de instrucciones y reloj observado, o bien potencia, utilización de cómputo y de memoria, reloj y temperatura del acelerador. El dispositivo que realizó el sondeo entra también como característica, en lugar de suponerse siempre el mismo.

Esta estrategia introduce además una noción que la anterior no necesita: el **estado de los recursos**. Una vez que un dispositivo fue inicializado, los candidatos que lo usan ya no pagan el costo de inicialización, mientras que los del otro dispositivo sí. El costo imputado a cada candidato depende por tanto de qué dispositivo realizó el sondeo, y esa dependencia se codifica explícitamente en la construcción del conjunto en lugar de promediarse.

Dos precisiones metodológicas sobre el sondeo. La primera: se toma la **primera** repetición y no el promedio de las tres disponibles en el experimento, porque en despliegue solo existiría una; promediar retrospectivamente introduciría en el modelo una precisión que la situación real no ofrece. La segunda: cuando la región de sondeo es más breve que el intervalo de muestreo, sus métricas puntuales quedan ausentes con indicador explícito, y el modelo las trata como tales mediante imputación con indicador de ausencia, en lugar de recibir un cero que significaría algo distinto.

### 2.4.7. Características de entrada y prevención de fuga

El criterio que delimita qué puede entrar al modelo es único: solo aquello que estaría disponible *antes* de ejecutar la configuración candidata. De ahí se sigue una lista explícita de columnas prohibidas — el tiempo, la energía, la potencia y el EDP de la configuración evaluada, así como la propia etiqueta, el margen, el número de repeticiones y los identificadores de configuración y de grupo de decisión.

Esa prohibición no se confía a la inspección visual del conjunto de datos. La lista de columnas prohibidas está declarada en el código, y la construcción del vector de entrada la verifica en cada ajuste, interrumpiendo la ejecución si alguna característica prohibida alcanza el modelo. La distinción entre las matrices de correlación exploratorias — que sí muestran los resultados medidos, porque su propósito es comprender el conjunto — y la lista blanca de características que alimentan el entrenamiento se mantiene explícita por la misma razón: son dos usos distintos del mismo conjunto y confundirlos es la vía habitual hacia una fuga inadvertida.

Conviene señalar qué queda deliberadamente fuera. El número de iteraciones no es una característica: es un factor de normalización experimental, propio del banco de pruebas y au-

sente en despliegue. Tampoco lo es la frecuencia absoluta en unidades de reloj del candidato; en su lugar entra la fracción del rango, que es comparable entre dos dispositivos cuyos rangos absolutos difieren en un orden de magnitud, mientras que el valor absoluto no lo sería sin normalizar.

## 2.4.8. Protocolo de validación

### Partición externa

Dado que el tamaño efectivo de la muestra es el número de configuraciones, y que las configuraciones de una misma operación a distintos tamaños comparten estructura algorítmica, una partición aleatoria a nivel de fila produciría una estimación optimista de la capacidad de generalización. La validación se realiza por tanto dejando fuera **una operación completa** en cada pliegue — seis pliegues, uno por operación —, de modo que el pliegue de prueba contenga únicamente configuraciones de una operación nunca vista en entrenamiento. Esta es la condición que el sistema enfrentaría en despliegue ante una aplicación cuyas fases no estuvieran representadas en el catálogo, y es deliberadamente más exigente que una partición por configuración, que dejaría en entrenamiento otros tamaños de la misma operación.

La ausencia de fuga entre particiones se verifica de forma automática: ninguna configuración, y por tanto ninguna de sus filas candidatas, puede aparecer simultáneamente en entrenamiento y en prueba.

### Búsqueda de hiperparámetros anidada

Dentro de cada pliegue externo, la búsqueda de hiperparámetros opera exclusivamente sobre las cinco operaciones de entrenamiento, y su propia validación vuelve a dejar fuera una operación de ese subconjunto. El pliegue externo no interviene en ninguna decisión de modelado, conforme a lo expuesto en la sección 1.1.10.

La búsqueda es multiobjetivo y minimiza simultáneamente dos cantidades: la pérdida de EDP media en la validación interna, y la latencia de inferencia en su percentil 99. El segundo objetivo no es un refinamiento opcional sino una exigencia directa de la pregunta de investigación, que condiciona la utilidad del sistema a que el costo de la inferencia no degrade el rendimiento. La latencia se mide sobre la puntuación del conjunto completo de candidatos de una decisión — que es la operación real del selector, no la predicción de una fila aislada — e incluye el preprocesamiento; se toma con 50 ejecuciones de calentamiento y 200 repeticiones medidas, y se restringe explícitamente a un solo hilo, de modo que familias con distinta capacidad de paralelización interna se comparen sobre la misma base.

De la aproximación al frente de Pareto resultante se elige un punto mediante una regla declarada: entre las configuraciones cuya pérdida de EDP no exceda en más de un 1 % a la mejor del frente, se toma la de menor latencia. Esta regla expresa la prioridad del problema — se prefiere una decisión marginalmente peor si es sustancialmente más barata de emitir — y se declara aquí para que la elección no quede como un criterio implícito de la implementación. La Tabla 2.11 resume los espacios explorados.

Tabla 2.11: Espacios de hiperparámetros explorados por familia.

| Familia             | Hiperparámetros y rangos                                                                                                                                                                                                                                                                                              |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Regresión logística | Inverso de la regularización $C \in [10^{-4}, 10^3]$ (escala logarítmica); penalización $\ell_1$ o $\ell_2$ ; tolerancia $\in [10^{-6}, 10^{-2}]$ .                                                                                                                                                                   |
| Árbol de decisión   | Profundidad máxima 1–12; criterio de impureza (Gini, entropía, pérdida logarítmica); mínimo de muestras para dividir 2–30; mínimo por hoja 1–20; fracción de características consideradas por división.                                                                                                               |
| Bosque aleatorio    | Número de árboles 50–500; profundidad máxima 2–20; mínimo para dividir 2–30; mínimo por hoja 1–20; fracción de características por división; uso o no de remuestreo.                                                                                                                                                  |
| XGBoost             | Número de árboles 50–600; profundidad máxima 2–10; tasa de aprendizaje $\in [0,01, 0,3]$ (logarítmica); peso mínimo por hijo $\in [1, 20]$ (logarítmica); submuestreo de filas y de columnas $\in [0,5, 1]$ ; complejidad mínima de división $\in [0, 5]$ ; regularización $\ell_1$ y $\ell_2$ en escala logarítmica. |

El desbalance de clases — una sola acción óptima entre 40 candidatos — se compensa dentro de cada pliegue mediante ponderación de clases, calculada exclusivamente sobre el subconjunto de entrenamiento. El preprocesamiento — imputación con indicador de ausencia, codificación de variables categóricas tolerante a categorías no vistas, y escalado solo para el modelo lineal — se ajusta también únicamente sobre entrenamiento.

## Líneas base

El aporte del modelo se mide contra un conjunto de políticas de referencia que no participan de la búsqueda de hiperparámetros, porque no tienen ninguno que ajustar: ejecutar siempre en CPU bajo el gobernador nativo; ejecutar siempre en CPU a la frecuencia máxima;

ejecutar siempre en el acelerador en su estado nativo; aplicar la mejor acción constante única; decidir al azar; y el oráculo con conocimiento posterior. La mejor acción constante se determina observando exclusivamente los pliegues de entrenamiento: elegirla mirando el pliegue de prueba le concedería información que en despliegue no tendría, e inflaría artificialmente el margen atribuido al modelo.

## Métricas reportadas

Se reportan, por familia y por pliegue, la pérdida de EDP respecto del óptimo alcanzable con conocimiento perfecto y el arrepentimiento porcentual asociado; la exactitud de la acción elegida y, por separado, la del dispositivo, dado que equivocar el dispositivo y equivocar el nivel de reloj tienen consecuencias distintas; la distancia en niveles entre el reloj elegido y el óptimo, que distingue un error contiguo de uno lejano; el valor F1 de la clase positiva y la precisión promedio como diagnósticos de desbalance; la latencia de inferencia en tres percentiles; el tamaño del modelo serializado; y el tiempo de ajuste. La métrica primaria es la pérdida de EDP: la exactitud de clasificación es diagnóstica, porque con una clase positiva entre cuarenta candidatos una exactitud alta puede obtenerse sin resolver el problema.

## Regla de selección de la familia final

La familia final se elige por menor pérdida de EDP externa media. Ahora bien, esa media se calcula sobre seis pliegues, y dos familias separadas por menos que la dispersión entre pliegues no son distinguibles con esa evidencia. Por eso la elegibilidad no se decide únicamente por una banda relativa fija: una familia es elegible si su pérdida cae dentro del 1 % de la mejor o dentro del error estándar combinado entre pliegues de ambas. Entre las elegibles se prefiere la de menor latencia en percentil 99 y, en caso de empate, el modelo serializado más pequeño. El conjunto de familias declaradas indistinguibles del ganador se reporta junto con el ganador, en lugar de presentar una única familia como vencedora inequívoca cuando la evidencia no lo sostiene.

### 2.4.9. Compuerta de validez sobre la distribución de la etiqueta

Un aparato de comparación como el descrito puede ejecutarse sobre un conjunto de datos en el que la etiqueta carezca de estructura aprendible, y producir de todos modos una tabla de resultados con apariencia de comparación legítima. Ese riesgo es real cuando una sola acción resulta óptima en casi todas las configuraciones: en tal caso la política constante correspondiente alcanza una pérdida de EDP próxima a cero por construcción, ninguna familia puede superarla de forma significativa, y las diferencias observadas entre familias reflejan

ruido de medición y no capacidad predictiva.

Para que esa situación no se lea como un resultado, el análisis incorpora una compuerta explícita, evaluada *antes* de interpretar cualquier comparación de modelos. Se calculan tres cantidades sobre la distribución de la etiqueta: la fracción de configuraciones en que gana la acción dominante, el número de acciones que ganan en una fracción no despreciable de los casos, y el margen relativo mediano entre la mejor y la segunda acción. Si la acción dominante supera el 90 % de los casos, o si menos de tres acciones alcanzan una masa del 5 %, o si el margen mediano cae por debajo del 2 % — piso de ruido establecido a partir de la dispersión observada entre repeticiones —, el resultado se declara **validación de tubería** y no comparación de modelos.

Esta distinción no es una salvaguarda hipotética: determina cómo debe leerse el resultado del eje de CPU, y se aplica en la sección 3.3.

#### 2.4.10. Alcance del eje de CPU en aislamiento

La campaña del eje de CPU concluyó antes que la del acelerador, y sobre ella puede ejecutarse el procedimiento completo. Conviene precisar qué establece y qué no ese ejercicio.

Lo que establece es que la tubería funciona de extremo a extremo: que la integración por regiones produce las cantidades esperadas, que la normalización por despacho opera correctamente, que las particiones no filtran información, que la búsqueda de hiperparámetros converge y es reanudable, que las cuatro familias se ajustan y serializan, y que las métricas y la latencia se calculan sobre la decisión completa. Establece también los órdenes de magnitud de la latencia de inferencia, que son propios del modelo y no del conjunto de datos.

Lo que no establece es cuál familia decide mejor el problema de esta investigación. El problema tiene dos ejes — dispositivo y frecuencia — y el eje de CPU en aislamiento contiene solo el segundo, precisamente aquel para el cual los resultados de la Fase 1 anticipan un margen estrecho en esta plataforma. Una comparación de familias sobre ese subconjunto respondería a una pregunta distinta de la formulada. Por esa razón, el resultado del eje de CPU se reporta como validación de la tubería, y la comparación de modelos que responde al cuarto objetivo específico se realiza sobre el conjunto completo, una vez incorporado el eje del acelerador.

Esta decisión es también la razón por la que el orden de ejecución del trabajo no coincide con el orden de exposición: la tubería se construyó y se validó primero, sobre los datos disponibles, en lugar de esperar a la campaña completa para escribir código no probado.

### 2.4.11. Reproducibilidad del procedimiento de modelado

Las garantías de reproducibilidad de la sección 2.3.9 cubren la adquisición; el procedimiento de modelado añade las suyas, porque introduce fuentes de variación que un manifiesto de campaña no contempla.

El procedimiento se ejecuta mediante etapas explícitas y encadenables — construcción del conjunto de nivel 2, análisis exploratorio, búsqueda de hiperparámetros y evaluación —, de modo que cada una pueda repetirse por separado sobre los artefactos de la anterior sin rehacer la cadena completa. Toda la aleatoriedad del procedimiento deriva de una semilla única declarada, que gobierna tanto el muestreo de configuraciones de hiperparámetros como los componentes estocásticos de los propios modelos.

Junto con el conjunto de datos se persiste un registro de procedencia que documenta las campañas de origen, la revisión exacta del código, los manifiestos empleados, la regla energética aplicada con el valor de su línea base y la corrida que la midió, las versiones de todas las bibliotecas intervinientes y el nodo en el que se ejecutó. Este último dato no es redundante: las bibliotecas de aprendizaje automático se resuelven contra paquetes del sistema que difieren entre nodos del mismo clúster, de modo que registrar únicamente el número de versión no basta para reproducir el entorno.

Los estudios de búsqueda de hiperparámetros se persisten en almacenamiento durable e identifican unívocamente la estrategia, la familia y el pliegue externo al que corresponden. Esa persistencia cumple dos funciones: permite reanudar una búsqueda interrumpida por el límite de tiempo del gestor de recursos desde los ensayos ya completados — sin repetirlos ni duplicarlos —, y deja auditable el recorrido completo de la búsqueda, no solo su resultado. Finalmente, el modelo seleccionado se serializa junto con su preprocesador, la lista de características empleadas y el contrato de acciones soportadas, de modo que la inferencia en despliegue no dependa de reconstruir esa configuración a partir del código.

## 2.5. Fase 3: Implementación del agente de selección

La tercera fase materializa el modelo en un agente de espacio de usuario que la aplicación consulta antes de despachar cada fase. Su ciclo de operación es, en consecuencia, distinto del de un demonio que muestrea periódicamente: no observa de forma continua para reaccionar, sino que responde a una consulta puntual con una decisión.

Ante cada operación por despachar, el agente construye el vector de características descrito en la sección 2.4.6, puntúa con él las 40 acciones candidatas, y aplica la de mayor

probabilidad estimada — selección de la rama de ejecución correspondiente al dispositivo elegido y fijación del estado de frecuencia en ese dispositivo — antes de ceder el control a la operación. La política es discreta: selecciona un elemento de un conjunto finito de configuraciones soportadas por el hardware, sin resolver un problema continuo de optimización.

Conviene precisar en qué consiste materialmente la “selección de la rama de ejecución”, porque es la forma en que la decisión del modelo se traduce en código ejecutado. La aplicación contiene, para cada fase, ambas realizaciones de la operación — la que despacha al acelerador y la que permanece en el anfitrión —, y la decisión del selector determina cuál de las dos se invoca mediante un condicional situado antes del despacho. No se trata, por tanto, de un tercer grado de libertad independiente de los dos anteriores: elegido el dispositivo, la rama queda determinada de forma unívoca, dado que cada operación dispone de exactamente una realización por dispositivo. Esta correspondencia uno a uno es la que permite que el espacio de acciones se describa íntegramente mediante el par dispositivo–frecuencia.

El diseño incorpora tres consideraciones derivadas del marco conceptual y de los resultados de la Fase 1. La primera es que el mecanismo de actuación debe adaptarse al controlador de frecuencia activo en cada dispositivo, dado que la vía para fijar un punto operativo difiere entre un controlador que expone un gobernador de espacio de usuario, uno que solo admite restringir el rango permitido, y la interfaz de bloqueo de reloj del acelerador. La segunda es que el costo de la propia decisión — inferencia más actuación, incluida la latencia de conmutación de frecuencia — debe amortizarse contra la duración de la operación que se va a despachar; una operación suficientemente corta no justifica el costo de decidir sobre ella, y ese umbral se determina empíricamente en la Fase 4 en lugar de fijarse a priori [13, 37]. La tercera es que el costo de inicialización del acelerador se paga una sola vez por proceso y no en cada despacho, por lo que el agente debe distinguir entre la primera operación que se envía al dispositivo y las siguientes; esa distinción es exactamente la que el contrato temporal de la sección 2.2.5 hace medible.

El agente alterna entre las dos estrategias de la sección 2.4.6 según la información de que disponga. Ante la primera aparición de una operación no existe telemetría sobre ella, y la decisión se emite con el vector puramente analítico; la ejecución resultante — que es trabajo útil, no una medición desechable — produce la telemetría que alimenta las decisiones siguientes sobre esa misma operación, que pasan a emitirse con el vector ampliado. El agente mantiene por tanto un registro por operación de si ya dispone de sondeo y de qué dispositivo lo produjo, dado que ese dato determina qué costo corresponde imputar a cada candidato. Ambas estrategias se evalúan por separado en la Fase 4, porque corresponden a situaciones de despliegue distintas y no a dos implementaciones alternativas de la misma.

## 2.6. Fase 4: Validación experimental y análisis de resultados

La fase final evalúa empíricamente el impacto del sistema sobre el consumo energético y el rendimiento, en concordancia con el cuarto objetivo específico. La evaluación se realiza sobre una aplicación sintética compuesta por varias fases de tipos distintos, construida de modo que el óptimo no sea constante entre ellas; una aplicación cuyas fases compartieran todas la misma configuración óptima no permitiría distinguir un selector de una configuración fija bien elegida.

Se comparan cuatro escenarios sobre esa misma aplicación, midiendo en todos ellos tiempo total, energía consumida, potencia media y EDP:

1. **Todo en CPU**, bajo el gobernador nativo del sistema operativo, que constituye la referencia exigida por la pregunta de investigación.
2. **Todo en GPU**, con el comportamiento nativo del acelerador.
3. **Selector propuesto**, decidiendo dispositivo y frecuencia fase por fase.
4. **Oráculo**, que aplica a cada fase la configuración de mínimo EDP conocida a posteriori.

El cuarto escenario no es un competidor sino una cota: acota el margen máximo alcanzable con esta formulación y permite separar dos causas distintas de un resultado insuficiente — que el margen no exista en la plataforma, o que el modelo no logre capturarlo. Sin esa separación, un resultado negativo sería ambiguo.

Se mide además, de forma explícita, el *overhead* atribuible al agente, descompuesto en latencia de inferencia y costo de actuación, y se determina el umbral de duración de operación a partir del cual ese costo queda amortizado. Esta magnitud responde directamente a la cláusula de la pregunta de investigación relativa a que el costo de la inferencia no degrade el rendimiento global.

Dado que el objetivo no es únicamente reportar métricas sino establecer si las diferencias observadas son estadísticamente defendibles, los resultados se analizan mediante técnicas seleccionadas según la distribución y la homogeneidad de varianzas de los datos, reportando medidas de dispersión y tamaños de efecto junto con las pruebas de significancia.



## 2.7. Aspectos éticos

La presente investigación no involucra participantes humanos, datos personales ni información sensible, por lo que no requiere aprobación de un comité de ética en investigación con seres humanos. No obstante, se adoptan las siguientes consideraciones éticas, pertinentes a la naturaleza del trabajo.

En cuanto al uso responsable de infraestructura compartida, los experimentos se ejecutan sobre recursos de cómputo institucionales de acceso compartido. En consecuencia, se adopta como principio que ninguna acción del experimento debe degradar las condiciones de ejecución de terceros: las mediciones se realizan mediante asignación exclusiva otorgada por el gestor de recursos, y toda actuación sobre la frecuencia se restringe a los núcleos efectivamente asignados, verificando previamente que el dominio de frecuencia afectado no exceda dicha asignación, de acuerdo con lo expuesto en la sección 1.1.6. Del mismo modo, el estado de frecuencia del sistema se restaura al concluir la campaña, incluso ante terminaciones anómalas.

En cuanto a la integridad de los datos y de los resultados, se adopta como criterio que ninguna observación se descarte de forma silenciosa: las ventanas que no satisfacen los criterios de calidad se conservan con su anotación correspondiente, y las corridas rechazadas preservan sus artefactos. Las limitaciones conocidas del instrumento — en particular, el carácter aproximado de la estimación de bytes movidos discutido en la sección 1.1.2 — se declaran explícitamente junto con los resultados que dependen de ellas, en lugar de omitirse.

En cuanto al uso de software de terceros, las suites de benchmarks empleadas se utilizan sin modificación de su código fuente y respetando sus condiciones de distribución, y su procedencia se registra mediante la suma de verificación de los archivos originales obtenidos. Finalmente, en cuanto a transparencia y reproducibilidad, el instrumento de medición, el orquestador y las definiciones de campaña se publican en un repositorio de acceso público, junto con la documentación de las decisiones metodológicas adoptadas durante el desarrollo, de modo que los resultados puedan ser verificados o refutados por terceros.

# Capítulo 3

## Resultados

Este capítulo reporta los resultados obtenidos hasta el cierre de este documento. Cubre la campaña de adquisición del eje de CPU del catálogo dual, la corrección del efecto del observador sobre la medición energética del acelerador ocioso, la distribución de la configuración óptima resultante, el catálogo dual completo con el eje del acelerador y el costo de arranque de CUDA, la sensibilidad de esa distribución a la resolución temporal de la región fría, la verificación de extremo a extremo del procedimiento de modelado, y el reanálisis exploratorio de la Fase R1 — el mapa de amortización  $K_{\text{break\_even}}$ , la separación entre dispositivo y frecuencia, el *headroom* de DVFS, el piso de ruido, el resultado de baselines y el comportamiento de movimiento de datos — que reformula el selector como una decisión de dispositivo por tamaño y horizonte de reutilización. La implementación del agente y la evaluación de EDP frente a los gobernadores nativos corresponden a las Fases 3 y 4.

### 3.1. Campaña del catálogo dual sobre el eje de CPU

La campaña del eje de CPU descrita en la sección 2.4.3 se ejecutó completa: 1 632 de 1 632 combinaciones aceptadas, sin rechazos ni reutilización de intentos anteriores. Las 68 configuraciones del catálogo dual quedaron cubiertas en los ocho niveles de la rejilla fina con sus tres repeticiones.

La aceptación íntegra no es en sí misma la verificación relevante — una campaña puede aceptar todas sus corridas y haber medido algo distinto de lo que declara —, de modo que se comprobaron por separado las tres condiciones de las que depende la validez del conjunto. Primera, la frecuencia efectivamente aplicada: los ocho niveles solicitados correspondieron a ocho frecuencias físicas distintas, verificadas por relectura sobre cada núcleo delegado, con

los valores de la Tabla 2.5. Segunda, el contrato temporal: las cinco marcas de la sección 2.2.5 están presentes, son monótonas y no duplicadas en la totalidad de las corridas aceptadas, condición que el instrumento verifica de forma cerrada. Tercera, la disponibilidad de las tres fuentes energéticas — dominios de paquete y de memoria del procesador, y serie del acelerador — en todas ellas.

Sobre esa base, el constructor de nivel 2 produjo las 68 decisiones esperadas con sus ocho acciones candidatas por decisión, sin exclusiones. El informe de completitud no registró ninguna configuración incompleta, de modo que ninguna etiqueta se construyó sobre un mínimo parcial.

## 3.2. Efecto del observador en la medición energética del acelerador ocioso

La campaña anterior reveló un artefacto del instrumento cuya corrección se describió en la sección 2.4.4, y cuya magnitud conviene reportar aquí porque condiciona el objetivo del eje de CPU.

Ninguna carga de esa campaña ejecuta código en el acelerador, y se verificó la ausencia de procesos ajenos sobre él. Aun así, la interfaz de gestión del dispositivo, consultada cada 5 ms, reportó una potencia media por corrida de entre 49.93 y 67.60 W, con mediana de 59.53 W, mientras declaraba cero por ciento de utilización. Una sonda independiente del mismo dispositivo en reposo, con consultas espaciadas, midió 34.84 W de media, 35.05 W en el percentil 95 y 35.20 W como máximo; relecturas posteriores confirmaron el dispositivo ocioso en torno a 36 W. La diferencia observada en el reloj es coherente con esa lectura: 1410 MHz durante el muestreo a alta cadencia, frente a 210 MHz en reposo genuino.

La magnitud del artefacto es por tanto de aproximadamente un 68 % sobre la línea base de reposo. Atribuirlo a la carga habría inflado el término de acelerador del objetivo de CPU en esa proporción, en una comparación cuyo propósito es precisamente contrastar dos dispositivos. Se subraya que la lectura instrumental no es incorrecta — el dispositivo consume realmente esa potencia en el estado que el propio sondeo provoca — y que este comportamiento no se extrapola a otros modelos de acelerador: es una propiedad observada en esta plataforma, no una afirmación general sobre la interfaz de gestión.

### 3.3. Distribución de la configuración óptima en el eje de CPU

Con el conjunto de nivel 2 construido, la compuerta de la sección 2.4.9 se aplicó sobre la distribución de la etiqueta antes de interpretar comparación alguna entre familias de modelos. Los dos vectores de entrada de la sección 2.4.6 arrojan veredictos distintos, y la diferencia es informativa por sí misma.

Para la estrategia sin ejecución previa, la acción dominante — el gobernador nativo — resulta óptima en el 36.8 % de las 68 configuraciones, cinco de las ocho acciones alcanzan una masa apreciable, y el margen relativo mediano entre la mejor y la segunda acción es del 11.3 %. Los tres criterios de la compuerta se satisfacen: la etiqueta posee variedad y sus márgenes superan holgadamente el piso de ruido de medición.

Para la estrategia con sondeo, en cambio, la acción dominante alcanza el 54.4 % de los casos, solo dos acciones superan la masa mínima, y el margen mediano cae al 0.73 %, por debajo del piso del 2 %. La compuerta declara ese resultado como validación de tubería. La razón de la divergencia es atribuible a la construcción del vector: la estrategia con sondeo imputa a los candidatos que comparten dispositivo con la sonda el costo de la región caliente, en la que el efecto de la frecuencia sobre un despacho aislado queda diluido por el costo fijo ya amortizado, mientras que la estrategia sin ejecución previa evalúa todos los candidatos sobre la región fría, donde ese efecto permanece visible.

Este resultado tiene una consecuencia práctica directa que conviene declarar sin ambigüedad, y es la que sostiene la sección 2.4.10: aun cuando la etiqueta de la primera estrategia sí posee estructura, el eje de CPU contiene únicamente uno de los dos grados de libertad del problema. La comparación de familias que responde al cuarto objetivo específico requiere el eje del acelerador, y hasta disponer de él el resultado del eje de CPU se reporta como validación del procedimiento.

### 3.4. El catálogo dual completo y el costo de arranque de CUDA

La campaña del eje del acelerador (sección 3.6) se cerró íntegramente el mismo día en que se redacta esta sección: 6528 de 6528 combinaciones aceptadas, sin rechazos sin resolver. Reconstruir el conjunto de nivel 2 en modo final — las 68 configuraciones con sus 40 acciones candidatas completas, CPU y GPU juntas — confirmó la matriz sin excepciones,

y la compuerta de la sección 2.4.9 se recalculó por primera vez sobre el catálogo íntegro, no sobre el eje de CPU en aislamiento.

El resultado para la estrategia sin ejecución previa es idéntico, cifra por cifra, al reportado en la sección 3.3: acción dominante en el 36.8 % de las decisiones, cinco acciones con masa apreciable, margen mediano del 11.3 %. La razón de esa identidad es en sí misma el resultado: **ninguna de las 68 decisiones cambia de ganador al incorporar las 32 acciones candidatas de GPU** — la acción óptima de esta estrategia es siempre una frecuencia de CPU, nunca una configuración del acelerador. Añadir el segundo grado de libertad no modificó una sola etiqueta.

Esto no es un error de medición ni una propiedad general del acelerador: es consecuencia directa de cómo esta estrategia define su candidato. La región fría de GPU incluye la creación del contexto CUDA y de los *handles* de biblioteca (cuBLAS/cuSOLVER/cuFFT), un costo de arranque que la literatura reporta habitualmente entre 200 ms y 1 s, independiente del tamaño del problema. Sobre las configuraciones más grandes del catálogo — multiplicación de matrices densas de  $4096 \times 4096$ , la mayor cantidad de trabajo por despacho disponible —, el mejor tiempo de GPU en la región fría fue 0.578 s frente a 0.274 s de CPU; para la misma operación, el cómputo puro a la tasa pico medida de la A100 (sección 2.3.8, análogo de GPU) representa apenas unos 14 ms de esos 0.578 s. El resto es arranque de biblioteca. El patrón se repite, casi sin variación, en las demás operaciones del catálogo: el mejor tiempo de GPU en frío se mantiene entre 0.49 y 0.81 s sin importar si la operación es una multiplicación de matrices densa o una combinación lineal de vectores trivial — la firma característica de un costo fijo dominante, no de trabajo que escale con el problema.

La consecuencia es que **ningún tamaño del catálogo actual (hasta  $N = 4096$  en álgebra densa, hasta  $N \approx 3,16 \times 10^7$  en operaciones vectoriales) es suficientemente grande para que el cómputo puro supere el costo de arranque en una comparación de despacho único**. La estrategia sin ejecución previa evalúa cada decisión como si fuera la primera llamada de un proceso nuevo, sin amortizar ese costo contra llamadas posteriores; bajo esa definición, estructuralmente no puede recomendar el acelerador con estos tamaños, con independencia de cuán favorable sea su tasa de cómputo pico. No es una limitación del hardware ni del instrumento: es una limitación del contrato de comparación de esta estrategia en particular, y queda documentada como tal en vez de leerse como evidencia de que la GPU es inconveniente en general.

Para la estrategia con sondeo, el veredicto tampoco cambia: sigue siendo `pipeline_smoke_only`, con margen mediano de apenas 1.03 % — por debajo del piso de ruido de medición — pese

a que en esta estrategia la GPU sí gana en una fracción sustancial de sus grupos de decisión (43 de 136). Por operación, la GPU es óptima en el 50 % de los grupos de *Cholesky* y de la transformada rápida de Fourier, el 46 % de *stencil*, el 42 % de multiplicación de matrices, el 31 % de multiplicación matriz–vector dispersa y solo el 13 % de combinación lineal de vectores. Esto resuelve la pregunta que la decimotercera limitación dejaba abierta: el veredicto de la estrategia con sondeo **no mejora** al incorporar el eje del acelerador completo. La GPU compite de verdad en esta estrategia, pero los márgenes entre sus mejores candidatos son demasiado estrechos para servir de etiqueta de entrenamiento confiable.

### 3.4.1. La región caliente: dónde vive la ventaja real de la GPU

El resultado anterior deja una pregunta abierta que conviene responder de forma explícita: si el costo de arranque de CUDA domina la región fría, ¿desaparece la ventaja de la GPU, o simplemente queda oculta detrás de ese costo fijo? La región caliente — llamadas posteriores a la primera, que ya reutilizan contexto y *handles* pero vuelven a transferir operandos y resultados (sección 2.2.5) — permite responder, porque en ella el costo de arranque ya está amortizado y el EDP por despacho refleja únicamente el costo recurrente de la operación.

La respuesta es que la ventaja de la GPU es real, grande, y sistemáticamente restringida al mismo régimen que el modelo Roofline ya predice como favorable al cómputo denso. Contrastando el mejor candidato de cada dispositivo en la región caliente, para el tamaño mayor de cada operación:

Tabla 3.1: Mejor EDP por despacho en la región caliente, CPU frente a GPU, para el tamaño mayor de cada operación del catálogo.

| Operación ( <i>N</i> mayor)                           | EDP CPU (Js) | EDP GPU (Js) | Razón          |
|-------------------------------------------------------|--------------|--------------|----------------|
| Cholesky (4096)                                       | 14.297       | 0.151        | GPU 94× mejor  |
| Multiplicación de matrices (4096)                     | 14.516       | 0.452        | GPU 32× mejor  |
| Transformada de Fourier (4096)                        | 6.477        | 0.467        | GPU 14× mejor  |
| <i>Stencil</i> (4096)                                 | 0.041        | 0.111        | CPU 2.7× mejor |
| Combinación lineal de vectores ( $3,16 \times 10^7$ ) | 0.141        | 0.839        | CPU 6× mejor   |
| Matriz–vector dispersa ( $3,16 \times 10^7$ )         | 0.509        | 14.868       | CPU 29× mejor  |

El patrón no es aleatorio: coincide exactamente con la clasificación *compute-bound/memory-bound* del marco conceptual (sección 1.1.2). *Cholesky*, la multiplicación de matrices y la transformada de Fourier son las tres operaciones de mayor intensidad operacional del catálogo —

reutilizan intensamente los datos que ya residen cerca del procesador —, y son precisamente las tres donde la GPU gana por un margen de un orden de magnitud o más una vez amortizado el arranque. *Stencil*, la combinación lineal de vectores y la multiplicación matriz–vector dispersa son de baja intensidad operacional — un paso sobre el dato con reutilización mínima o nula —, y son las tres donde la CPU conserva la ventaja incluso en caliente: el ancho de banda de transferencia *host–device* que cada despacho de la región caliente vuelve a pagar pesa más que cualquier ganancia de cómputo que la GPU pudiera ofrecer.

Contando no solo el tamaño mayor sino las 68 configuraciones completas, la GPU es la ganadora en caliente en 8 de 13 tamaños de *Cholesky*, 8 de 13 de la transformada de Fourier y 6 de 13 de multiplicación de matrices — mayoritaria en las tres operaciones *compute-bound*, pero no en todas sus configuraciones, porque a tamaños pequeños ni siquiera el trabajo recurrente basta para compensar la transferencia por despacho. En las tres operaciones *memory-bound* la CPU gana en caliente en la totalidad de sus configuraciones, sin una sola excepción.

La lectura conjunta de las dos regiones es entonces la siguiente: la oportunidad de la GPU en este catálogo es real y sustancial, pero vive enteramente en la región caliente y está condicionada al régimen *compute-bound*. Ninguna de las dos estrategias de entrada de la sección 2.4.6 la explota bien — la primera nunca la ve, por construcción; la segunda la ve, pero con demasiado ruido para entrenar sobre ella — lo que deja como trabajo futuro explícito una estrategia que decida a partir del régimen de la operación y del número de despachos esperados, en lugar de un único despacho aislado.

### 3.5. Sensibilidad de la etiqueta a la resolución temporal de la región fría

La estrategia sin ejecución previa construye tanto sus candidatos como su etiqueta exclusivamente sobre la región fría, sin telemetría de la región caliente que la respalde. Dado que esa región es, en las configuraciones más pequeñas, más breve que el intervalo de muestreo, procede cuantificar en qué medida la etiqueta depende de mediciones de baja resolución en lugar de asumir que el efecto es despreciable.

De las 544 acciones candidatas del conjunto, 32 — un 5.9 % — corresponden a corridas cuya región fría no alcanza un intervalo de muestreo completo. La dependencia de la etiqueta respecto de ellas se distribuye como sigue: en 5 de las 68 decisiones la acción ganadora es una de baja resolución; en 3 decisiones ninguna acción alcanza resolución nominal, de modo

que no existe alternativa contra la cual contrastar; y al recalcular el óptimo excluyendo las acciones de baja resolución, el ganador cambia en 2 decisiones de 68. Ambos casos pertenecen a la operación de combinación lineal de vectores, que es la de menor trabajo por despacho del catálogo y por tanto aquella en la que la región fría es más breve.

Estas filas no se excluyen del conjunto. Excluir las sesgaría la muestra en contra de las configuraciones pequeñas, que son precisamente aquellas en las que la frontera de conveniencia entre dispositivos resulta más interesante. Se conservan con su anotación de baja resolución, y esta auditoría se reporta junto con los resultados del modelo para que la fracción de la etiqueta que descansa sobre mediciones de resolución limitada sea explícita y no haya que inferirla.

### 3.6. Verificación de extremo a extremo del procedimiento de modelado

Sobre el conjunto del eje de CPU se ejecutó el procedimiento completo de la sección 2.4.8: seis pliegues externos por operación, búsqueda multiobjetivo de hiperparámetros anidada dentro de cada pliegue para las cuatro familias, medición de latencia a un hilo, y serialización del modelo seleccionado.

La verificación confirmó los aspectos del procedimiento que no dependen de que la etiqueta tenga estructura: que ninguna configuración aparece simultáneamente en entrenamiento y prueba; que la búsqueda de hiperparámetros crea estudios reanudables, de modo que una interrupción por límite de tiempo del gestor de recursos continúa desde los ensayos ya completados en lugar de repetirlos; que las cuatro familias se ajustan, puntúan y serializan a través de la misma interfaz; y que las métricas de decisión y de latencia se calculan sobre el conjunto completo de candidatos de cada decisión.

Conforme a lo establecido en las secciones 2.4.9 y 2.4.10, las magnitudes comparativas entre familias obtenidas en esta ejecución no se reportan como resultado de la investigación: el eje de CPU en aislamiento no contiene el grado de libertad de dispositivo, y la comparación que responde al cuarto objetivo específico se realizará sobre el conjunto completo. Lo que esta ejecución sí establece es que el procedimiento está operativo y auditado, de modo que la incorporación del eje del acelerador no exige construir ni depurar la tubería sobre datos ya adquiridos.



### 3.7. Reanálisis por tamaño y amortización (Fase R1)

Las secciones siguientes reportan la Fase R1 de la reformulación del selector: un reanálisis exploratorio de las 8 160 corridas ya aceptadas del catálogo dual, ejecutado antes de solicitar cualquier hora de cómputo adicional. La unidad independiente sigue siendo `config_id` (operación  $\times$  tamaño): 68 configuraciones. El conjunto compacto que este reanálisis produce contiene 204 filas porque cada configuración se observa bajo tres estados de recurso — `none_ready`, `cpu_ready` y `gpu_ready` —, y esas 204 filas son tres vistas correlacionadas de las mismas 68 configuraciones físicas, no 204 experimentos independientes.

Este reanálisis es exploratorio y no sustituye la evaluación confirmatoria: una campaña separada reserva 9 configuraciones nuevas de *GEMM*, *Cholesky* y transformada de Fourier en  $N \in \{8\,192, 12\,288, 16\,384\}$ , bajo un protocolo congelado el mismo día antes de observar ninguna de sus mediciones. Esas 9 configuraciones **aún no se han observado** al momento de redactar este capítulo; los resultados que siguen se calculan exclusivamente sobre las 68 configuraciones exploratorias, y donde corresponda se señala qué pregunta queda pendiente de la evaluación confirmatoria.

#### 3.7.1. El mapa de amortización $K_{\text{break\_even}}$

La estrategia sin ejecución previa (sección 3.4) compara dispositivos sobre un único despacho frío, lo que estructuralmente impide que la GPU compita mientras el costo de arranque de CUDA domine la comparación. Esa comparación es incompleta si la operación se repite: el costo de arranque se paga una sola vez y se amortiza contra las repeticiones siguientes. Para cuantificar esa amortización se define, por dispositivo  $d$  y horizonte de despachos  $K$ ,

$$E_{\text{total}}(d, K) = E_{\text{cold}}(d) + (K - 1) \cdot E_{\text{warm}}(d)$$

$$T_{\text{total}}(d, K) = T_{\text{cold}}(d) + (K - 1) \cdot T_{\text{warm}}(d)$$

$$EDP_{\text{total}}(d, K) = E_{\text{total}}(d, K) \cdot T_{\text{total}}(d, K)$$

y se llama  $K_{\text{break\_even}}$  al menor  $K$  para el cual  $EDP_{\text{total}}(\text{GPU}, K) < EDP_{\text{total}}(\text{CPU}, K)$ . Por construcción, ese cruce existe si y solo si la GPU gana en la región caliente para esa configuración: si CPU domina también en caliente, ningún número de repeticiones hace que la curva de GPU cruce por debajo, y  $K_{\text{break\_even}}$  es infinito.

Tabla 3.2: Cruces finitos de  $K_{\text{break\_even}}$  por operación, sobre las medias de las tres repeticiones.

| Operación      | Configs. | $K$ finito | $K$ mínimo | $K$ mediano finito | $K$ máximo |
|----------------|----------|------------|------------|--------------------|------------|
| AXPY           | 8        | 0          | —          | —                  | —          |
| Cholesky       | 13       | 8          | 2          | 49                 | 2 497      |
| FFT            | 13       | 8          | 4          | 81,5               | 12 294     |
| GEMM           | 13       | 6          | 3          | 37,5               | 1 075      |
| SpMV           | 8        | 0          | —          | —                  | —          |
| <i>Stencil</i> | 13       | 0          | —          | —                  | —          |

En total, 22 de las 68 configuraciones presentan un cruce finito, y las 46 restantes no amortizan GPU bajo el contrato de transferencias medido dentro del horizonte analizado. *Combinación lineal de vectores*, matriz–vector dispersa y *stencil* no tienen ningún cruce finito: son las tres operaciones de baja intensidad operacional identificadas en la sección 3.4.1, en las que la CPU ya gana en la región caliente, de modo que ninguna cantidad de repeticiones favorece a la GPU. No es correcto, por tanto, afirmar que una operación completa amortiza GPU a todos sus tamaños solo porque sus tamaños grandes lo hagan: *Cholesky*, la transformada de Fourier y la multiplicación de matrices densa tienen cruce en menos de la mitad de sus 13 tamaños cada una (8, 8 y 6 respectivamente).

Los primeros tamaños con cruce observado son *GEMM* en  $N = 768$  con  $K_{\text{break\_even}} = 1\,075$ , la transformada de Fourier en  $N = 192$  con  $K_{\text{break\_even}} = 12\,294$ , y *Cholesky* en  $N = 384$  con  $K_{\text{break\_even}} = 2\,497$ . La presencia del cruce no es monótona en el tamaño: la transformada de Fourier presenta un cruce aislado en  $N = 192$ , vuelve a favorecer CPU en  $N = 256$  y  $N = 384$ , y solo recupera cruces de forma sostenida desde  $N = 512$ . Este comportamiento es en sí mismo una razón para tratar el tamaño como eje experimental medido en lugar de imponerle una frontera teórica suavizada.

En el extremo superior del catálogo actual, el horizonte necesario para amortizar GPU se reduce drásticamente: *GEMM* en  $N = 4096$  tiene  $K_{\text{break\_even}} = 3$ , la transformada de Fourier en  $N = 4096$  tiene  $K_{\text{break\_even}} = 4$ , y *Cholesky* en  $N = 4096$  tiene  $K_{\text{break\_even}} = 2$ . Con apenas dos a cuatro repeticiones, la GPU ya amortiza su arranque en las tres operaciones *compute-bound* del catálogo a su mayor tamaño disponible.

$K_{\text{break\_even}}$  no se presenta aquí como un entero exacto libre de incertidumbre. Los costos *cold* y *warm* se estiman a partir de tres repeticiones experimentales, y la región fría hereda el piso de ruido más alto de la sección 3.7.4. Una envolvente conservadora — que combina los extremos marginales observados de energía y tiempo en los sentidos favorable y desfavorable

a GPU, sin suponer que el mínimo de energía y el de tiempo pertenecen a la misma repetición — produce una banda de sensibilidad con mediana del 41 % y máximo del 78 % sobre el valor central de  $K_{\text{break\_even}}$ . Esa banda es una cota conservadora, no un intervalo probabilístico ni el resultado de un remuestreo pareado, y debe leerse junto con la cifra central en todo momento.

### 3.7.2. La decisión de dispositivo y su separación de la frecuencia

La reformulación de la sección 3.7 separa la decisión de dispositivo de la decisión de frecuencia, en lugar de clasificar directamente entre las 40 acciones `dispositivo`  $\times$  `frecuencia` como hacía la formulación anterior. Esa separación solo se justifica si la decisión de dispositivo obtenida a la frecuencia de referencia coincide con la que obtendría un oráculo que explora las 40 acciones.

Tabla 3.3: Dispositivo óptimo a frecuencia REF por estado de recurso, sobre las 204 filas del conjunto compacto (68 configuraciones  $\times$  3 estados).

| Estado                  | CPU óptima | GPU óptima | Separadas | Inciertas |
|-------------------------|------------|------------|-----------|-----------|
| <code>none_ready</code> | 68         | 0          | 68        | 0         |
| <code>cpu_ready</code>  | 68         | 0          | 68        | 0         |
| <code>gpu_ready</code>  | 12         | 56         | 68        | 0         |

La decisión de dispositivo obtenida comparando únicamente CPU y GPU a frecuencia REF coincide con la del oráculo que explora las 40 acciones en los 204 contextos, sin una sola discrepancia. El producto cartesiano completo de frecuencias no cambió ninguna etiqueta de dispositivo en el conjunto actual. Este resultado respalda la arquitectura de dos capas de la sección 3.7: la decisión de dispositivo puede resolverse a REF, dejando la exploración de frecuencia como una capa posterior sobre el dispositivo ya elegido.

### 3.7.3. El *headroom* de DVFS por estado

Una vez fijado el dispositivo, resta cuantificar cuánto puede ganar todavía la actuación de frecuencia frente a REF, antes de descontar el costo de la propia actuación. Se compara, por estado de recurso, la mejor frecuencia del dispositivo ganador contra REF:

Tabla 3.4: Mejora potencial de frecuencia sobre REF, por estado de recurso, antes de descontar el costo de actuación.

| Estado                  | Casos sobre 3,11 % | Mediana | p95     | Máxima  |
|-------------------------|--------------------|---------|---------|---------|
| <code>none_ready</code> | 38/68              | 4,98 %  | 50,70 % | 57,96 % |
| <code>cpu_ready</code>  | 3/68               | 0,25 %  | 2,77 %  | 8,54 %  |
| <code>gpu_ready</code>  | 56/68              | 9,45 %  | 45,25 % | 68,47 % |

Sobre este resultado conviene distinguir dos afirmaciones que no son equivalentes y que deben mantenerse separadas. La primera es que el margen relativo mediano entre la mejor frecuencia y la segunda mejor, dentro de la estrategia con sondeo, es del 1,03 %: ese número indica que el **argmin** fino — identificar exactamente cuál de las ocho frecuencias es la óptima puntual — es inestable frente al ruido de medición. La segunda es que **gpu\_ready** conserva, en 56 de sus 68 configuraciones, una mejora potencial sobre REF por encima del piso de ruido del 3,11 % (sección 3.7.4), con mediana del 9,45 %. Que el óptimo puntual sea inestable no implica que REF sea equivalente al conjunto de mejores frecuencias: hay un margen mediano recuperable de un orden de magnitud mayor que el piso de ruido en el estado que domina el catálogo tras la selección de dispositivo. La tarea pendiente para la capa DVFS no es, entonces, localizar el óptimo exacto entre ocho niveles, sino identificar un conjunto de frecuencias estadísticamente equivalentes y robustas frente al ruido, y descontar después el costo real de la actuación.

### 3.7.4. El piso de ruido y su descomposición regional

El piso de ruido de medición usado como referencia en las secciones anteriores se calibra como el coeficiente de variación mediano del EDP entre las tres repeticiones de una misma acción, sobre el conjunto exploratorio completo:

Tabla 3.5: Coeficiente de variación mediano del EDP entre repeticiones, global, por dispositivo y por región temporal.

| Desglose                 | CV mediano |
|--------------------------|------------|
| Global                   | 3,11 %     |
| CPU                      | 2,57 %     |
| GPU                      | 3,16 %     |
| Región <code>cold</code> | 5,76 %     |
| Región <code>warm</code> | 1,80 %     |

La descomposición por región es la más relevante para interpretar los resultados anteriores: la región fría es aproximadamente tres veces más ruidosa que la región caliente. Esto no es un detalle secundario, porque el estado `none_ready` — y con él, la totalidad de los términos  $E_{\text{cold}}$  y  $T_{\text{cold}}$  que alimentan el mapa de amortización de la sección 3.7.1 — descansa enteramente sobre mediciones de la región más ruidosa del experimento. La banda de incertidumbre del 41 % mediano reportada para  $K_{\text{break\_even}}$  es, en buena medida, una consecuencia directa de heredar ese piso más alto.

### 3.7.5. El resultado de baselines y su alcance

En `none_ready` y `cpu_ready`, la política `always_cpu_ref` coincide con el oráculo de dispositivo en la totalidad de los pliegues evaluados, de modo que la única rama con una comparación no trivial es `gpu_ready`. Sobre ella se evaluaron las ocho baselines obligatorias del protocolo (§9 del plan de reformulación) en los dos regímenes de partición por tamaño:

Tabla 3.6: Mejor baseline frente al oráculo de dispositivo en `gpu_ready`, para los regímenes de interpolación y extrapolación.

| Régimen       | Mejor baseline               | Balanced accuracy | Razón EDP/oráculo | Brecha recuperada |
|---------------|------------------------------|-------------------|-------------------|-------------------|
| Interpolación | umbral de tamaño             | 0,778             | 1,0029            | 0,285             |
| Extrapolación | tabla de cruce por operación | 0,867             | 1,0130            | 1,283             |

Ambas brechas recuperables — 0,285 % en interpolación y 1,283 % en extrapolación — quedan por debajo del piso de ruido global del 3,11 %. Bajo la regla bloqueante del protocolo congelado (superar la mejor baseline pertinente por encima del piso de ruido, o conservar la baseline), este resultado ya es suficiente para anticipar un veredicto negativo para el aprendizaje automático de dispositivo en estos pliegues: si ni siquiera el oráculo logra separarse de la mejor baseline por más del piso de ruido, ningún modelo entrenado sobre los mismos datos puede hacerlo mejor.

Dos salvedades deben mantenerse explícitas al leer esta conclusión, y no deben omitirse. Primera, es una **conclusión contractual** derivada de una regla bloqueante predeclarada, no una prueba estadística de ausencia de señal: el 3,11 % es el coeficiente de variación mediano de acciones *individuales* entre repeticiones, no una estimación de la incertidumbre de la diferencia *agregada* entre dos políticas evaluadas sobre el conjunto de prueba completo; ambas cantidades responden preguntas distintas y no deben confundirse. Segunda, este resultado

está acotado a  $K = 1$  — un único despacho restante —, porque ninguna de las ocho baselines de esta comparación plantea la pregunta de horizonte que introduce la sección 3.7.6. La Fase R2 del plan de reformulación sigue siendo obligatoria para completar la comparación predeclarada con los modelos candidatos, aun cuando este resultado ya acota por arriba la mejora que esos modelos podrán demostrar en  $K = 1$ .

### 3.7.6. El *gap* estado $\times K$

Las secciones anteriores describen la decisión de dispositivo para el despacho siguiente, es decir, para  $K = 1$ . La formulación original del estado `cpu_ready` (sección 3.7.2) fija la regla “permanecer en CPU” porque CPU es óptima en los 68 grupos de ese estado a  $K = 1$ . Esa regla es correcta solo para el despacho siguiente: al extender la comparación a un horizonte  $K$  de despachos, mediante la misma formulación  $EDP_{\text{total}}(d, K)$  de la sección 3.7.1 — pagando el costo de arranque únicamente cuando el dispositivo destino no está ya inicializado en ese estado —, la decisión correcta deja de ser constante.

Desde `cpu_ready`, 22 de las 68 configuraciones migran a GPU en algún  $K$  dentro del horizonte analizado; los primeros cruces son *Cholesky* en  $N = 4096$  en  $K = 2$ , *GEMM* en  $N = 4096$  en  $K = 3$  y la transformada de Fourier en  $N = 4096$  en  $K = 5$ . En sentido inverso, desde `gpu_ready` 46 de las 68 configuraciones migran a CPU en algún  $K$ ; los primeros cruces inversos son *stencil* en  $N = 3072$  en  $K = 2$ , combinación lineal de vectores en  $N = 31\,623$  en  $K = 3$  y matriz–vector dispersa en  $N = 1\,000\,000$  en  $K = 3$ . Los tres estados de recurso convergen al mismo conjunto asintótico de 22 configuraciones en las que GPU gana en la región caliente ( $22 + 46 = 68$ ), que es exactamente el conjunto identificado en la sección 3.7.1; esta coincidencia es una comprobación de consistencia interna del mapa de amortización, no un hallazgo independiente.

La política de dispositivo correcta para los tres estados es entonces

$$\text{decisión}(\text{estado}, K) = \underset{d}{\operatorname{argmin}} \ EDP_{\text{total}}(d, K \mid \text{estado}),$$

y no una regla constante por estado. El resultado de la sección 3.7.5 — que una tabla de umbrales iguala al oráculo dentro del piso de ruido — se conserva sin cambios, pero acotado explícitamente a  $K = 1$ : ninguna de las ocho baselines evaluadas allí plantea la pregunta de horizonte, de modo que ese resultado no puede extenderse a  $K > 1$  sin baselines adicionales que resuelvan explícitamente  $\underset{d}{\operatorname{argmin}} \ EDP_{\text{total}}(d, K)$  y que se comparen contra la política de permanecer en el dispositivo preparado para todo el horizonte.

### 3.7.7. El comportamiento de movimiento de datos

La sección 3.4.1 mostró que, en la región caliente, la ventaja de la GPU se concentra en las tres operaciones de mayor intensidad operacional y desaparece — e incluso se invierte — en las tres de menor intensidad. Para entender por qué, se calculó un rendimiento implícito de movimiento de datos en GPU: los bytes lógicos por despacho (Tabla 2.10) divididos entre el tiempo por despacho en la región caliente, para cada operación.

Ese rendimiento implícito converge a un rango estrecho de entre 8 y 10 GB/s en las seis operaciones del catálogo, con independencia de cuán distinta sea su intensidad aritmética: combinación lineal de vectores entre 9,4 y 9,5 GB/s, matriz–vector dispersa entre 9,9 y 10,1 GB/s, *stencil* entre 9,2 y 9,4 GB/s, multiplicación de matrices densa entre 8,2 y 8,6 GB/s, *Cholesky* entre 7,9 y 8,7 GB/s, y la transformada de Fourier entre 9,2 y 9,3 GB/s.

Esta convergencia es **consistente con** un cuello de botella de movimiento de datos entre anfitrión y acelerador: un límite dominado por cómputo no produciría cifras tan parecidas entre operaciones cuya intensidad aritmética difiere en órdenes de magnitud, porque cada una satura un recurso de cómputo distinto a una tasa distinta. Sin embargo, esta observación debe leerse con dos salvedades que no se relajan. Primera, el número reportado es una **cota inferior** del ancho de banda real, no una medición de él: el tiempo por despacho incluye el kernel de cómputo y el *overhead* de lanzamiento además de la transferencia, de modo que el ancho de banda efectivo de la transferencia aislada es, como mínimo, igual o mayor a esta cifra. Segunda, este proyecto no realizó ninguna medición aislada del bus PCIe entre anfitrión y acelerador, de modo que no se reporta — ni se compara contra — ninguna cifra de ancho de banda PCIe de la plataforma. La convergencia observada no constituye una demostración causal del mecanismo; es una observación consistente con él, y se reporta con ese alcance.

Esta interpretación explica, sin necesidad de invocar una causa distinta por operación, por qué combinación lineal de vectores, matriz–vector dispersa y *stencil* favorecen a la GPU en los tamaños pequeños del catálogo y dejan de hacerlo en los tamaños grandes (sección 3.4.1): si el tiempo por despacho está acotado por un recurso de transferencia que no escala con la intensidad de cómputo, entonces cualquier crecimiento del tamaño del problema en una operación de baja intensidad aritmética añade tiempo de transferencia sin que el cómputo, ya subordinado a ese cuello de botella, pueda compensarlo — lo opuesto de lo que la intuición basada solo en el trabajo aritmético total sugeriría.

# Capítulo 4

## Discusión

Los resultados de esta fase sostienen dos tipos de contribución distintos: uno metodológico, sobre cómo verificar con rigor cada mecanismo de un instrumento de caracterización antes de confiar en él; y uno empírico, sobre lo que el catálogo dual permite y no permite demostrar acerca de la asignación entre CPU y acelerador.

### 4.1. Sobre la unidad de decisión y la variable objetivo

La formulación adoptada en la sección 2.4.1 — decidir sobre una llamada a una operación, y predecir directamente la configuración de mínimo EDP en lugar de una etiqueta de régimen — merece una justificación explícita, porque a primera vista podría leerse como un debilitamiento del segundo objetivo específico. No lo es, y la distinción importa.

Ese objetivo exige que la identificación del comportamiento de la carga ocurra en tiempo de ejecución y con baja latencia de inferencia, y habla de “las fases de ejecución de las aplicaciones”. No prescribe una unidad temporal, y la formulación adoptada es la lectura más literal de su enunciado: una fase de una aplicación HPC es una de las operaciones que la componen. Las dos exigencias que el objetivo sí formula se conservan íntegras, y el tercer objetivo específico — que pide políticas *proactivas* — se satisface de forma más estricta que con una política que observe la ejecución en curso, dado que la decisión se emite antes de que la operación comience.

El cambio de variable objetivo admite una lectura análoga. La distinción *compute-bound/memory-bound* no era el fin del sistema sino un medio: un proxy de la pregunta “qué punto operativo conviene”. Un proxy es defendible mientras su relación con la variable de interés sea estable, y esa relación no lo es en este caso: el techo de cómputo que fija el



punto de inflexión del modelo Roofline depende de la frecuencia de operación, de modo que el umbral que define la etiqueta se mueve con la propia variable que se pretende decidir. Predecir el EDP directamente elimina esa dependencia circular y alinea el objetivo de entrenamiento con el criterio de evaluación del cuarto objetivo específico. La etiqueta de régimen conserva su valor como herramienta de *caracterización* — describe por qué una operación responde como responde al escalado de frecuencia — pero no como variable a predecir.

Conviene subrayar que esta unidad no es una particularidad de este trabajo. Los tres sistemas publicados con los que se compara deciden en la misma escala — la aplicación [12], el programa completo tras haber probado y descartado explícitamente el ajuste función por función [5], y el trabajo completo en producción [3] —, y que tres grupos independientes hayan convergido en ella sugiere que responde a la granularidad en que el fenómeno resulta efectivamente predecible, y no a una restricción de sus montajes experimentales.

# Capítulo 5

## Conclusiones

Este trabajo completó la construcción y validación empírica del instrumento de caracterización CPU–GPU y determinó, sobre evidencia propia, la formulación que debe adoptar el sistema de decisión de las fases siguientes: decidir conjuntamente dispositivo y frecuencia, sobre la unidad de una llamada a una operación, prediciendo directamente la configuración de mínimo EDP en lugar de una etiqueta de régimen intermedia cuya estabilidad depende del punto de operación en el que se mide. Esa formulación fija la unidad de decisión y la variable objetivo descritas en la sección 2.4.1.

En consecuencia, el aporte propio de esta fase respecto de las siguientes no es únicamente el conjunto de datos ni el instrumento, sino esa formulación fundamentada, que se sostiene sobre el catálogo dual y no sobre una etiqueta de régimen intermedia.

Respecto del segundo objetivo específico, el trabajo se encuentra parcialmente ejecutado y el alcance de lo conseguido debe declararse con precisión. El catálogo dual está construido y verificado, la campaña de adquisición de su eje de CPU concluyó íntegramente (sección 3.1), y el procedimiento completo de construcción del conjunto de nivel 2, de entrenamiento y de validación anidada está implementado y auditado de extremo a extremo (sección 3.6). Lo que no está ejecutado es la comparación de modelos que responde a la pregunta de investigación, porque requiere el eje del acelerador: el eje de CPU en aislamiento contiene solo uno de los dos grados de libertad del problema, y la compuerta de validez de la sección 2.4.9 lo hace explícito en lugar de dejar que una tabla de métricas se lea como si respondiera a algo que no responde. Los objetivos específicos tercero y cuarto — implementación del agente y evaluación de EDP — corresponden a las Fases 3 y 4 y no se presentan como ejecutados.

El aporte metodológico central de esta fase, más allá de los datos concretos recolectados,

es haber demostrado — y no solo enunciado — que ningún mecanismo de este instrumento se dio por funcional sin verificación directa contra el estado real del hardware: ni la escritura de frecuencia de CPU y GPU, ni la disponibilidad de los contadores de rendimiento, ni la frontera de medición de energía, ni el efecto del propio muestreo sobre el estado del dispositivo observado. En varias ocasiones documentadas en este trabajo, esa verificación expuso una falla real que una confianza no verificada en la documentación del fabricante, en una medición previa ya dada por válida, o en el diseño original del instrumento, habría dejado pasar silenciosamente hacia el conjunto de datos.

## 5.1. Limitaciones

Este trabajo reconoce las siguientes limitaciones. Primera, el presupuesto simultáneo de contadores de PMU depende del estado de mecanismos del sistema operativo ajenos al proyecto; se redujo a nueve eventos para convivir con una reserva encontrada durante el desarrollo, pero una reserva futura adicional exigiría repetir la verificación. Segunda, el catálogo y los mecanismos de actuación de frecuencia están restringidos a la plataforma de la Tabla 2.1; no se reclama portabilidad de esos mecanismos sin repetir sus verificaciones bajo carga en cada estado de plataforma.

Tercera, la unidad de decisión acota lo que este trabajo puede demostrar: el sistema decide entre fases y no dentro de ellas. Una aplicación cuyo comportamiento cambiara sustancialmente a lo largo de una misma operación quedaría fuera del alcance de esta formulación, y su tratamiento exigiría un mecanismo de grano más fino, con el costo de conmutación que ello implica. Este trabajo no verifica empíricamente la homogeneidad de comportamiento dentro de una misma operación; es un supuesto del diseño y no una propiedad general de las aplicaciones HPC que se dé por demostrada.

Cuarta, la temperatura de paquete no está conectada a un *gate* bloqueante de sensor.

Quinta, el producto energía-retardo sobre el que se construye la etiqueta del selector mide exclusivamente el despacho de la operación y **no incluye el costo de actuar la frecuencia** entre decisiones sucesivas. Esa separación es deliberada — el costo de conmutación depende del estado previo y no de la operación, por lo que mezclarlo con el costo del núcleo de cómputo produciría una cantidad que no describe ninguno de los dos —, pero implica que toda ganancia derivada de este objetivo debe leerse como una **cota superior** y no como ganancia neta. El costo de actuación se mide por separado en la Fase 4, y es uno de los dos términos del umbral de amortización allí determinado.

Sexta — limitación ya resuelta con el catálogo dual completo, se conserva la redacción original seguida del resultado por transparencia del proceso. La etiqueta del vector de entrada con sondeo resultó, sobre el eje de CPU en aislamiento, prácticamente degenerada: una sola acción óptima en más de la mitad de las configuraciones y un margen mediano del 0.73 %, por debajo del piso de ruido de medición (sección 3.3). Ese resultado no invalidaba la estrategia — su veredicto podía cambiar al incorporar el eje del acelerador, donde el margen entre dispositivos es de otro orden —, pero sí impedía extraer conclusión alguna sobre familias de modelos a partir de ese subconjunto. Con el catálogo dual completo (sección 3.4) la pregunta queda cerrada: el veredicto **no cambia**. La etiqueta con sondeo sigue siendo `pipeline_smoke_only` (margen mediano 1.03 % sobre las 136 combinaciones de estado y configuración), pese a que la GPU sí gana en 43 de esos 136 grupos — el margen entre sus mejores candidatos es demasiado estrecho para servir de etiqueta de entrenamiento, no la ausencia de estructura lo que lo impide.

Séptima, el vector de entrada sin ejecución previa construye tanto sus candidatos como su etiqueta sobre la región fría, que en las configuraciones más pequeñas es más breve que el intervalo de muestreo. Un 5.9 % de sus acciones descansa sobre estimaciones de baja resolución, y excluirlas cambiaría la acción óptima en 2 de las 68 decisiones (sección 3.5). La magnitud es acotada y está cuantificada, pero no es nula, y las conclusiones sobre configuraciones pequeñas de la operación de combinación lineal de vectores deben leerse con esa reserva.

## 5.2. Trabajo Futuro

Las dos primeras tareas de esta sección, cerrar la adquisición del eje del acelerador y repetir el procedimiento de modelado sobre el conjunto completo, ya se ejecutaron; se conserva su redacción original seguida del resultado, por la misma razón de transparencia que otras limitaciones ya resueltas de este capítulo.

Primero — ya completado. El eje de CPU quedó cerrado con la totalidad de sus 1 632 combinaciones aceptadas bajo el contrato temporal de la sección 2.2.5. El eje del acelerador, cuyo producto cartesiano de frecuencias de ambos dispositivos lo hace considerablemente más costoso, se ejecutó en sesiones sucesivas sobre un mismo directorio de salida, reanudando desde las combinaciones ya aceptadas: 6 528 de 6 528 combinaciones aceptadas, sin rechazos sin resolver.

Segundo — ya ejecutado, con un hallazgo de implementación real en el camino. Reconstruir el conjunto de nivel 2 en modo final expuso que el mecanismo de reanudación entre

sesiones (registrar por separado lo aceptado en la sesión actual y lo ya aceptado en sesiones previas) no se propagaba al constructor del dataset: una sesión final que no acepta combinaciones nuevas — todo ya estaba hecho — dejaba la lista de corridas nuevas vacía, y el constructor leía solo esa lista, produciendo un conjunto vacío en silencio pese a que los 6 528 archivos eran válidos en disco. Corregido antes de construir el conjunto reportado en la sección 3.4, con pruebas de regresión específicas para el caso que lo expuso. Con él, la compuerta de la sección 2.4.9 se recalculó por primera vez sobre el catálogo íntegro: la estrategia sin ejecución previa mantiene su veredicto interpretable sin que una sola etiqueta cambie al incorporar la GPU, y la estrategia con sondeo confirma su veredicto de validación de tubería. Ambos resultados son publicables — que una familia supere a las líneas base, o que ninguna lo haga y la mejor política sea una constante bien elegida — y la búsqueda de familias sobre la estrategia sin ejecución previa, la única con veredicto interpretable, ya está en ejecución al momento de escribir esta sección.

Tercero, implementar el agente y evaluarlo. La comparación de cuatro escenarios descrita en la sección 2.6 — ejecución íntegra en CPU, íntegra en el acelerador, guiada por el selector, y guiada por un oráculo con conocimiento posterior — es la que permite atribuir un resultado insuficiente a la ausencia de margen en la plataforma o a la incapacidad del modelo para capturarlo, distinción que sin el cuarto escenario resultaría ambigua. La medición del umbral de amortización del costo de decisión — inferencia más actuación de frecuencia, este último excluido deliberadamente del objetivo de entrenamiento según la quinta limitación — responde directamente a la cláusula de la pregunta de investigación relativa al *overhead* de la inferencia.

Más allá del alcance comprometido, cuatro extensiones quedan abiertas. La primera es la política de repeticiones adaptativa: la evidencia recogida sobre convergencia del coeficiente de variación indica que tres repeticiones son un mínimo operativo defendible para la mayoría de las operaciones del catálogo, pero no para todas, y una política que escale las repeticiones únicamente donde el margen entre las dos mejores configuraciones sea estrecho — que es exactamente la condición que la anotación de óptimo incierto de la sección 2.4.2 ya identifica — aprovecharía mejor el tiempo de cómputo que un valor uniforme.

La segunda es la extensión del criterio de calidad por ventana al eje del acelerador: el instrumento registra por muestra el reloj y la temperatura del dispositivo, pero no los emplea todavía como compuerta automática, de modo que una eventual limitación térmica durante una sesión prolongada se detectaría hoy mediante análisis posterior y no durante la propia campaña.

La tercera es una variante del sondeo que la formulación actual no contempla. En el diseño vigente, la ejecución que produce la telemetría de sondeo es la primera ejecución real de la operación, con su resultado íntegro; una alternativa sería ejecutar una réplica reducida de la operación con el único fin de observarla, antes de comprometer la ejecución real. Esa variante permitiría sondear sin arriesgar una decisión sobre trabajo útil, pero introduce un costo que la formulación actual no paga y que debería contabilizarse en el mismo umbral de amortización, junto con la pregunta de si conviene pagarlo siempre o solo cuando el margen entre las dos mejores acciones sea estrecho. No se adopta aquí porque su diseño no está resuelto, y se registra para que la omisión sea deliberada y no un olvido.

La cuarta es una campaña suplementaria de tamaños mayores, dirigida específicamente por el hallazgo de la sección 3.4.1. El costo de arranque de CUDA (0.49–0.81 s en la región fría, prácticamente constante entre operaciones) es lo que impide que la GPU gane en la estrategia sin ejecución previa a ningún tamaño del catálogo actual; el mismo análisis muestra que, una vez amortizado ese costo, la ventaja de la GPU en las operaciones *compute-bound* llega a superar en un orden de magnitud a la CPU. Ampliar el catálogo con tamaños mayores en *Cholesky*, multiplicación de matrices y transformada de Fourier — las tres operaciones donde la región caliente favorece al acelerador — desplazaría el punto en el que el cómputo puro supera el arranque también en la región fría, y permitiría observar directamente en qué tamaño ese cruce ocurre para cada operación, en lugar de inferirlo de la comparación entre regiones. La extensión es aditiva: exige nuevas entradas de catálogo con el mismo `config_id` compartido entre su variante CPU y GPU (sección 2.4.3), una campaña dirigida solo a esas configuraciones, y una reconstrucción del conjunto de nivel 2 que la incorpore sin repetir ninguna de las 8 160 corridas ya aceptadas.

# Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>

# Bibliografía

- [1] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [2] Oscar Antepara, Leonid Oliker, and Samuel Williams. Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1205–1216, 2025. doi: 10.1145/3731599.3767497.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [4] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages*, 1 (OOPSLA):1–27, 2017. doi: 10.1145/3133876.
- [5] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccone. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.
- [6] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops ’25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
- [7] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.



- [8] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous java performance evaluation. In *Proc. 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA '07)*, pages 57–76, Montreal, QC, Canada, 2007. doi: 10.1145/1297027.1297033.
- [9] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.
- [10] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
- [11] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
- [12] J. Guerreiro, N. Roma, P. Tomás, F. Pratas, L. S. G. Carvalho, and G. Gaydadjiev. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [13] R. Hebbbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.
- [14] IBM. What is logistic regression? IBM Think Topics, 2026. URL <https://www.ibm.com/think/topics/logistic-regression>. Accessed: Apr. 3, 2026.
- [15] IBM. What is random forest? IBM Think Topics, 2026. URL <https://www.ibm.com/think/topics/random-forest>. Accessed: Apr. 3, 2026.
- [16] IBM. What is XGBoost? IBM Think Topics, 2026. URL <https://www.ibm.com/think/topics/xgboost>. Accessed: Apr. 3, 2026.
- [17] Intel Corporation. *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore Performance Monitoring Reference Manual*, May 2021. URL <https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>. Revision 1.0.
- [18] Intel Corporation. perfmon: Performance monitoring events for intel architecture processors (evento fp\_arith\_inst\_retired, familia ice lake), 2026. URL <https://github.com/intel/perfmon>. Accessed: Aug. 10, 2026.

- [19] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [20] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.
- [21] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2\_9.
- [22] Linux man-pages project. clock\_gettime(3) — linux manual page, 2026. URL [https://man7.org/linux/man-pages/man3/clock\\_gettime.3.html](https://man7.org/linux/man-pages/man3/clock_gettime.3.html). Accessed: Aug. 29, 2026.
- [23] Linux man-pages project. perf\_event\_open(2) — linux manual page, 2026. URL [https://man7.org/linux/man-pages/man2/perf\\_event\\_open.2.html](https://man7.org/linux/man-pages/man2/perf_event_open.2.html). Accessed: Aug. 29, 2026.
- [24] Linux perf project. perf-stat(1) — linux manual page, 2026. URL <https://man7.org/linux/man-pages/man1/perf-stat.1.html>. Accessed: Aug. 29, 2026.
- [25] L. Littman and T. Deakin. Classifying performance bounds using machine learning. In *Proc. SC25: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025.
- [26] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [27] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. Accessed: Apr. 3, 2025.
- [28] National Energy Research Scientific Computing Center (NERSC). Roofline performance model. NERSC Documentation, 2026. URL <https://docs.nersc.gov/tools/performance/roofline/>. Accessed: Aug. 7, 2026.

- [29] NVIDIA. Nvidia management library (nvml). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/deploy/nvml-api/index.html>. Accessed: Apr. 3, 2026.
- [30] NVIDIA. Nsight compute. NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/>. Accessed: Aug. 8, 2026.
- [31] D. Pandey, K. Niwaria, and B. Chourasia. Machine learning algorithms: A review. *International Journal of Machine Learning and Networked Applications*, 6(2):916–922, 2019. doi: 10.21275/ART20203995.
- [32] ScienceDirect. Computational kernel. Computer Science Topics, 2026. URL <https://www.sciencedirect.com/topics/computer-science/computational-kernel>. Accessed: Apr. 3, 2026.
- [33] S.-K. Shekafteh et al. Metric selection for GPU kernel classification. *ACM Transactions on Architecture and Code Optimization*, 15(4):1–27, 2019. doi: 10.1145/3295690.
- [34] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [35] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [36] J. Treibig, G. Hager, and G. Wellein. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *Proc. 39th International Conference on Parallel Processing Workshops (ICPPW)*, pages 207–216, San Diego, CA, USA, 2010. doi: 10.1109/ICPPW.2010.38.
- [37] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.
- [38] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.

- [39] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.
- [40] V. M. Weaver and S. A. McKee. Can hardware performance counters be trusted? In *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, Seattle, WA, USA, 2008. doi: 10.1109/IISWC.2008.4636099.
- [41] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.
- [42] Z. Zhang, Z. Wu, J. Liu, and L. Mottola. SparseDVFS: Sparse-aware DVFS for energy-efficient edge inference. *arXiv preprint arXiv:2603.21908*, 2026.